CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Faculty of Technology & Engineering | Department of Information Technology

ITUE301: Advanced Web Development Frameworks (AWDF) - AY 2026-27

INDEX

Student Name: Krunal Vaghamshi
Student ID: D25IT137

Course Code: ITUE301
Batch / Class: 5IT-2(C2)

Sr. No.	Practical Name	Date
1	Building Component-Based UI using React, JSX & Props	08/07/2026
2	State Management, Event Handling, Controlled Components & Navigation	15/07/2026
3	Asynchronous Data Fetching, Lifecycle Hooks & REST API Consumption	20/07/2026
4	RESTful API Development using Node.js, Express & In-Memory CRUD	27/07/2026
5	Database Integration with MongoDB & Mongoose Schema Validation	03/08/2026
6	Full-Stack Integration (Connecting React Frontend to Express & MongoDB)	14/08/2026
7	User Authentication & Authorization with JWT and Middleware Pipeline	17/08/2026
8	Performance Optimization & Lazy Loading in React (React.lazy & Suspense)	18/09/2026
9	In-Memory Caching and Query Optimization using Node-Cache	18/09/2026
10	Asynchronous Processing with Event-Driven Architecture (EventEmitter)	30/09/2026
11	Application Containerization with Docker and Docker Compose Orchestration	07/10/2026
12	Automated CI/CD Pipeline Implementation using GitHub Actions	27/10/2026
13	External AI Service Integration (OpenAI/Gemini) & Graceful Degradation	27/10/2026

D25IT137

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

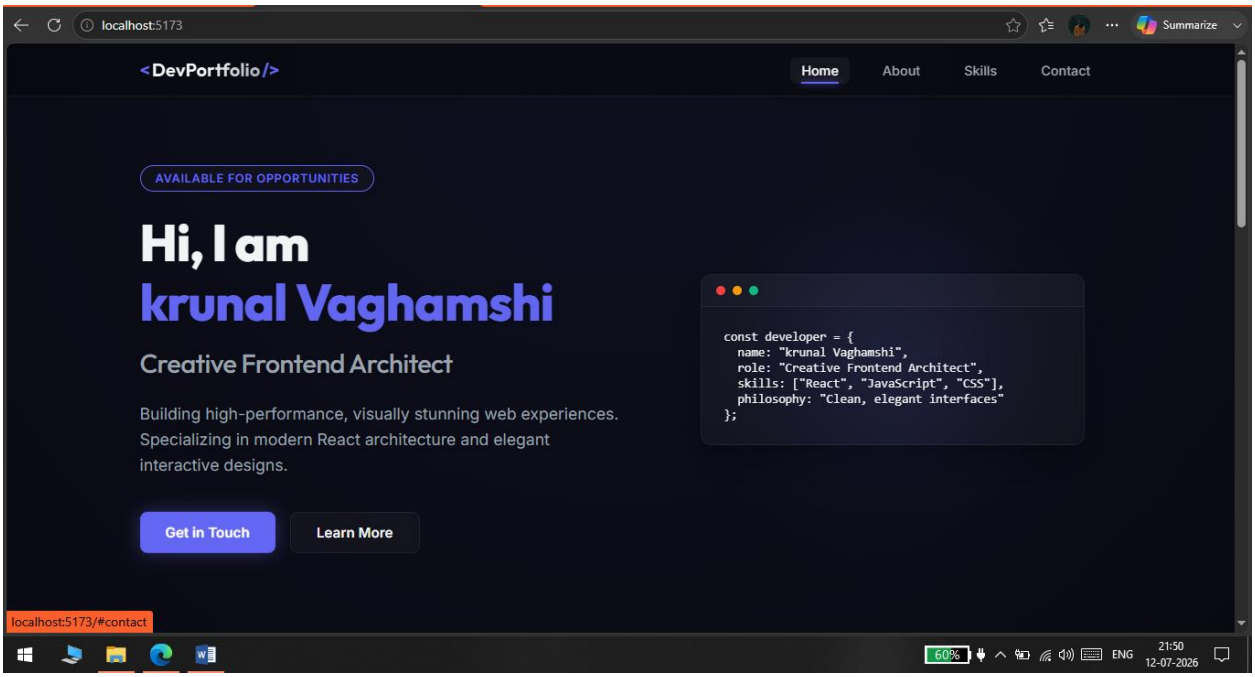
Chandubhai S Patel Institute of Technology

DEPSTAR

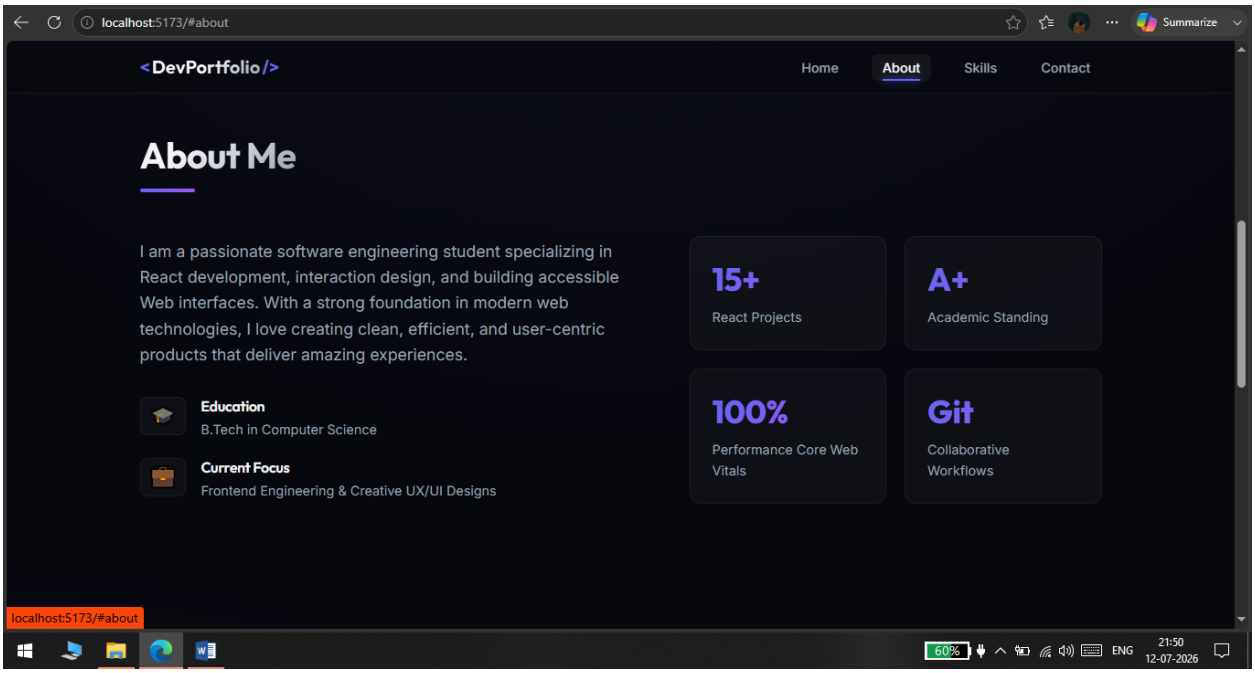
Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

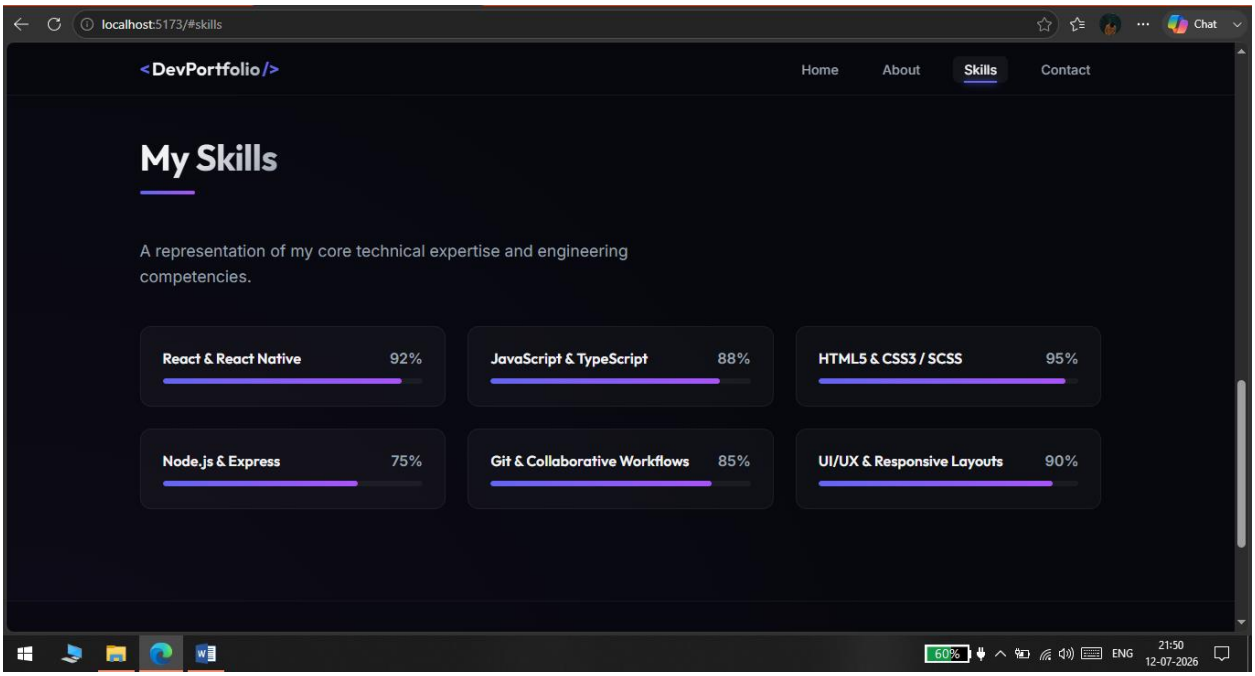
Problem Definition: - Create a React application using Vite for a student portfolio page. The application must include at least 4 reusable components: Header, About, Skills, and Footer. Each component must be independently structured and composed into a single-page layout. No logic or JSX should be duplicated across components. Props must be used to pass at least one piece of data into a minimum of 2 components.

Screenshots:-

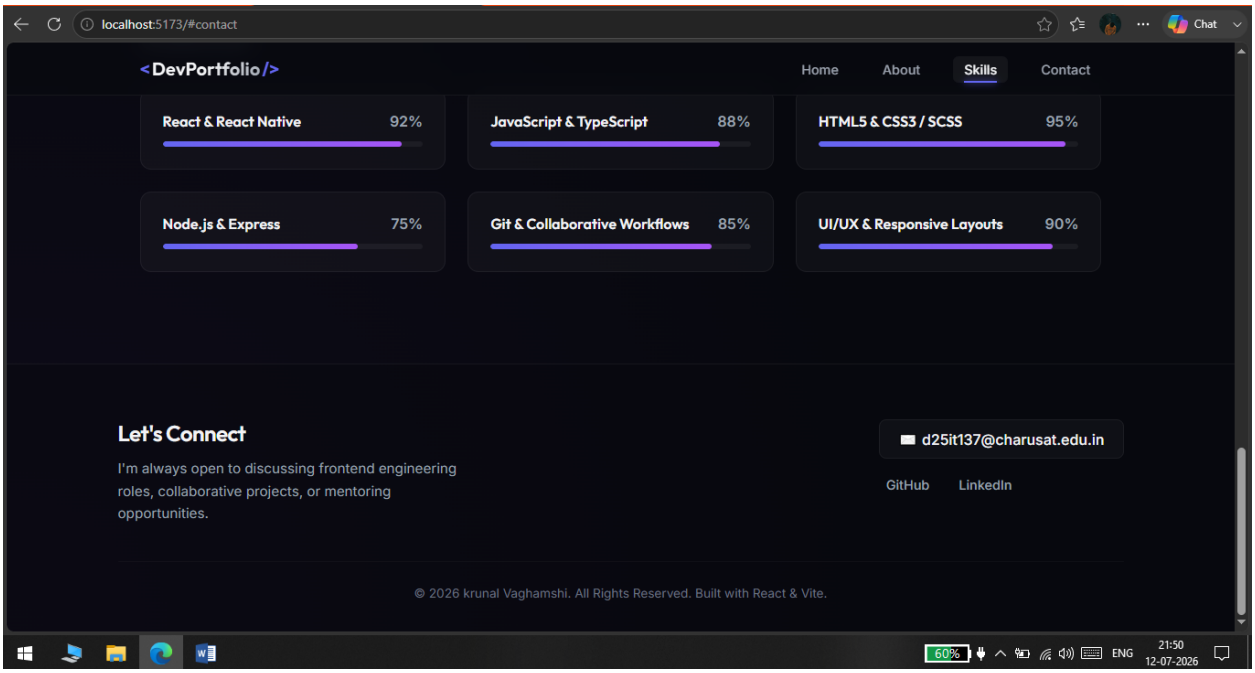


D25IT137


localhost:5173/#about
Summarize
<DevPortfolio/>
Home
About
Skills
Contact
About Me
I am a passionate software engineering student specializing in React development, interaction design, and building accessible Web interfaces. With a strong foundation in modern web technologies, I love creating clean, efficient, and user-centric products that deliver amazing experiences.
Education
B.Tech in Computer Science
Current Focus
Frontend Engineering & Creative UX/UI Designs
15+
React Projects
A+
Academic Standing
100%
Performance Core Web Vitals
Git
Collaborative Workflows
localhost:5173/#about
60%
ENG
21:50
12-07-2026


localhost:5173/#skills
Chat
<DevPortfolio/>
Home
About
Skills
Contact
My Skills
A representation of my core technical expertise and engineering competencies.
React & React Native 92%
JavaScript & TypeScript 88%
HTML5 & CSS3 / SCSS 95%
Node.js & Express 75%
Git & Collaborative Workflows 85%
UI/UX & Responsive Layouts 90%
60%
ENG
21:50
12-07-2026

D25IT137



Github Link:- [AWDF/p1 at main · krunal9116/AWDF]

D25IT137

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

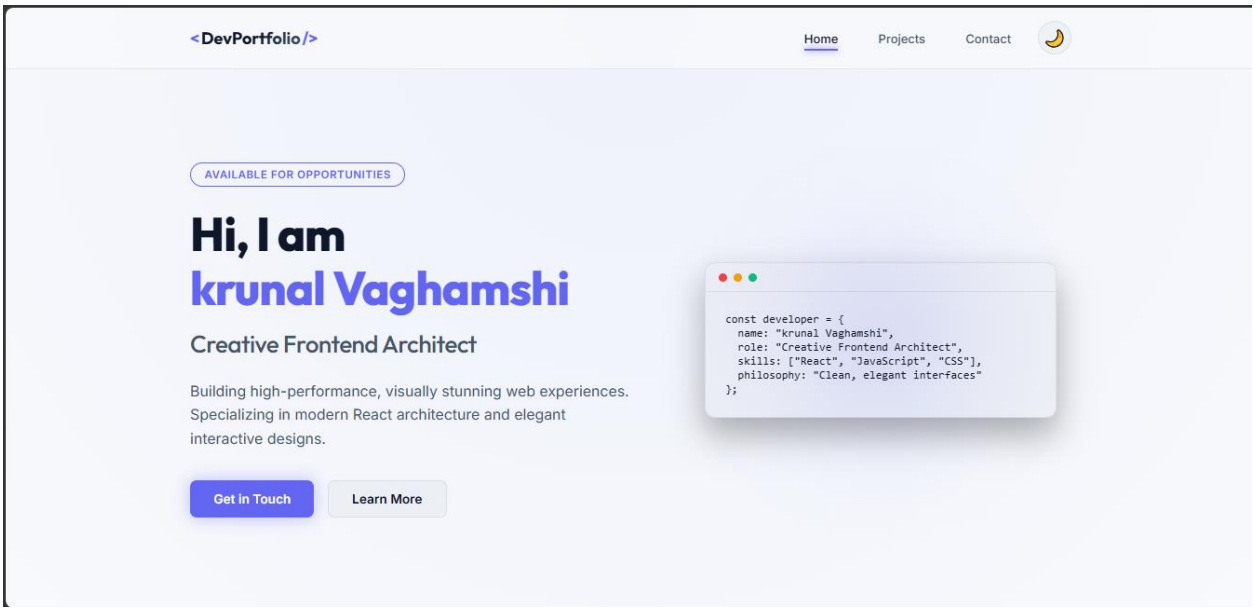
Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-2

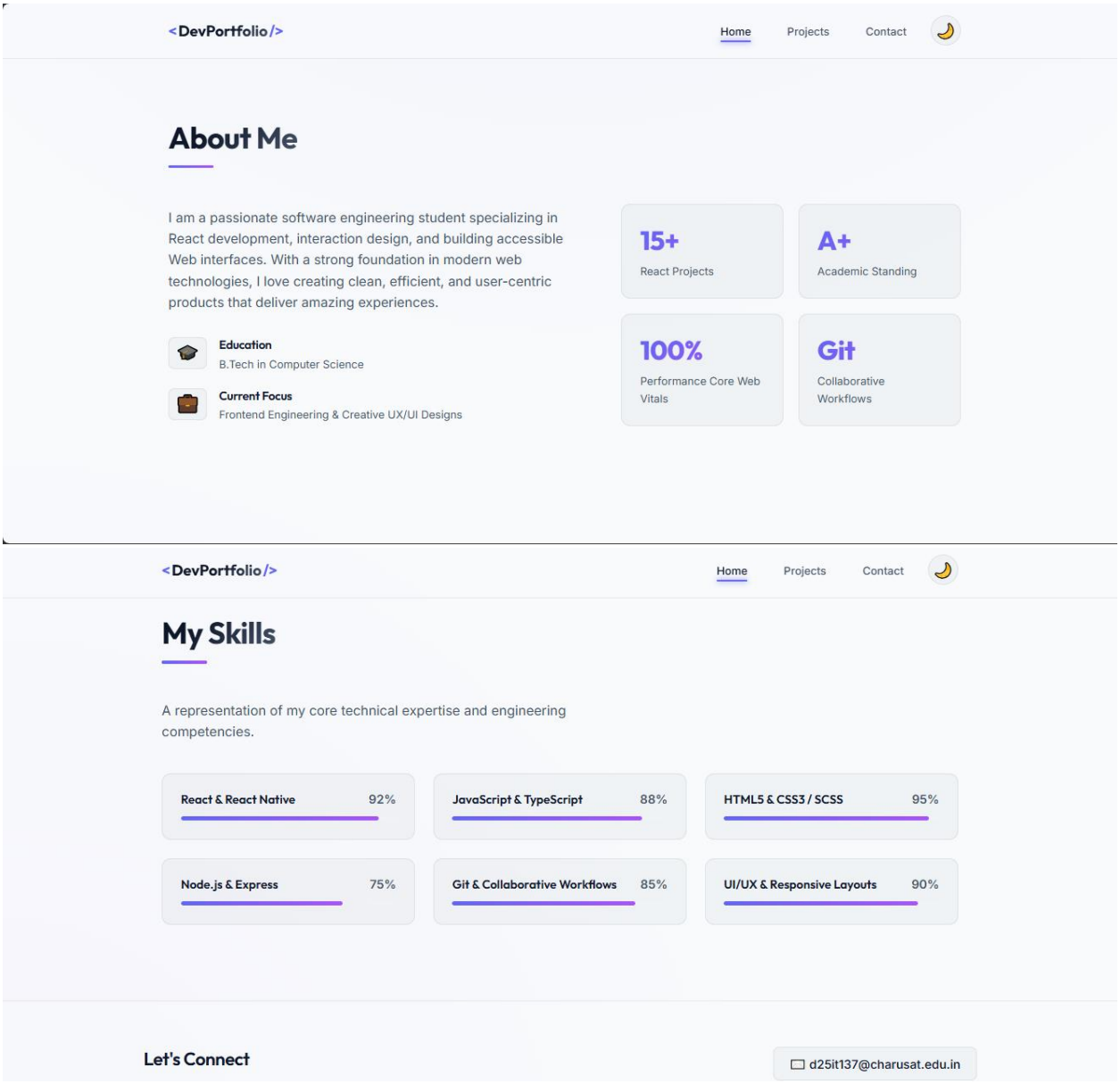
Problem Definition: -

- Extend the portfolio application from Practical 1 by adding React Router.
- Implement a navigation bar with at least 3 routes: Home, Projects, and Contact.
- Each route must render a distinct component without a full page reload.
- Add at least 2 useState variables used meaningfully one for toggling UI visibility and one for managing a form input on the Contact page.
- The Contact page must include a controlled input that captures and displays user input in real time.

Screenshots:-Home-Page:

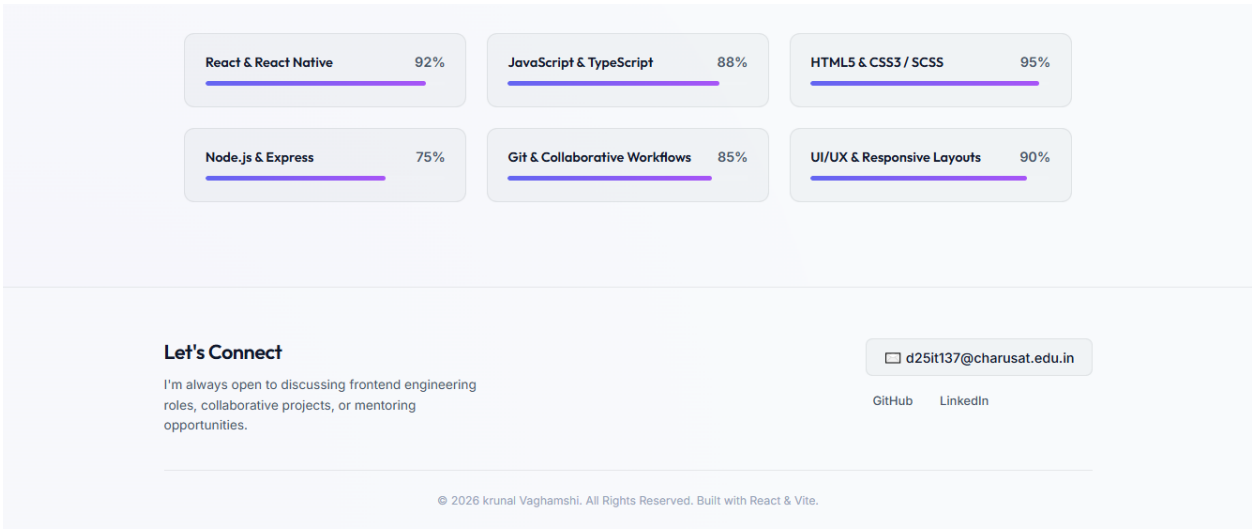


D25IT137

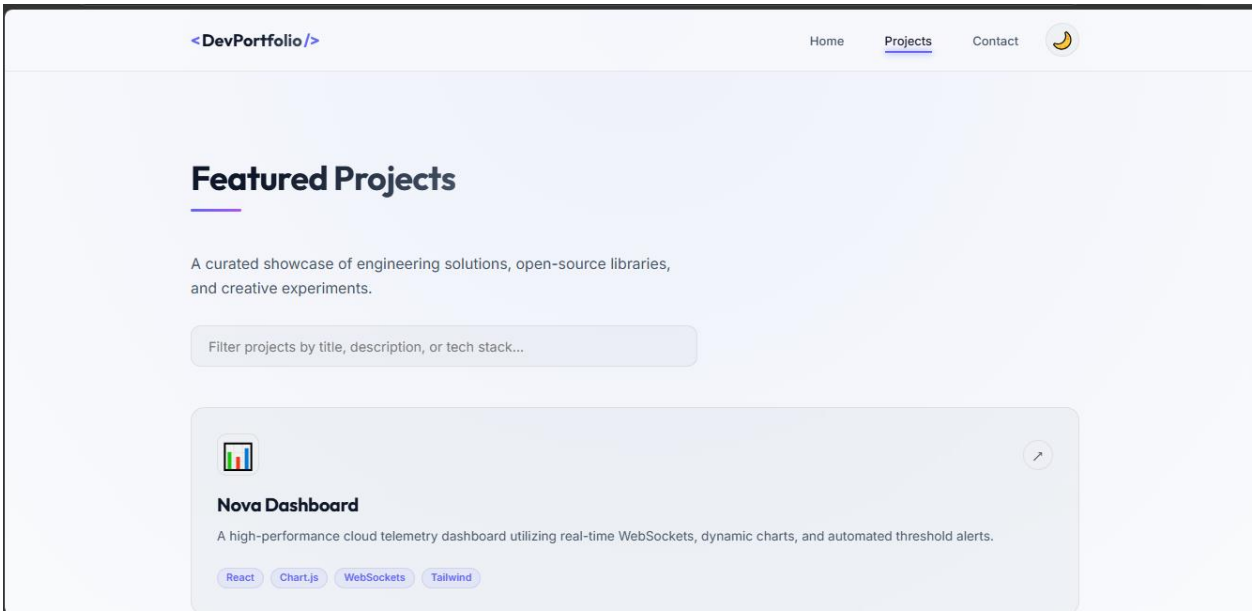

<DevPortfolio/>
Home
Projects
Contact
About Me
I am a passionate software engineering student specializing in React development, interaction design, and building accessible Web interfaces. With a strong foundation in modern web technologies, I love creating clean, efficient, and user-centric products that deliver amazing experiences.
Education
B.Tech in Computer Science
Current Focus
Frontend Engineering & Creative UX/UI Designs
15+
React Projects
A+
Academic Standing
100%
Performance Core Web Vitals
Git
Collaborative Workflows

<DevPortfolio/>
Home
Projects
Contact
My Skills
A representation of my core technical expertise and engineering competencies.
React & React Native 92%
JavaScript & TypeScript 88%
HTML5 & CSS3 / SCSS 95%
Node.js & Express 75%
Git & Collaborative Workflows 85%
UI/UX & Responsive Layouts 90%
Let's Connect
d25it137@charusat.edu.in

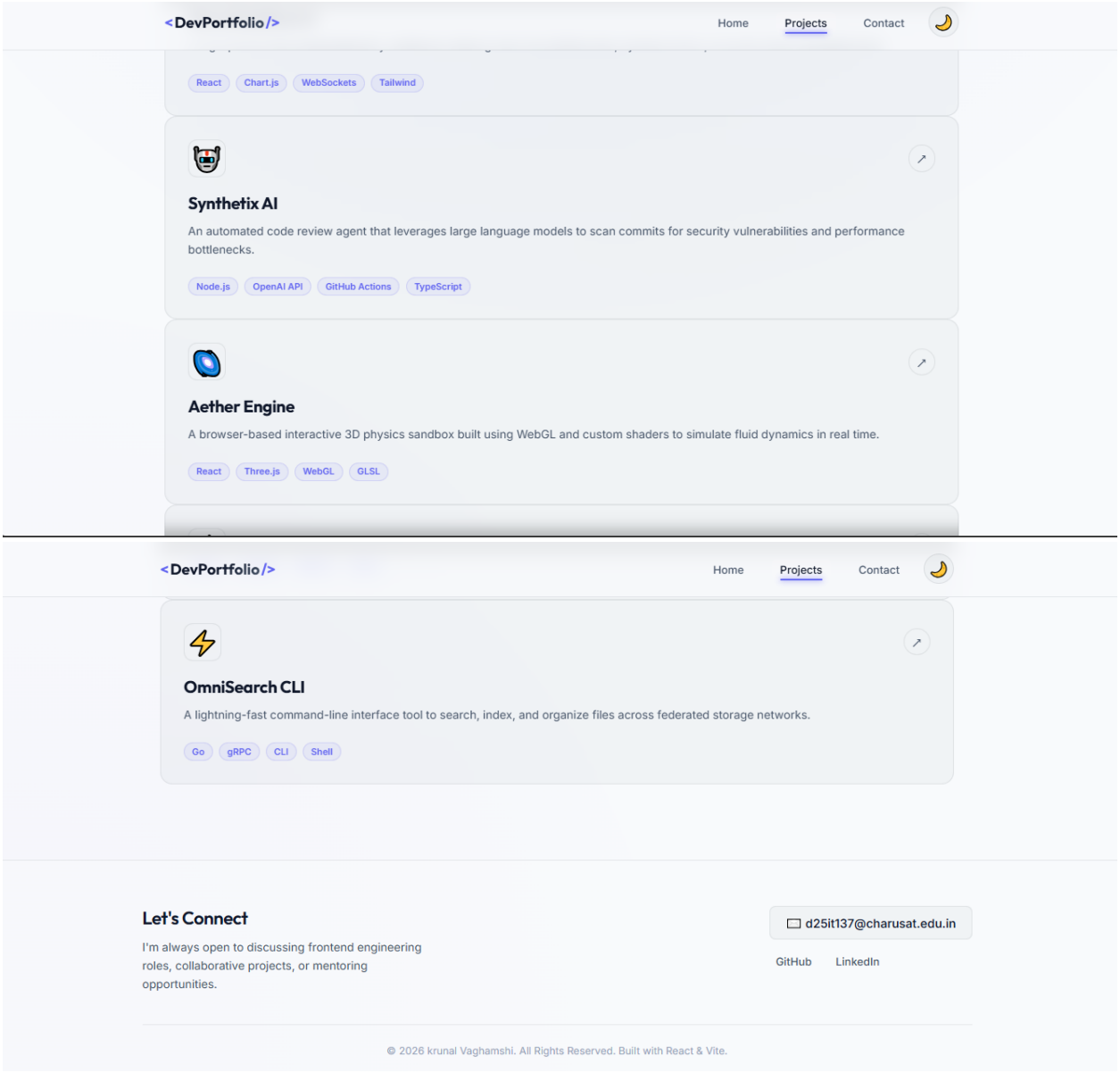
D25IT137



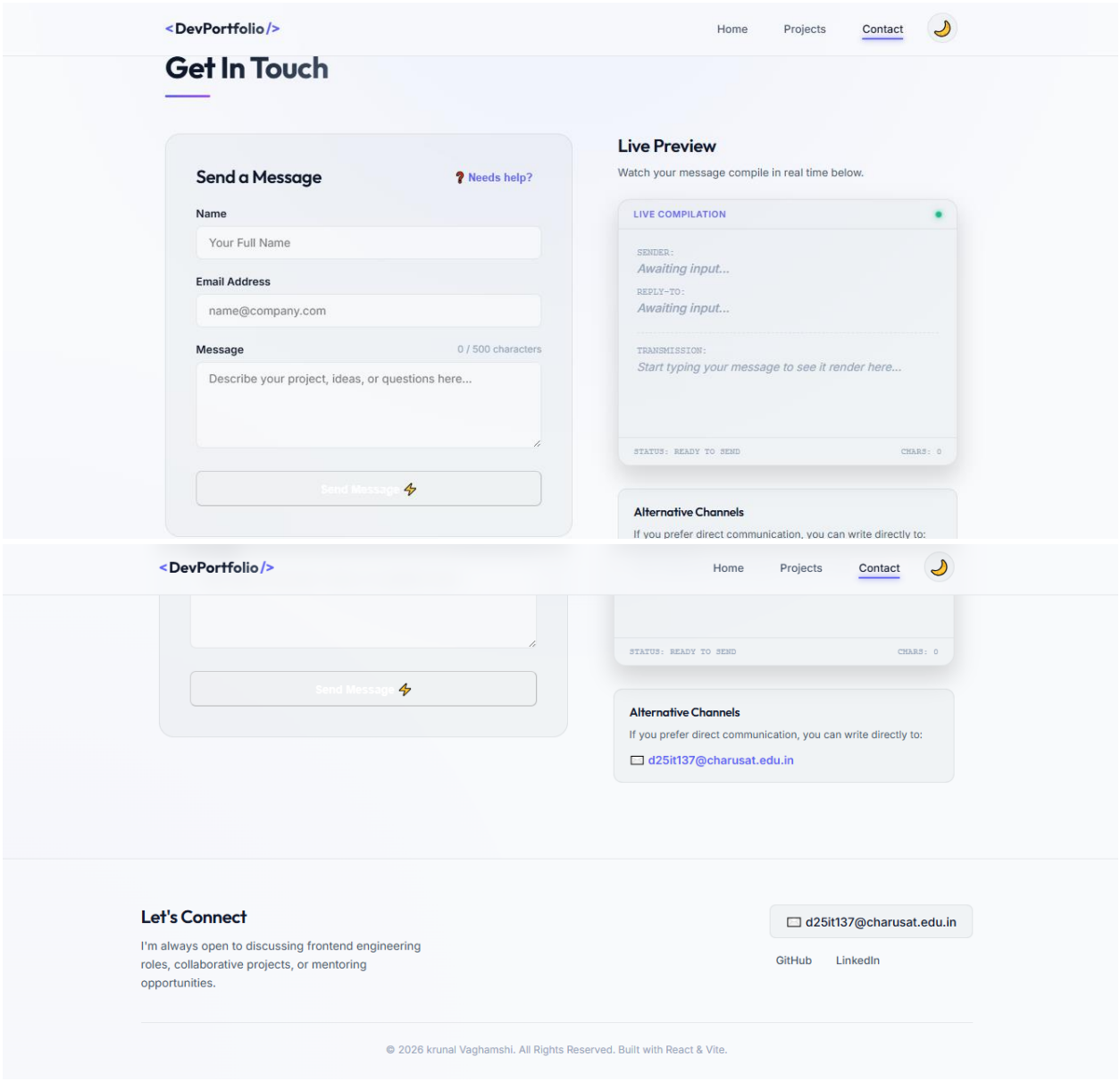
Projects-Page:



D25IT137


<DevPortfolio/>
Home
Projects
Contact
React
Chart.js
WebSockets
Tailwind
Synthetix AI
An automated code review agent that leverages large language models to scan commits for security vulnerabilities and performance bottlenecks.
Node.js
OpenAI API
GitHub Actions
TypeScript
Aether Engine
A browser-based interactive 3D physics sandbox built using WebGL and custom shaders to simulate fluid dynamics in real time.
React
Three.js
WebGL
GLSL
<DevPortfolio/>
Home
Projects
Contact
OmniSearch CLI
A lightning-fast command-line interface tool to search, index, and organize files across federated storage networks.
Go
gRPC
CLI
Shell
Let's Connect
I'm always open to discussing frontend engineering roles, collaborative projects, or mentoring opportunities.
d25it137@charusat.edu.in
GitHub
LinkedIn
© 2026 krunal Vaghamshi. All Rights Reserved. Built with React & Vite.

Contact Page:



Github Link:- [AWDF/p1 at main · krunal9116/AWDF]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

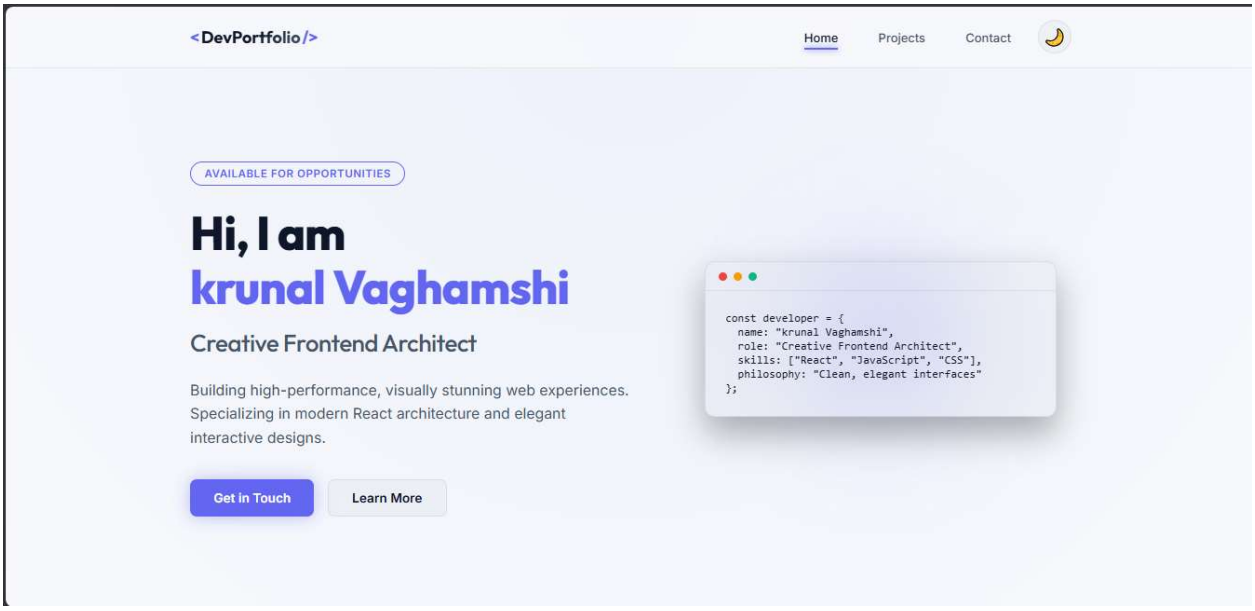
Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-3

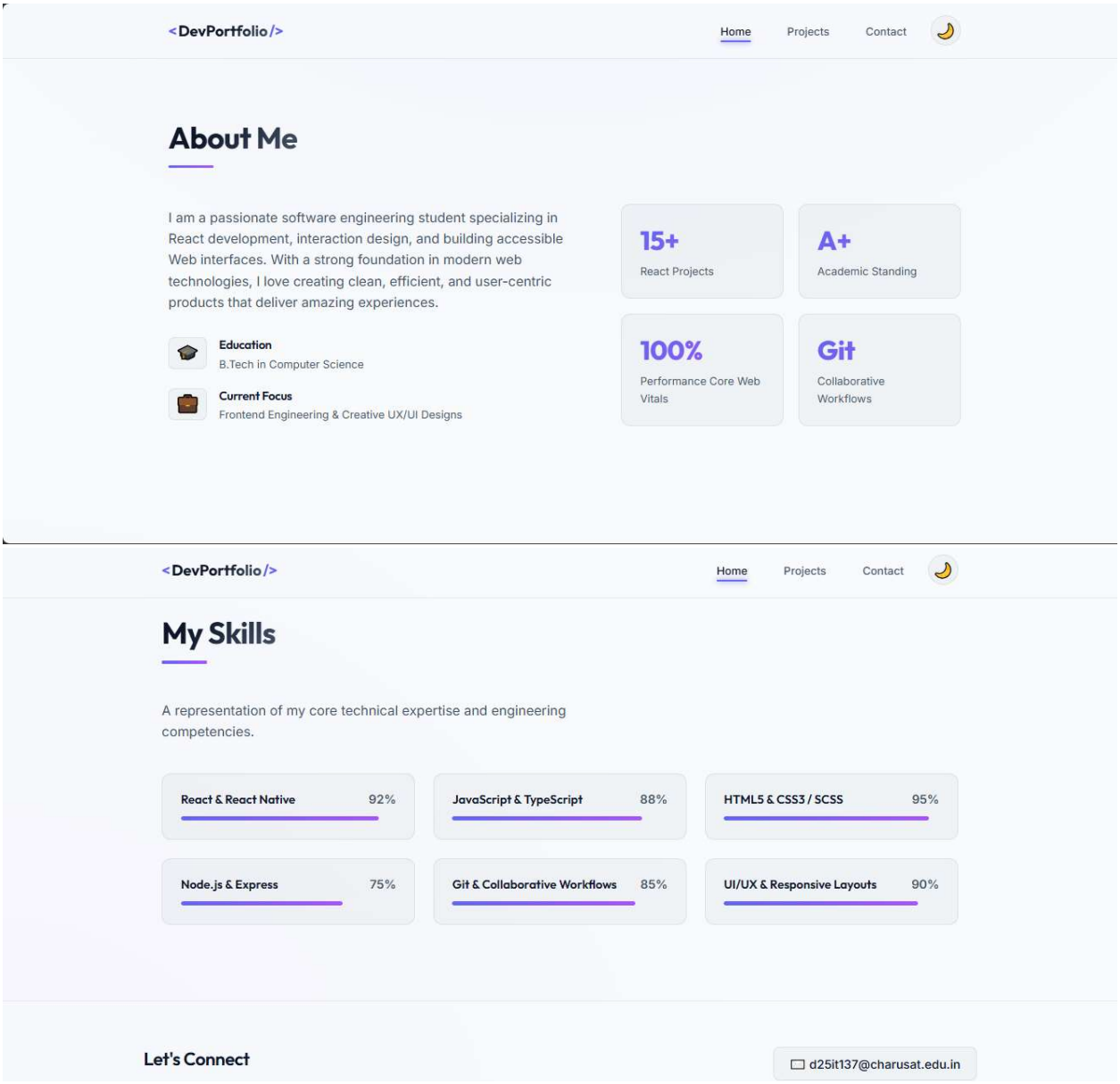
Problem Definition: -

• Integrate a public REST API (e.g., GitHub API) into the portfolio application to fetch and display a list of repositories dynamically.
• Use the Projects page built in Practical 2 as the integration point.
• Implement a loading spinner component shown while the request is in progress.
• Implement an error message component shown if the API call fails.
• Render at least the repository name and URL for each item returned.

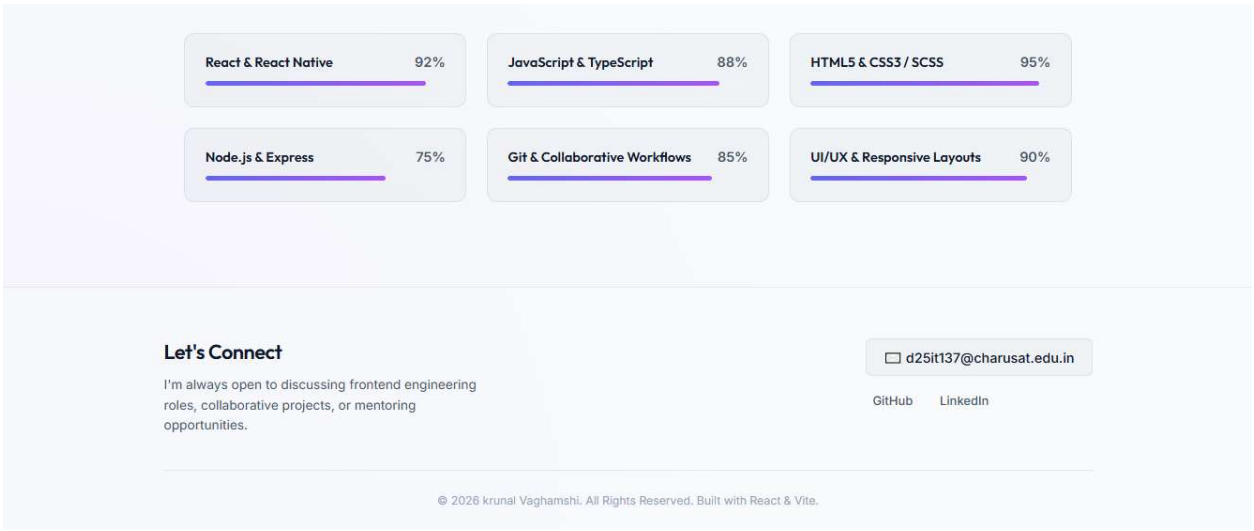
Screenshots:-Home-Page:



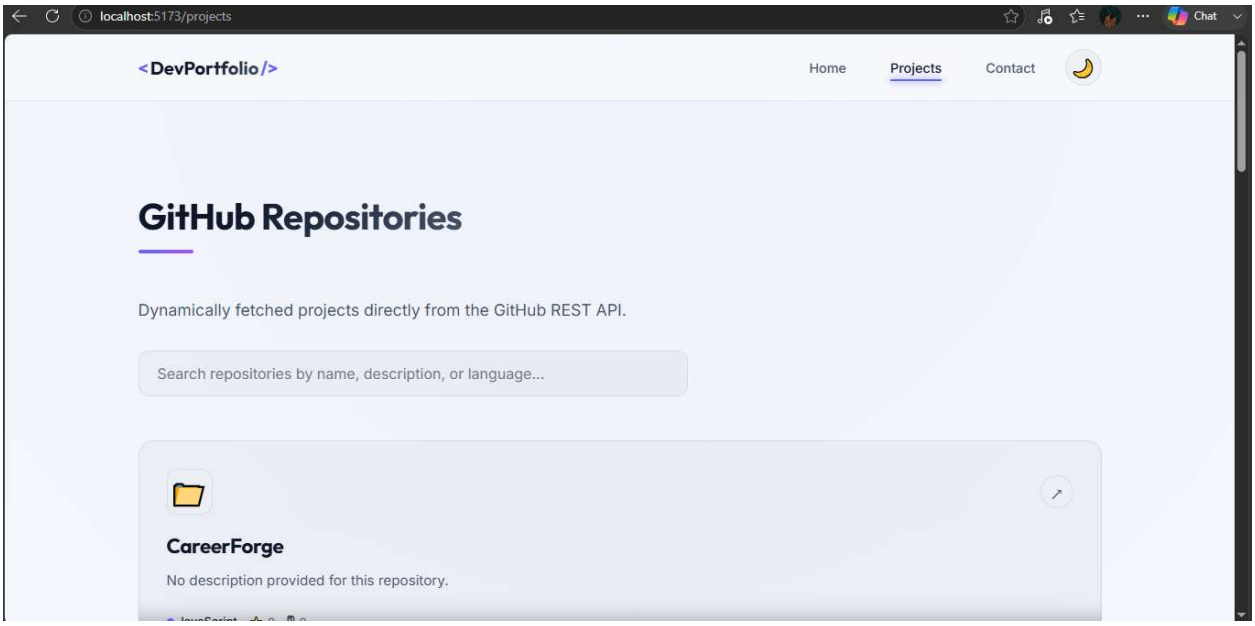
D25IT137


<DevPortfolio/>
Home
Projects
Contact
About Me
I am a passionate software engineering student specializing in React development, interaction design, and building accessible Web interfaces. With a strong foundation in modern web technologies, I love creating clean, efficient, and user-centric products that deliver amazing experiences.
Education
B.Tech in Computer Science
Current Focus
Frontend Engineering & Creative UX/UI Designs
15+
React Projects
A+
Academic Standing
100%
Performance Core Web Vitals
Git
Collaborative Workflows

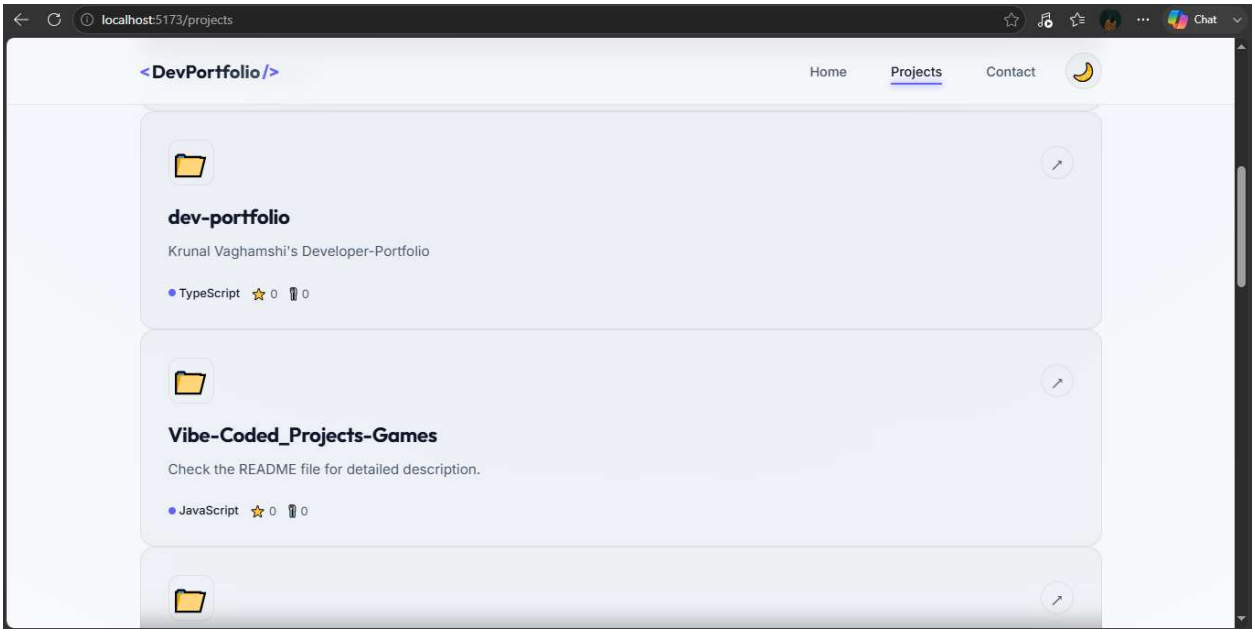
<DevPortfolio/>
Home
Projects
Contact
My Skills
A representation of my core technical expertise and engineering competencies.
React & React Native 92%
JavaScript & TypeScript 88%
HTML5 & CSS3 / SCSS 95%
Node.js & Express 75%
Git & Collaborative Workflows 85%
UI/UX & Responsive Layouts 90%
Let's Connect
d25it137@charusat.edu.in

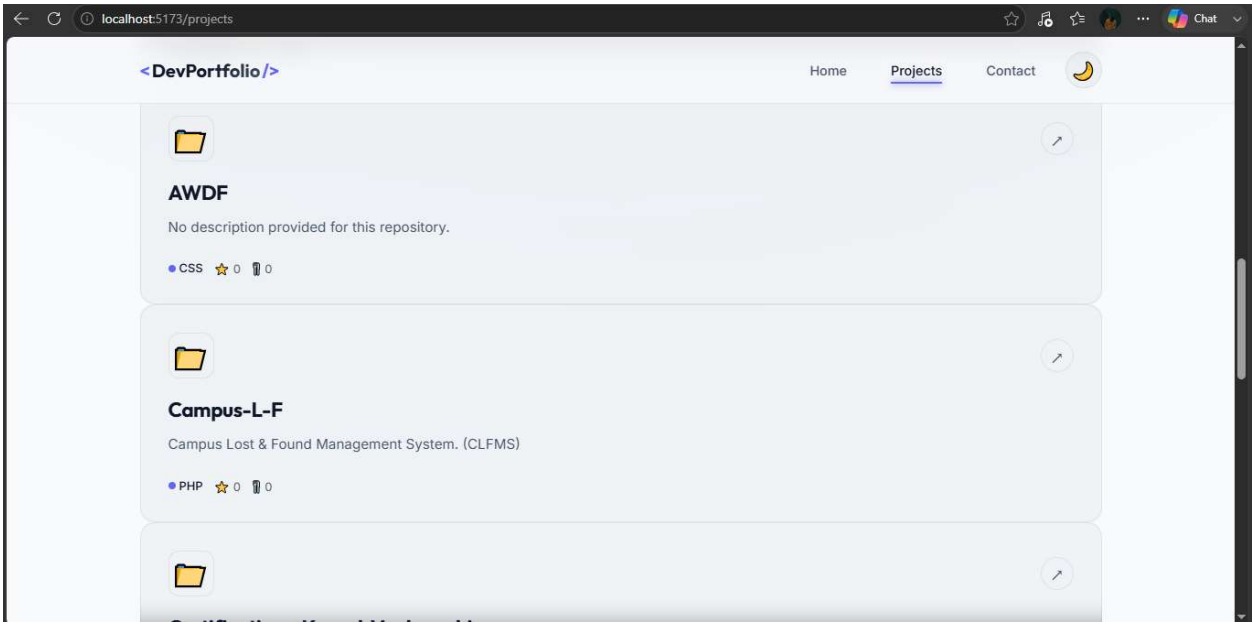


Projects-Page:

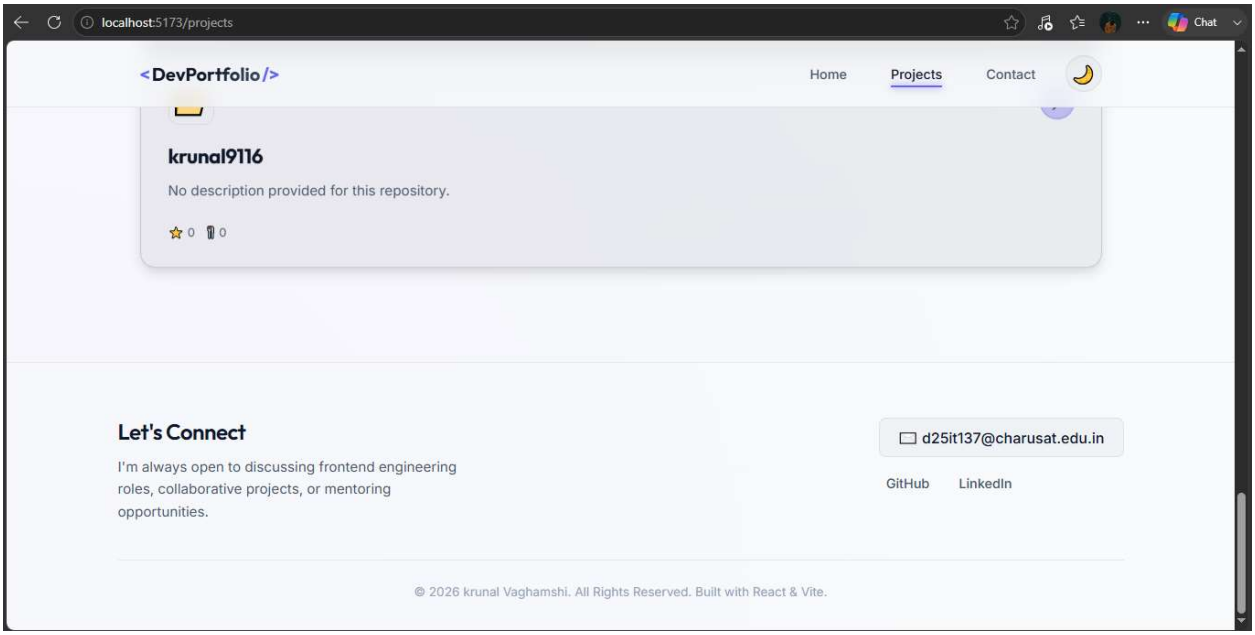


D25IT137

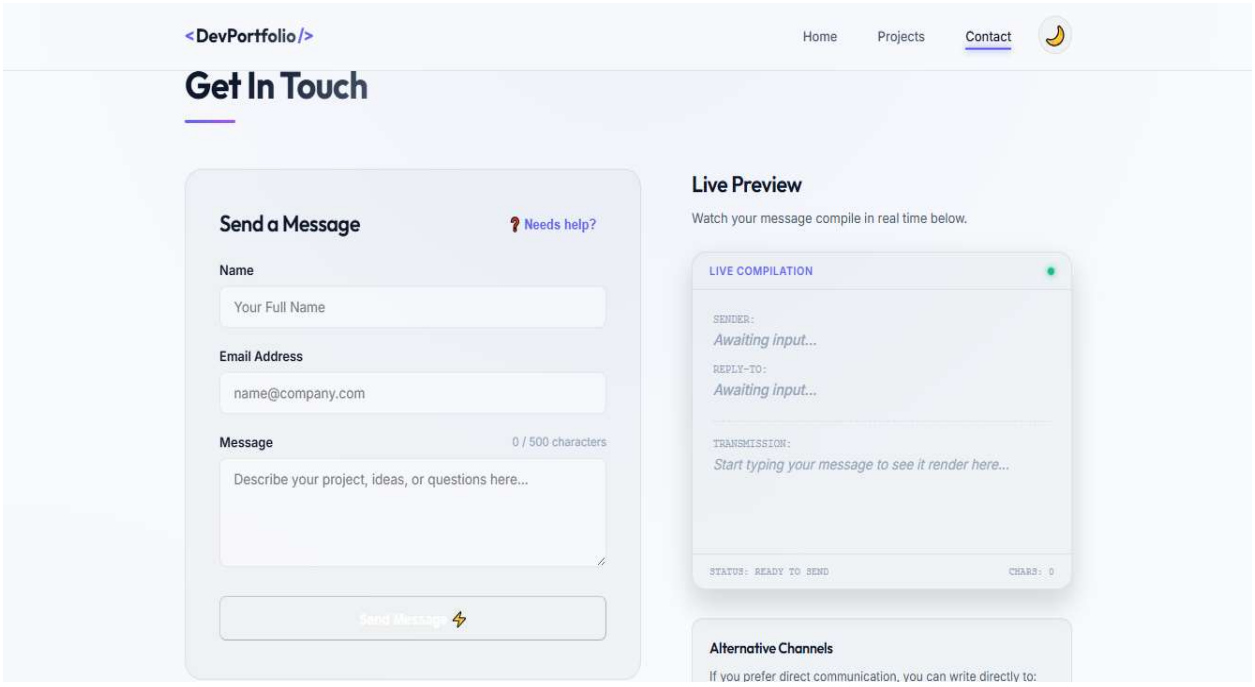

localhost:5173/projects
Chat
<DevPortfolio/>
Home
Projects
Contact
dev-portfolio
Krunal Vaghamshi's Developer-Portfolio
TypeScript 0 0
Vibe-Coded_Projects-Games
Check the README file for detailed description.
JavaScript 0 0


localhost:5173/projects
Chat
<DevPortfolio/>
Home
Projects
Contact
AWDF
No description provided for this repository.
CSS 0 0
Campus-L-F
Campus Lost & Found Management System. (CLFMS)
PHP 0 0

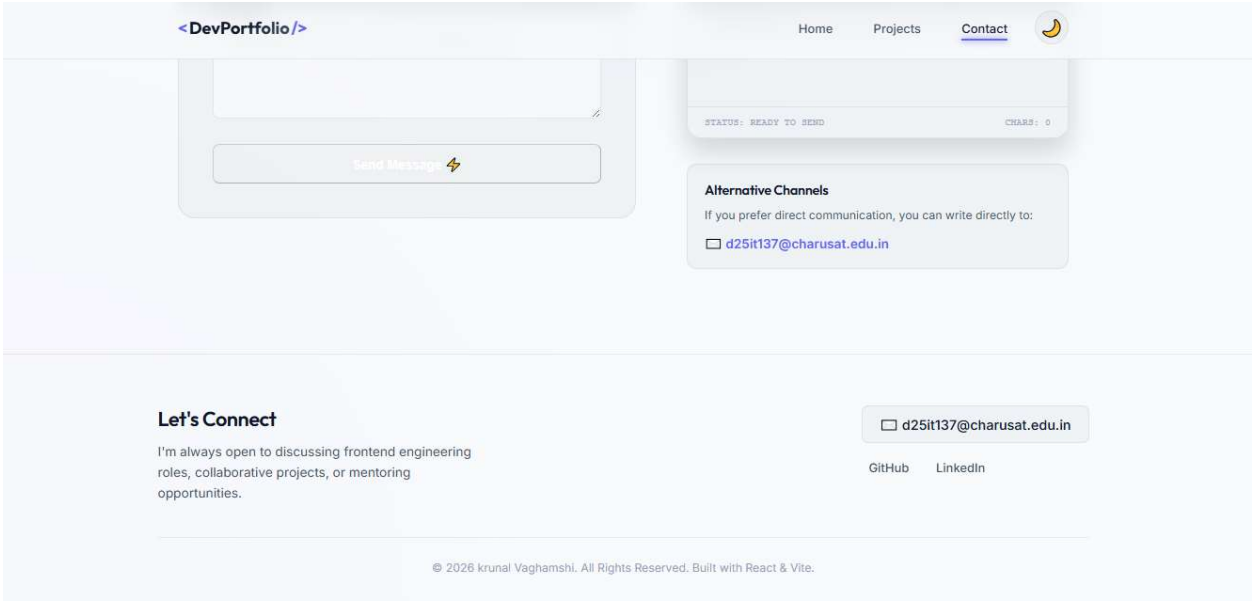
D25IT137



Contact-Page:



D25IT137



Github Link:- [AWDF/p1 at main · krunal9116/AWDF]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-4

Problem Definition: -

- Build a Node/Express backend for a Task Management system.
- Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet).
- Apply a request logging middleware that logs method, URL, and timestamp for every incoming request.
- Apply a global error handling middleware as the last middleware in the pipeline.
- Use correct HTTP status codes (200, 201, 404, 500) for each response.

Code:- Server.js:

```
const express = require('express');
const app = express();

let tasks = [
  { id: 1, title: 'Learn Node.js basics', completed: true },
  { id: 2, title: 'Build Express REST API', completed: false },
  { id: 3, title: 'Understand Express Middleware', completed: false }
];
let nextId = 4;

app.use(express.json());

// Request logger
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});
// Content-Type validator for POST/PUT
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
        message: 'Content-Type header must be application/json for POST and PUT requests'
```

```
      });
    }
  }
  next();
};

app.use(requireJsonContent);
// ID validation middleware
const validateTaskId = (req, res, next) => {
  const idParam = req.params.id;
  const parsedId = parseInt(idParam, 10);

  if (isNaN(parsedId) || parsedId.toString() !== idParam || parsedId <= 0) {
    return res.status(400).json({
      error: 'Invalid Request',
      message: `Task ID '${idParam}' must be a positive integer`
    });
  }
  req.taskId = parsedId;
  next();
};
// CRUD Routes
app.get('/tasks', (req, res) => {
  res.status(200).json({ success: true, count: tasks.length, data: tasks });
});

app.get('/tasks/:id', validateTaskId, (req, res) => {
  const task = tasks.find(t => t.id === req.taskId);
  if (!task) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not
found` });
  }
  res.status(200).json({ success: true, data: task });
});

app.post('/tasks', (req, res) => {
  const { title, completed } = req.body;
  if (!title || typeof title !== 'string' || title.trim() === '') {
    return res.status(400).json({ error: 'Validation Error', message: 'Task title is required'
});
  }
  const newTask = { id: nextId++, title: title.trim(), completed: !!completed };
  tasks.push(newTask);
  res.status(201).json({ success: true, message: 'Task created successfully', data: newTask
});
});
app.put('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.taskId);
  if (taskIndex === -1) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not
found` });
  }
  const { title, completed } = req.body;
  if (title !== undefined && (typeof title !== 'string' || title.trim() === '')) {
```

```
    return res.status(400).json({ error: 'Validation Error', message: 'Task title must be non-
empty string' });
  }
  if (completed !== undefined && typeof completed !== 'boolean') {
    return res.status(400).json({ error: 'Validation Error', message: 'Completed must be a
boolean' });
  }

  tasks[taskIndex] = {
    ...tasks[taskIndex],
    ...(title !== undefined && { title: title.trim() }),
    ...(completed !== undefined && { completed })
  };
  res.status(200).json({ success: true, message: 'Task updated successfully', data:
tasks[taskIndex] });
});

app.delete('/tasks/:id', validateTaskId, (req, res) => {
  const taskIndex = tasks.findIndex(t => t.id === req.taskId);
  if (taskIndex === -1) {
    return res.status(404).json({ error: 'Not Found', message: `Task with ID ${req.taskId} not
found` });
  }
  const deletedTask = tasks.splice(taskIndex, 1)[0];
  res.status(200).json({ success: true, message: `Task ${req.taskId} deleted`, data:
deletedTask });
});

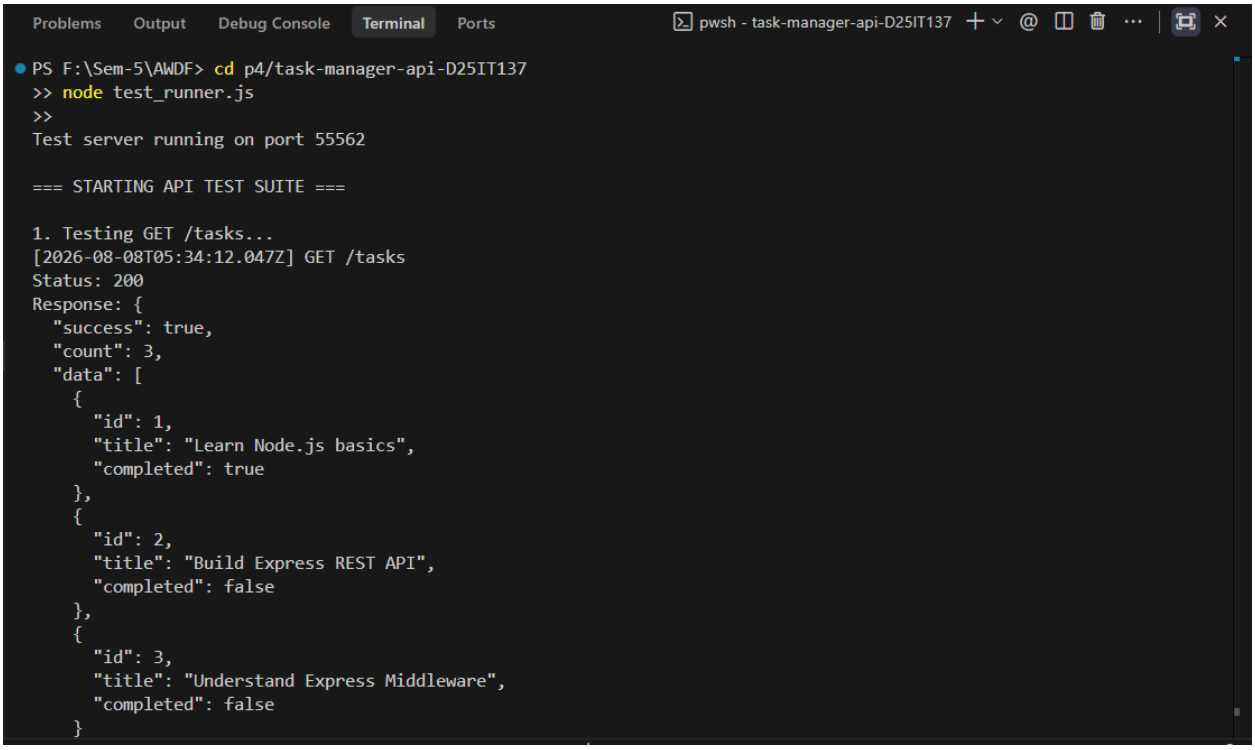
app.get('/cause-error', (req, res, next) => {
  next(new Error('Simulated internal server error'));
});

// 404 Handler
app.use((req, res) => {
  res.status(404).json({ error: 'Not Found', message: `Cannot ${req.method} ${req.url} -
Endpoint does not exist` });
});

// Global Error Handler
app.use((err, req, res, next) => {
  console.error(`[Error Logged]: ${err.stack}`);
  res.status(500).json({ error: 'Internal Server Error', message: 'Something went wrong on the
server' });
});

module.exports = app;
if (require.main === module) {
  const PORT = process.env.PORT || 5000;
  app.listen(PORT, () => {
    console.log(`Server listening on port ${PORT}`);
  });
}
```

Screenshots / Terminal Execution:-

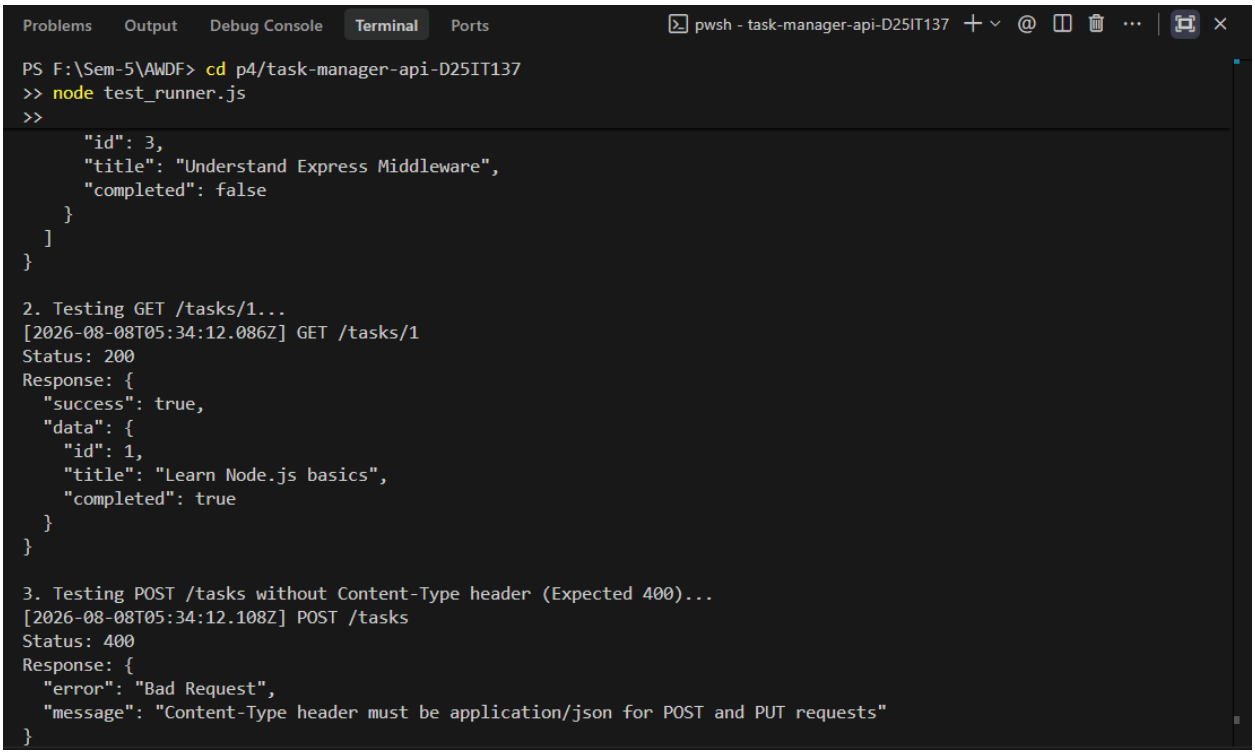


```
Problems  Output  Debug Console  Terminal  Ports        pwsh - task-manager-api-D25IT137

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
Test server running on port 55562

=== STARTING API TEST SUITE ===

1. Testing GET /tasks...
[2026-08-08T05:34:12.047Z] GET /tasks
Status: 200
Response: {
  "success": true,
  "count": 3,
  "data": [
    {
      "id": 1,
      "title": "Learn Node.js basics",
      "completed": true
    },
    {
      "id": 2,
      "title": "Build Express REST API",
      "completed": false
    },
    {
      "id": 3,
      "title": "Understand Express Middleware",
      "completed": false
    }
```

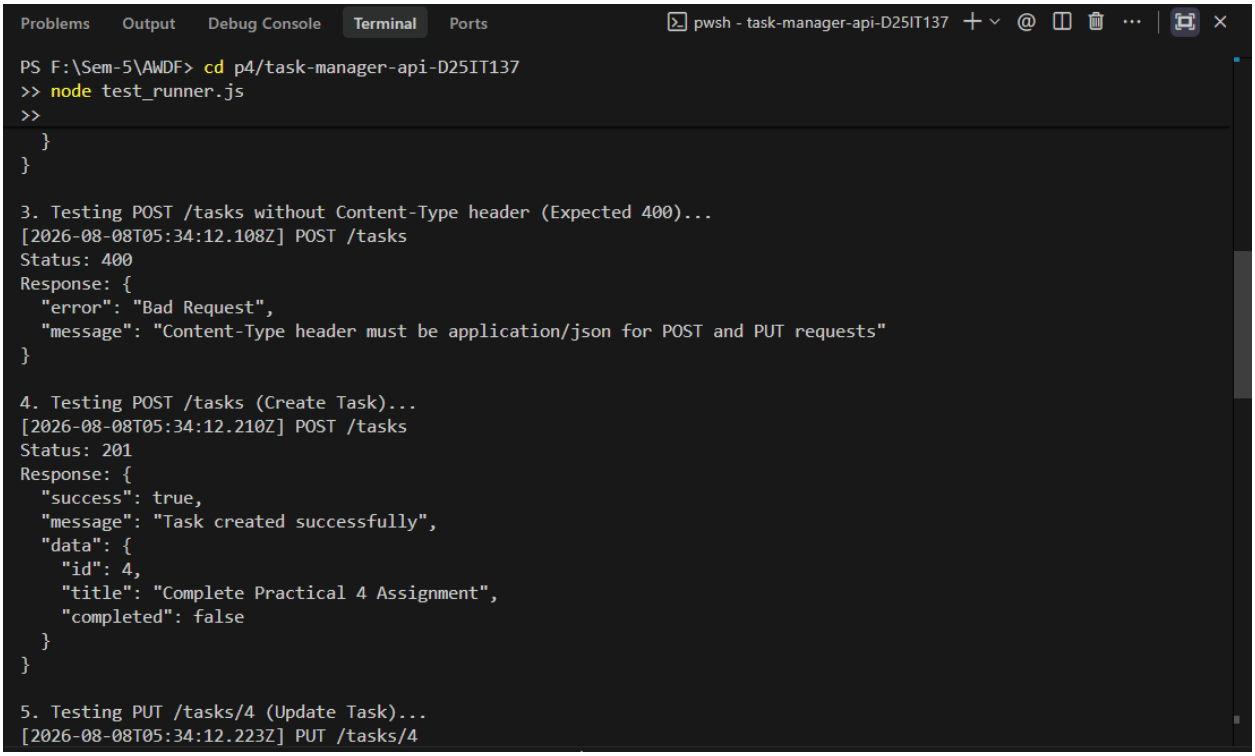


```
Problems  Output  Debug Console  Terminal  Ports        pwsh - task-manager-api-D25IT137

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
      "id": 3,
      "title": "Understand Express Middleware",
      "completed": false
    }
  ]
}

2. Testing GET /tasks/1...
[2026-08-08T05:34:12.086Z] GET /tasks/1
Status: 200
Response: {
  "success": true,
  "data": {
    "id": 1,
    "title": "Learn Node.js basics",
    "completed": true
  }
}

3. Testing POST /tasks without Content-Type header (Expected 400)...
[2026-08-08T05:34:12.108Z] POST /tasks
Status: 400
Response: {
  "error": "Bad Request",
  "message": "Content-Type header must be application/json for POST and PUT requests"
}
```



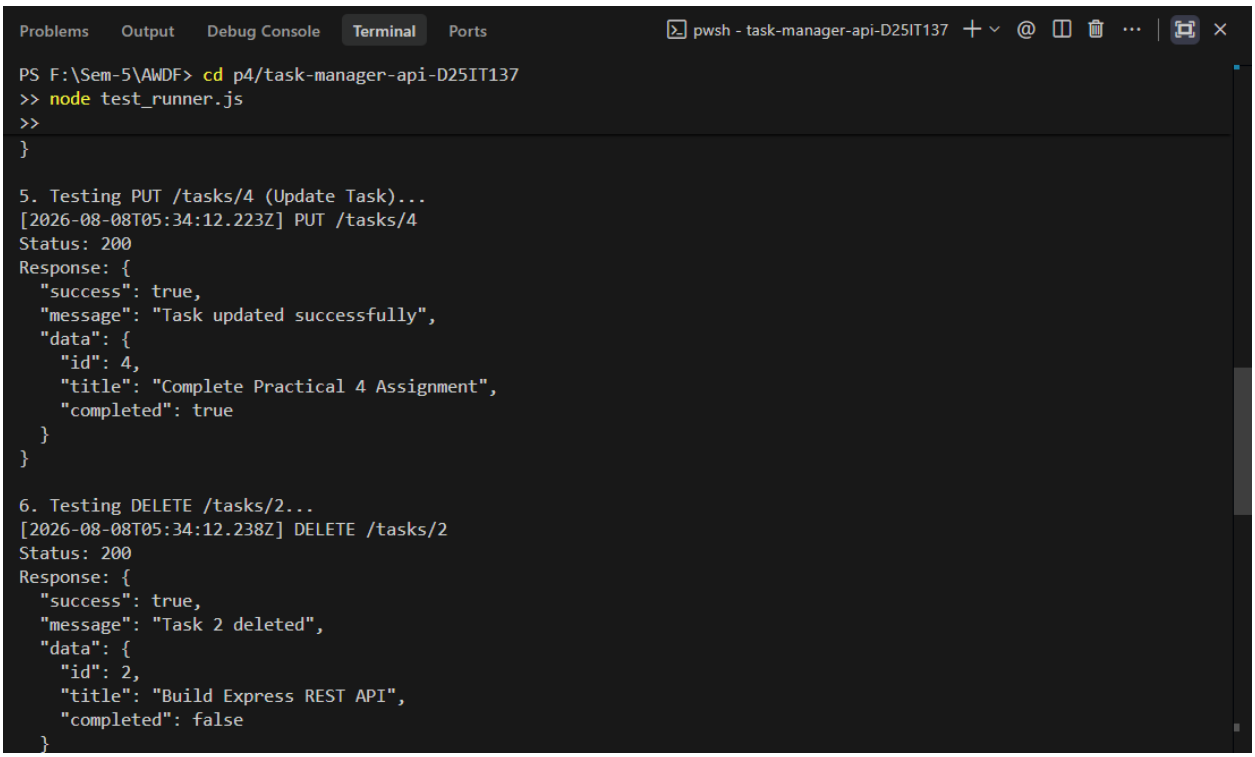
```
Problems  Output  Debug Console  Terminal  Ports        pwsh - task-manager-api-D25IT137

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
  }
}

3. Testing POST /tasks without Content-Type header (Expected 400)...
[2026-08-08T05:34:12.108Z] POST /tasks
Status: 400
Response: {
  "error": "Bad Request",
  "message": "Content-Type header must be application/json for POST and PUT requests"
}

4. Testing POST /tasks (Create Task)...
[2026-08-08T05:34:12.210Z] POST /tasks
Status: 201
Response: {
  "success": true,
  "message": "Task created successfully",
  "data": {
    "id": 4,
    "title": "Complete Practical 4 Assignment",
    "completed": false
  }
}

5. Testing PUT /tasks/4 (Update Task)...
[2026-08-08T05:34:12.223Z] PUT /tasks/4
```



```
Problems  Output  Debug Console  Terminal  Ports        pwsh - task-manager-api-D25IT137

PS F:\Sem-5\AWDF> cd p4/task-manager-api-D25IT137
>> node test_runner.js
>>
}

5. Testing PUT /tasks/4 (Update Task)...
[2026-08-08T05:34:12.223Z] PUT /tasks/4
Status: 200
Response: {
  "success": true,
  "message": "Task updated successfully",
  "data": {
    "id": 4,
    "title": "Complete Practical 4 Assignment",
    "completed": true
  }
}

6. Testing DELETE /tasks/2...
[2026-08-08T05:34:12.238Z] DELETE /tasks/2
Status: 200
Response: {
  "success": true,
  "message": "Task 2 deleted",
  "data": {
    "id": 2,
    "title": "Build Express REST API",
    "completed": false
  }
```

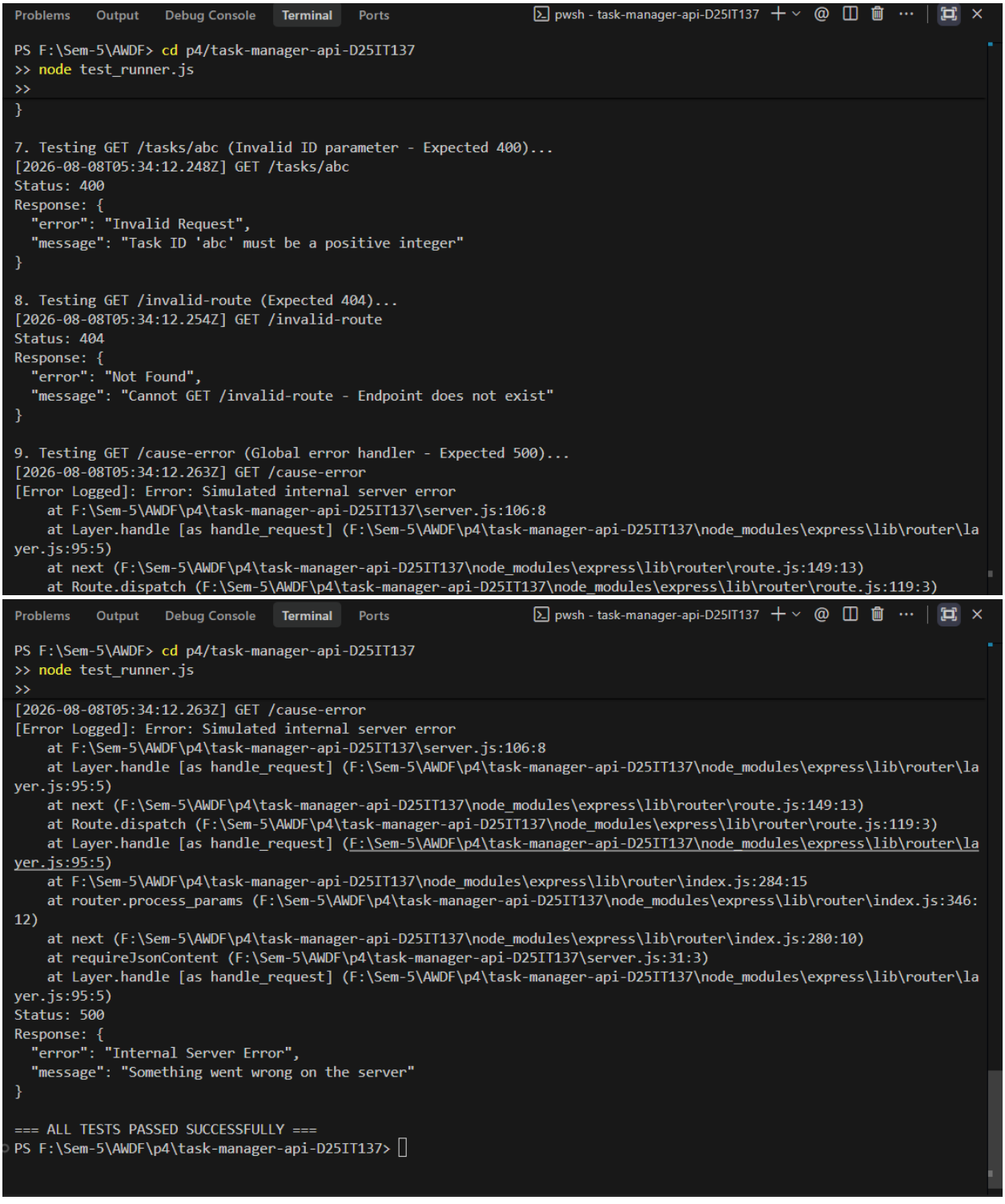


Figure 1:- We can see that all Tests are passed successfully with all status codes.

Github Link:- [krunal9116/task-manager-api-D25IT137]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-5

Problem Definition: -

• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.

• Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).

• Replace the in-memory array from Practical 4 with real Mongoose model operations.

• Test all CRUD operations against the live database using Postman.

• Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

Screenshots:-

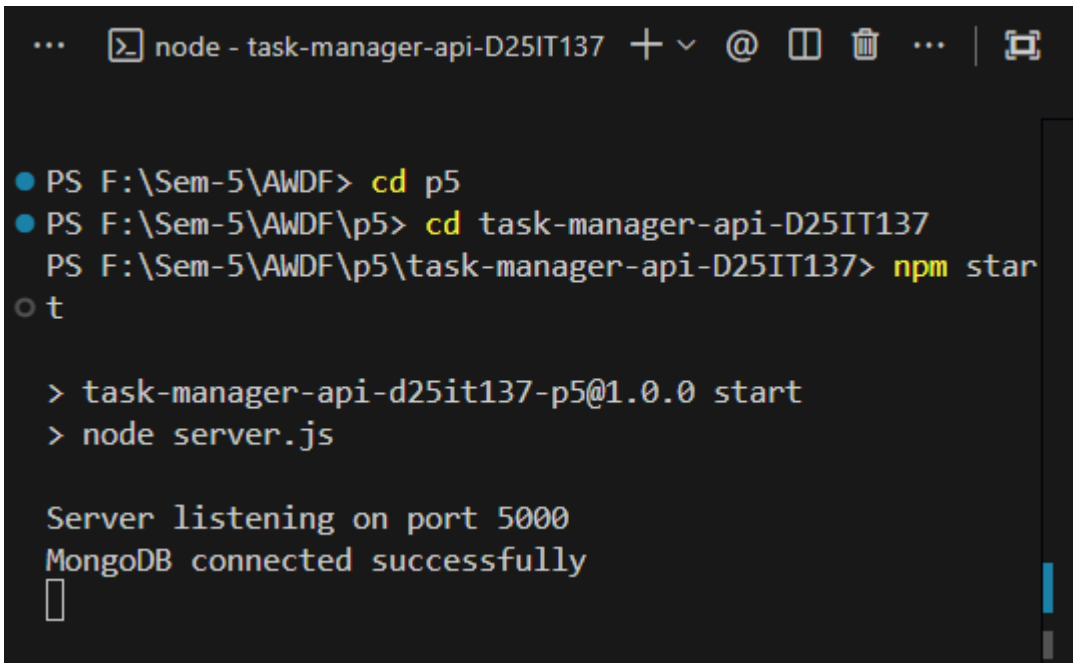
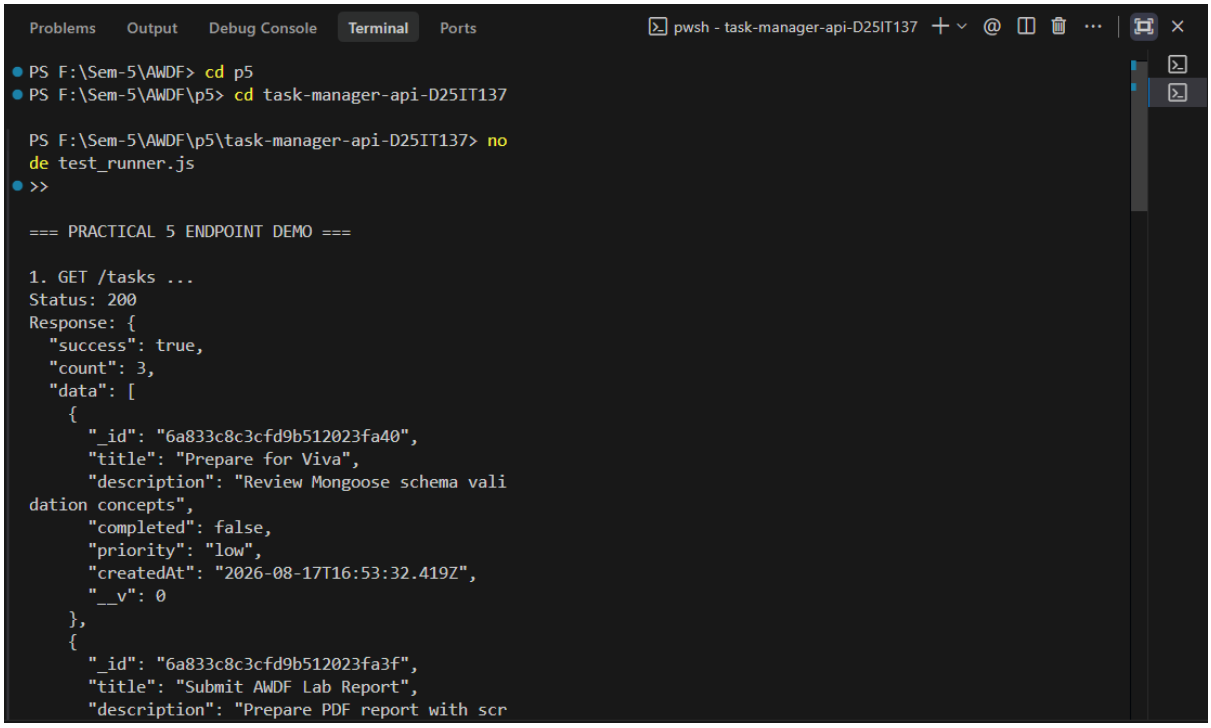
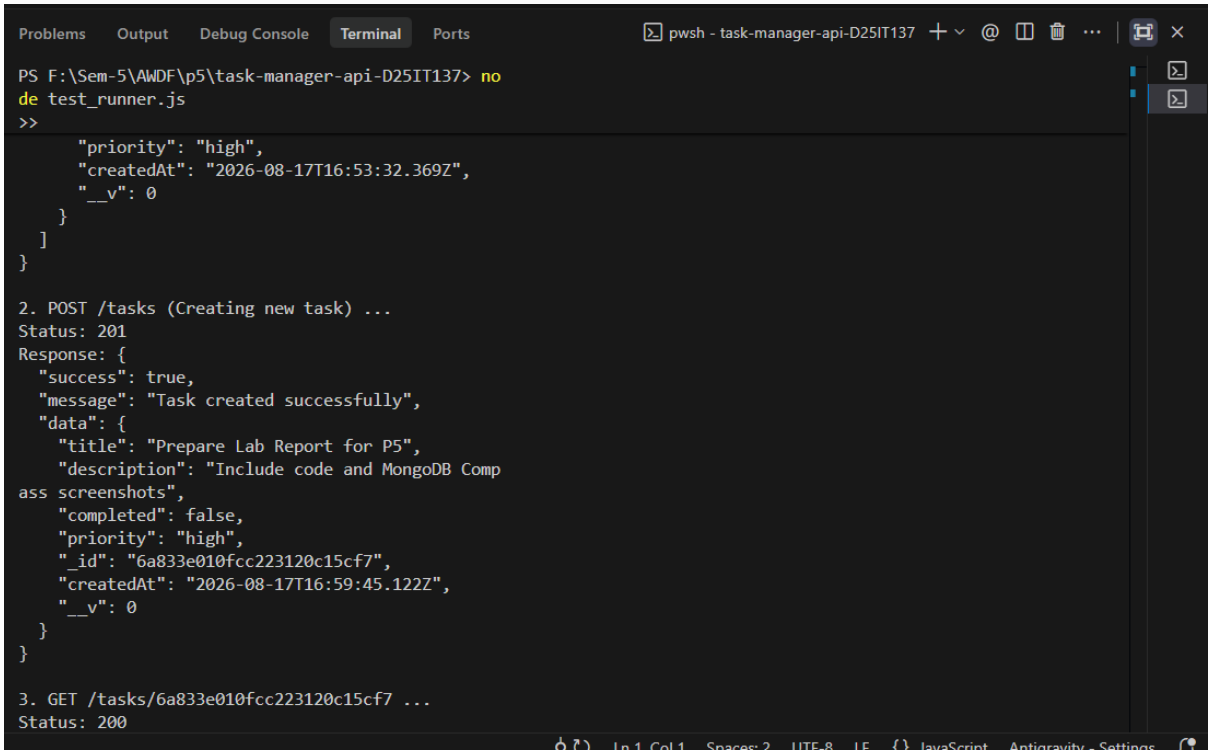


Figure 1-Connected Mongodb


Problems
Output
Debug Console
Terminal
Ports
pwsh - task-manager-api-D25IT137
PS F:\Sem-5\AWDF> cd p5
PS F:\Sem-5\AWDF\p5> cd task-manager-api-D25IT137
PS F:\Sem-5\AWDF\p5\task-manager-api-D25IT137> no
de test_runner.js
>>
=== PRACTICAL 5 ENDPOINT DEMO ===
1. GET /tasks ...
Status: 200
Response: {
"success": true,
"count": 3,
"data": [
{
"_id": "6a833c8c3cfd9b512023fa40",
"title": "Prepare for Viva",
"description": "Review Mongoose schema vali
dation concepts",
"completed": false,
"priority": "low",
"createdAt": "2026-08-17T16:53:32.419Z",
"__v": 0
},
{
"_id": "6a833c8c3cfd9b512023fa3f",
"title": "Submit AWDF Lab Report",
"description": "Prepare PDF report with scr


Problems
Output
Debug Console
Terminal
Ports
pwsh - task-manager-api-D25IT137
PS F:\Sem-5\AWDF\p5\task-manager-api-D25IT137> no
de test_runner.js
>>
"priority": "high",
"createdAt": "2026-08-17T16:53:32.369Z",
"__v": 0
}
]
}
2. POST /tasks (Creating new task) ...
Status: 201
Response: {
"success": true,
"message": "Task created successfully",
"data": {
"title": "Prepare Lab Report for P5",
"description": "Include code and MongoDB Comp
ass screenshots",
"completed": false,
"priority": "high",
"_id": "6a833e010fcc223120c15cf7",
"createdAt": "2026-08-17T16:59:45.122Z",
"__v": 0
}
}
3. GET /tasks/6a833e010fcc223120c15cf7 ...
Status: 200

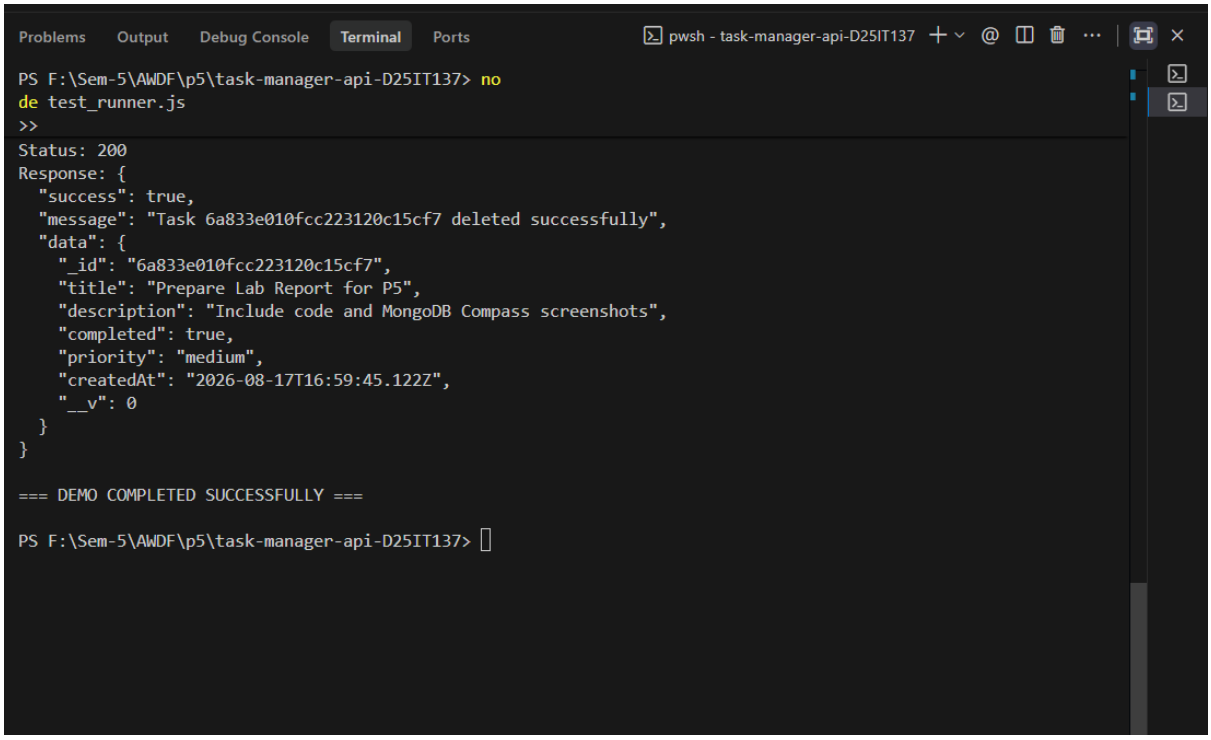


Figure 2-Executed test_runner.js

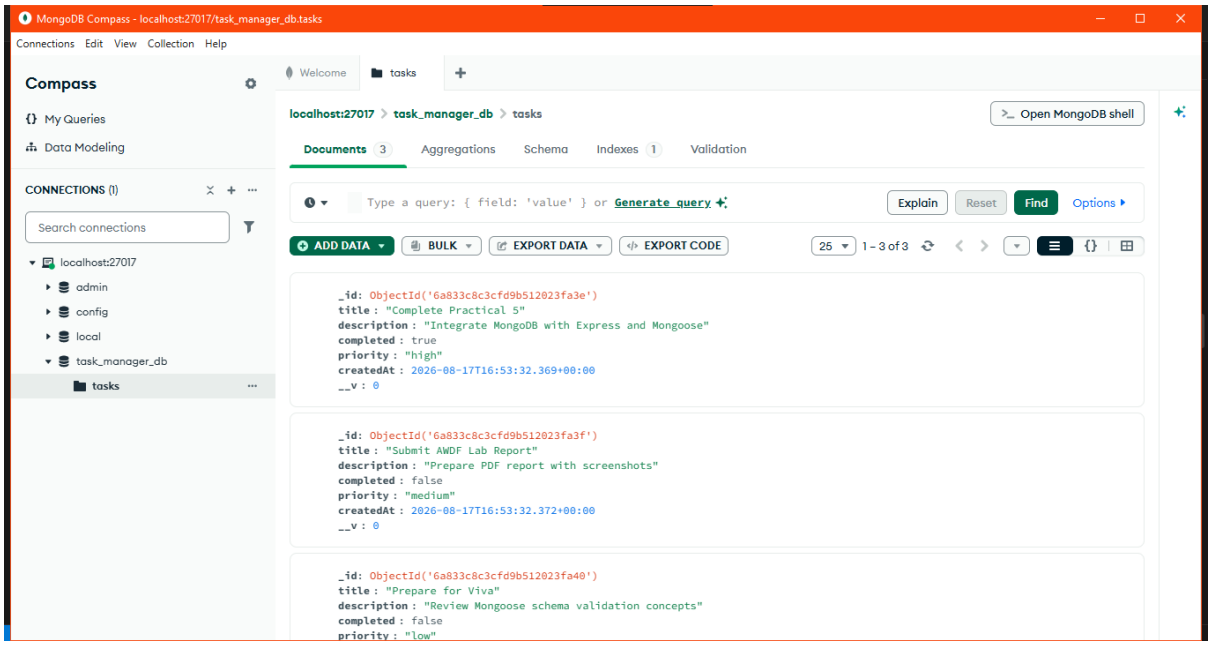


Figure 3-Mongodb Compass Structure & Data

Github Link:- krunal9116/task-manager-api-D25IT137

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-6

Problem Definition: -

• Connect the React Task UI to the Node+MongoDB backend built in Practicals 4 and 5.

• Replace the GitHub repo data from Practical 3 with task data from your own backend.

• The application must support creating, viewing, updating, and deleting tasks from the React interface through API calls.

• All data must be persisted in MongoDB confirmed by refreshing the browser and seeing data remain.

• Handle loading and error states for every API interaction, not just the initial fetch.

Screenshots:-

Server.js:

p6 > backend > JS server.js > ...

```
const express = require('express');
const mongoose = require('mongoose');
require('dotenv').config();

const Task = require('./models/Task');

const cors = require('cors');

const app = express();

app.use(cors());
app.use(express.json());

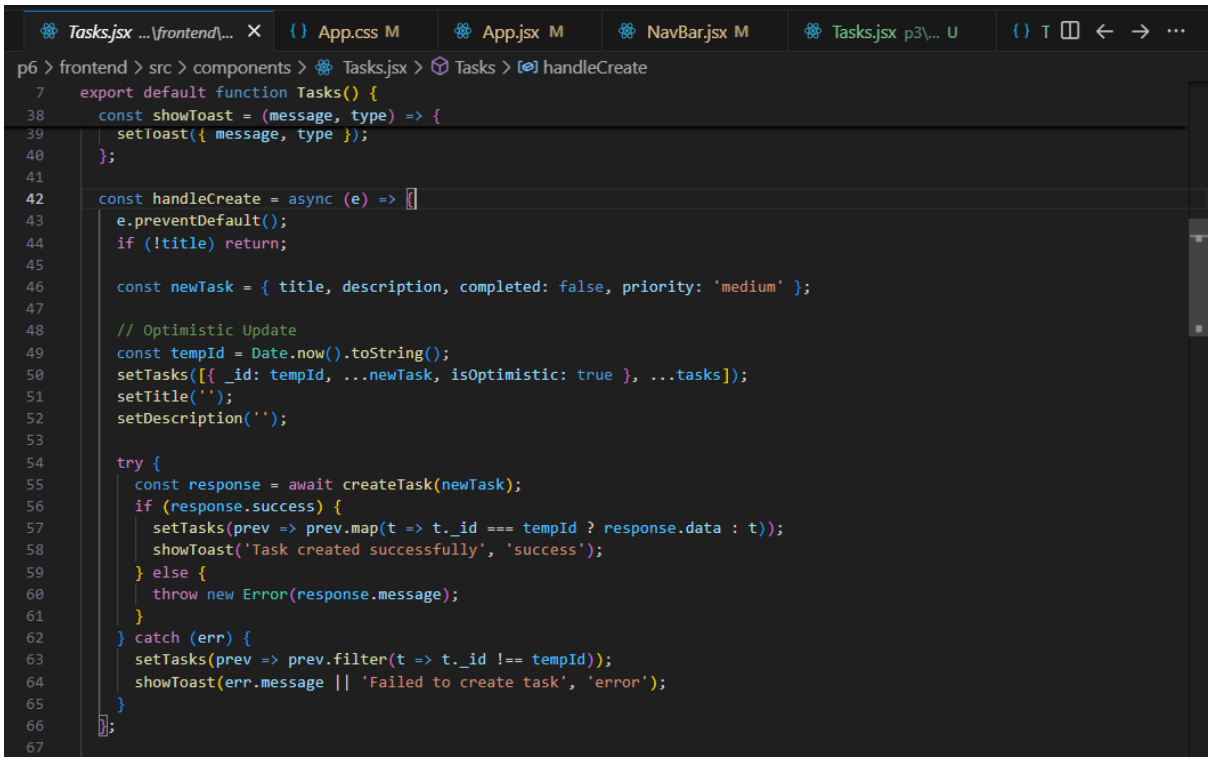
// Request logger middleware
app.use((req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`[${timestamp}] ${req.method} ${req.url}`);
  next();
});

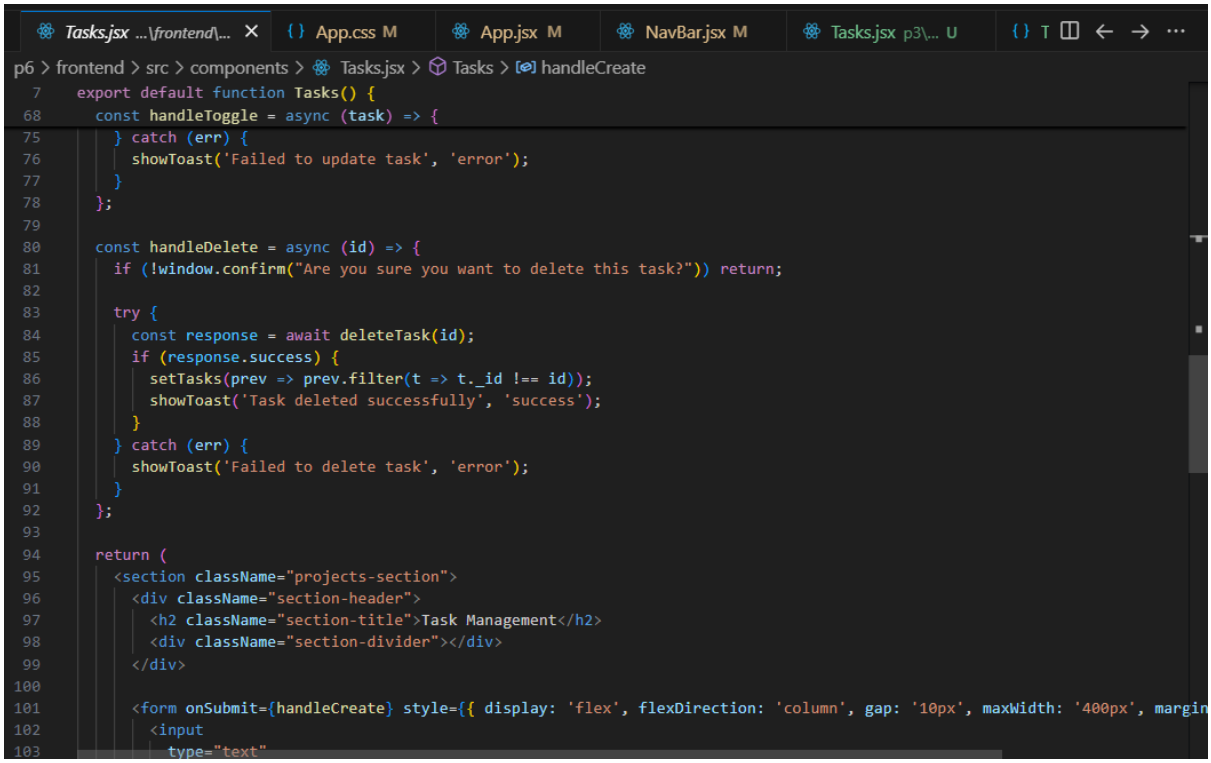
// Content-Type validator for POST/PUT
const requireJsonContent = (req, res, next) => {
  if (['POST', 'PUT'].includes(req.method)) {
    const contentType = req.headers['content-type'];
    if (!contentType || !contentType.includes('application/json')) {
      return res.status(400).json({
        error: 'Bad Request',
        message: 'Content-Type header must be application/json for POST and PUT requests'
      });
    }
  }
```

Api.js:



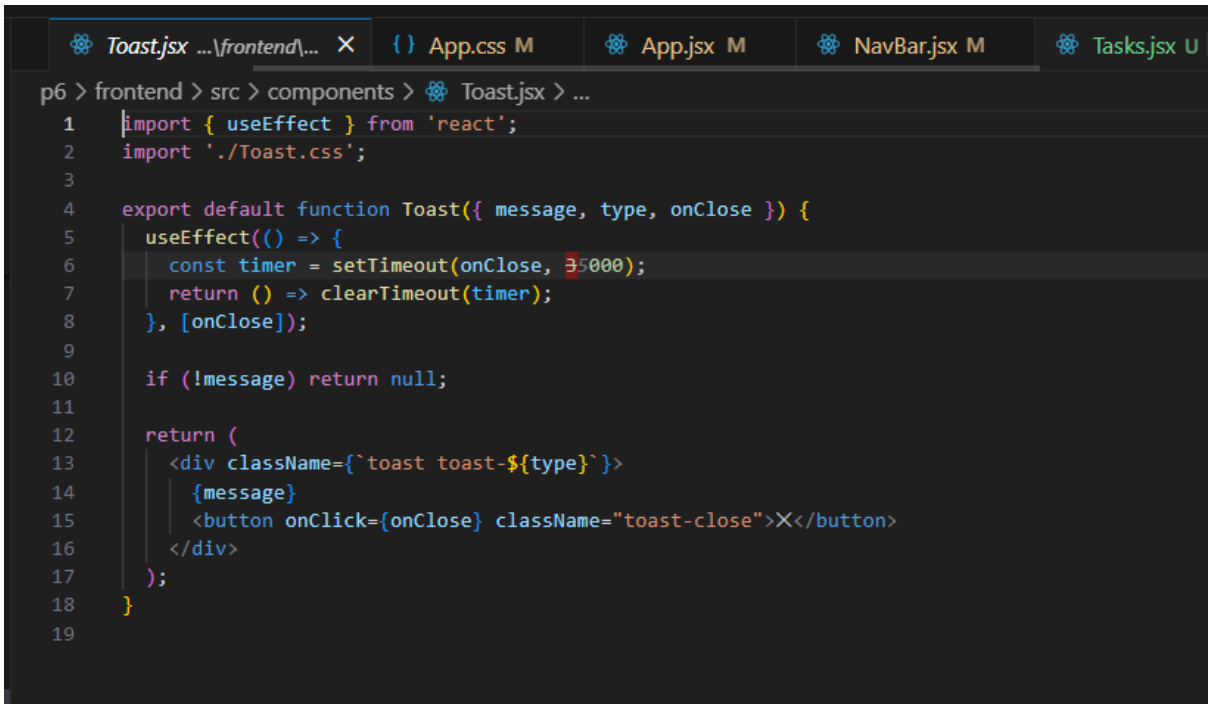
Task.jsx:





```
Tasks.jsx ...\frontend\... | App.css M | App.jsx M | NavBar.jsx M | Tasks.jsx p3\... U
p6 > frontend > src > components > Tasks.jsx > Tasks > handleCreate
 7   export default function Tasks() {
68     const handleToggle = async (task) => {
75       } catch (err) {
76         showToast('Failed to update task', 'error');
77       }
78     };
79
80     const handleDelete = async (id) => {
81       if (!window.confirm("Are you sure you want to delete this task?")) return;
82
83       try {
84         const response = await deleteTask(id);
85         if (response.success) {
86           setTasks(prev => prev.filter(t => t._id !== id));
87           showToast('Task deleted successfully', 'success');
88         }
89       } catch (err) {
90         showToast('Failed to delete task', 'error');
91       }
92     };
93
94     return (
95       <section className="projects-section">
96         <div className="section-header">
97           <h2 className="section-title">Task Management</h2>
98           <div className="section-divider"></div>
99         </div>
100
101        <form onSubmit={handleCreate} style={{ display: 'flex', flexDirection: 'column', gap: '10px', maxWidth: '400px', margin
102          <input
103            type="text"
```

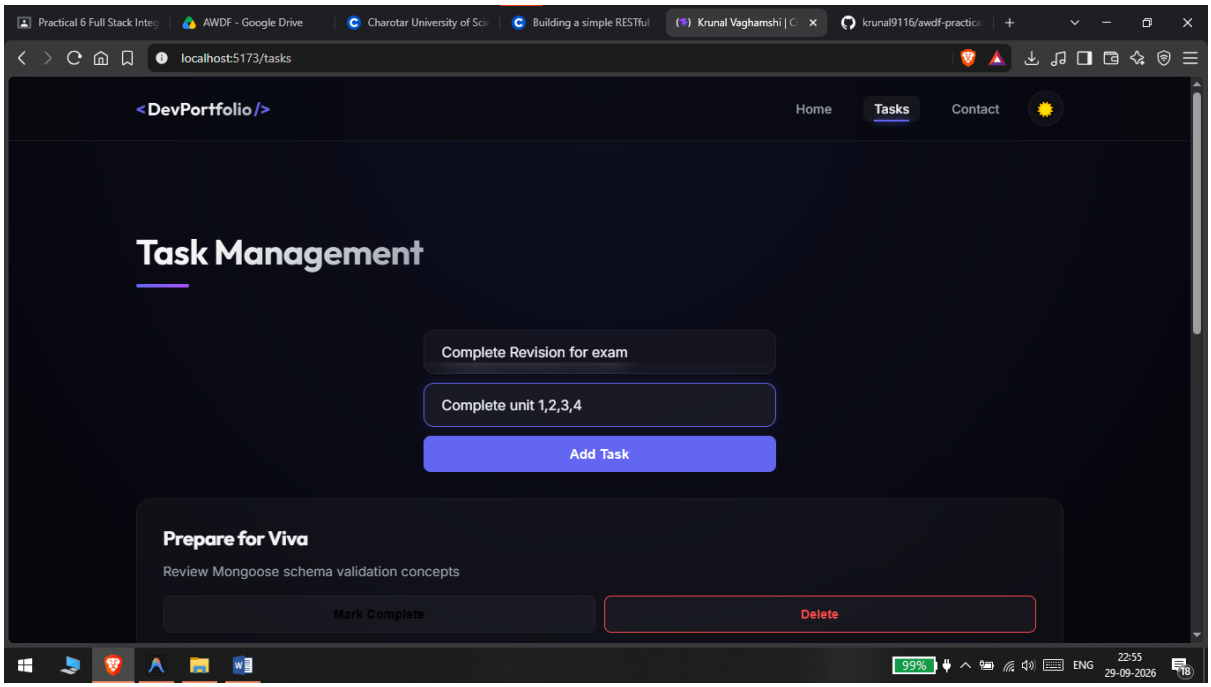
Toast.jsx:



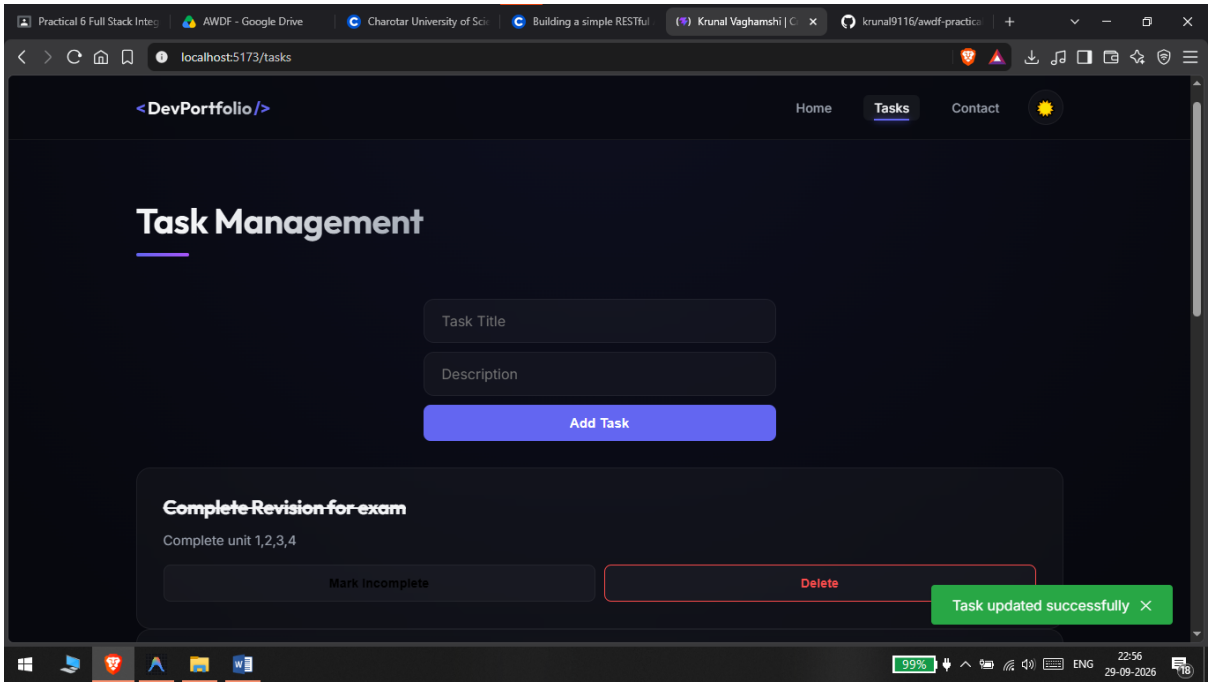
```
Toast.jsx ...\frontend\... | App.css M | App.jsx M | NavBar.jsx M | Tasks.jsx U
p6 > frontend > src > components > Toast.jsx > ...
1   import { useEffect } from 'react';
2   import './Toast.css';
3
4   export default function Toast({ message, type, onClose }) {
5     useEffect(() => {
6       const timer = setTimeout(onClose, 5000);
7       return () => clearTimeout(timer);
8     }, [onClose]);
9
10    if (!message) return null;
11
12    return (
13      <div className={`toast toast-${type}`}>
14        {message}
15        <button onClick={onClose} className="toast-close">X</button>
16      </div>
17    );
18  }
19
```

Output:

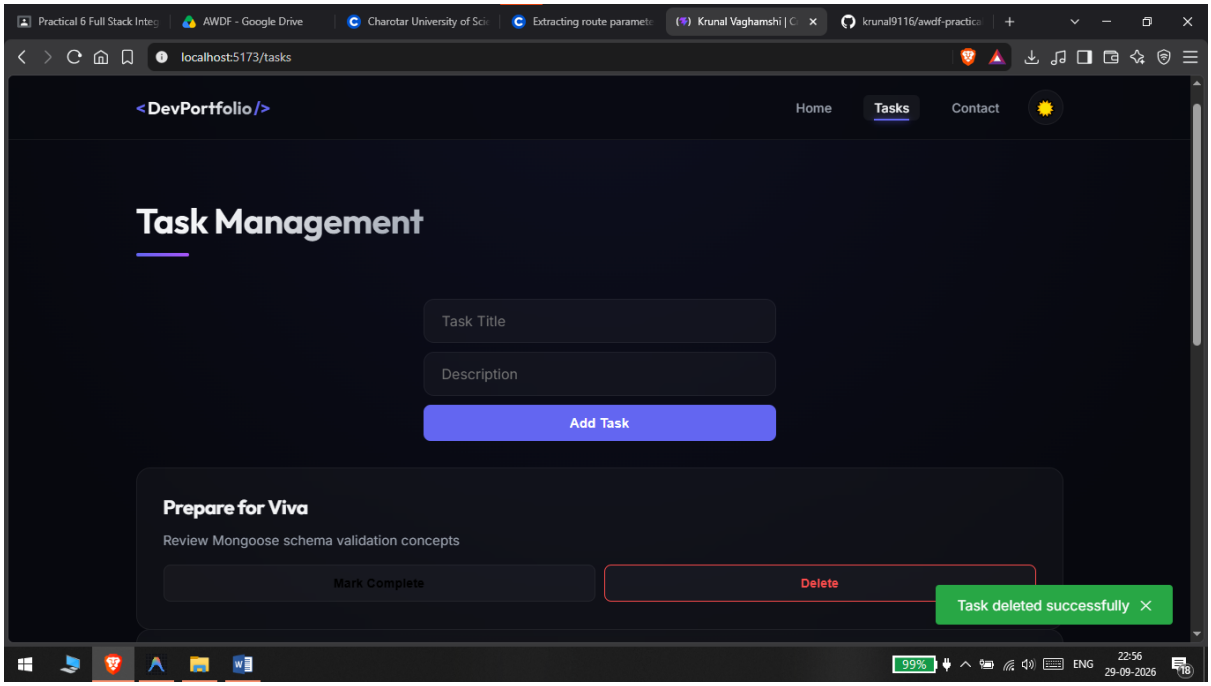
Task creation:



Updating the task to Completed:



Deleting the Task:



Github Link:- [Practical-6 Repository (AWDF)]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

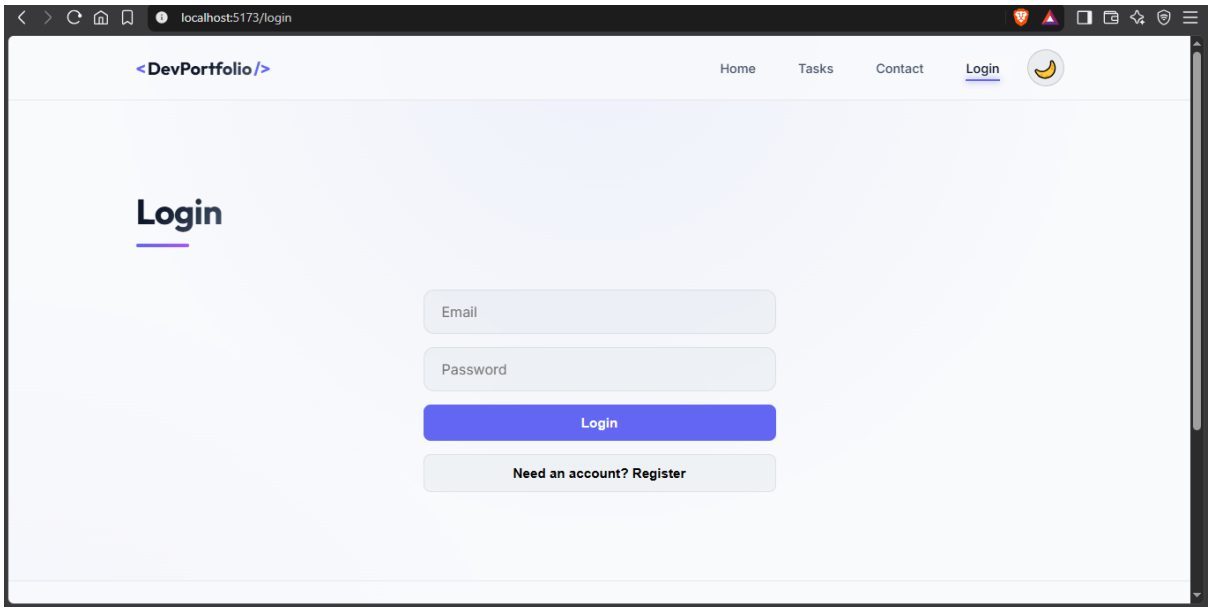
Practical-7

Problem Definition: -

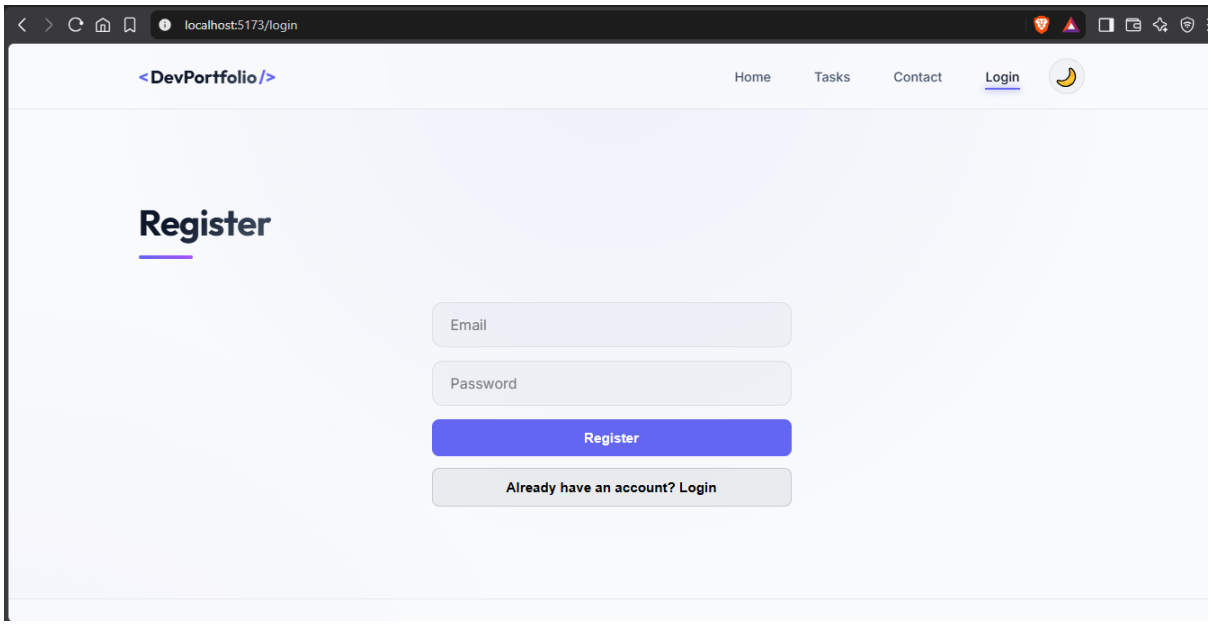
- Add user registration and login functionality to the Task Management application.
- Hash passwords using bcrypt before saving the User document to MongoDB.
- Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour)
- Protect all task routes using an authentication middleware that verifies the JWT.
- Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.

Screenshots:-

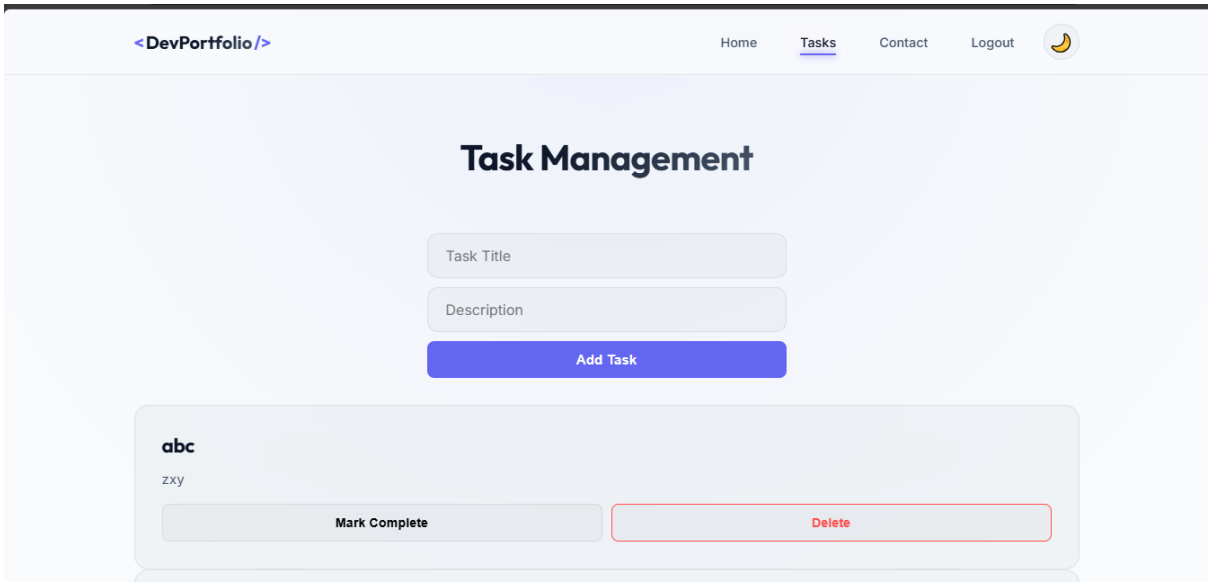
Login page:



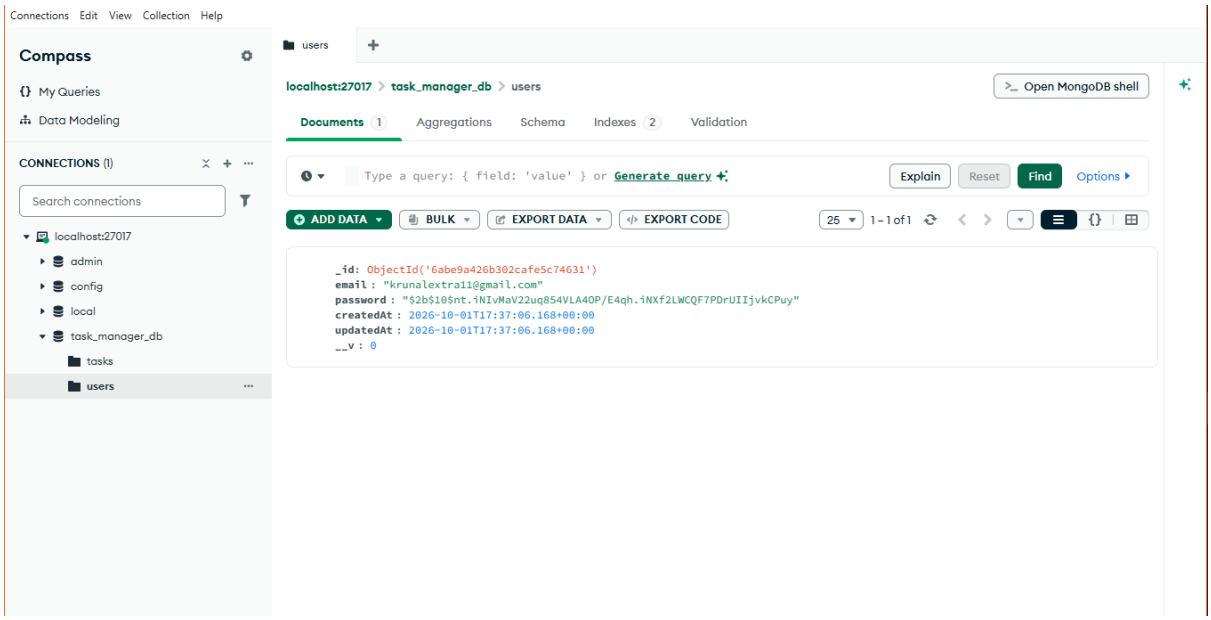
Register page:



Task page:

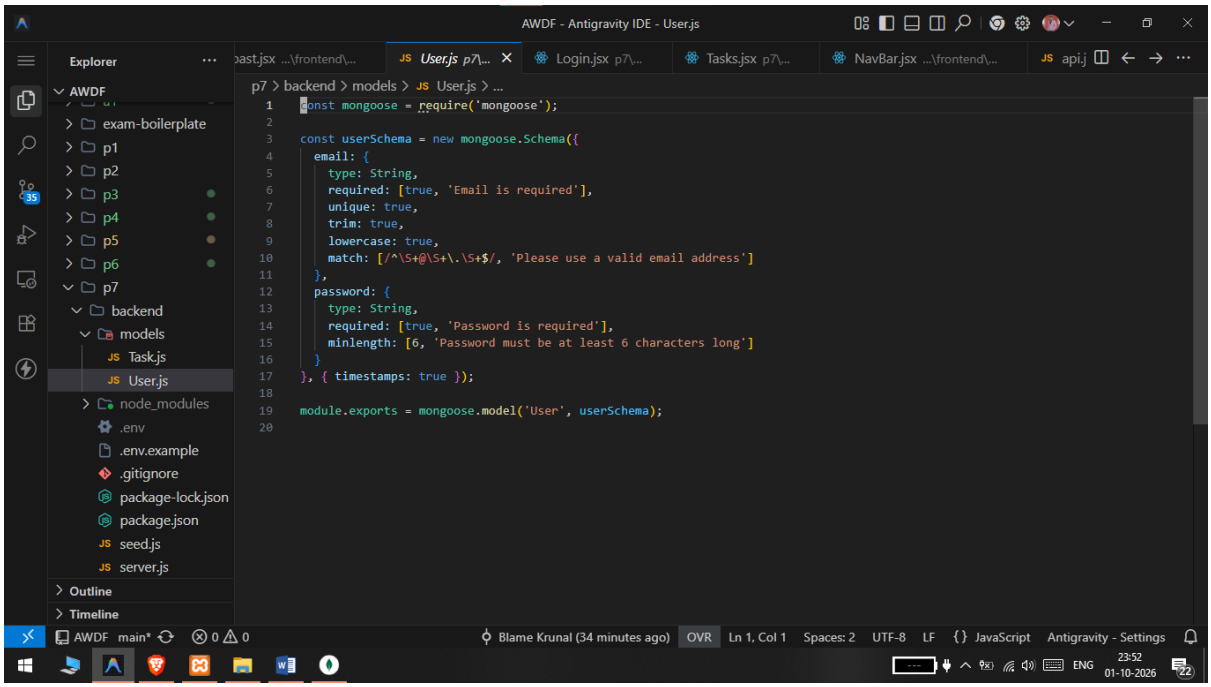


Database:

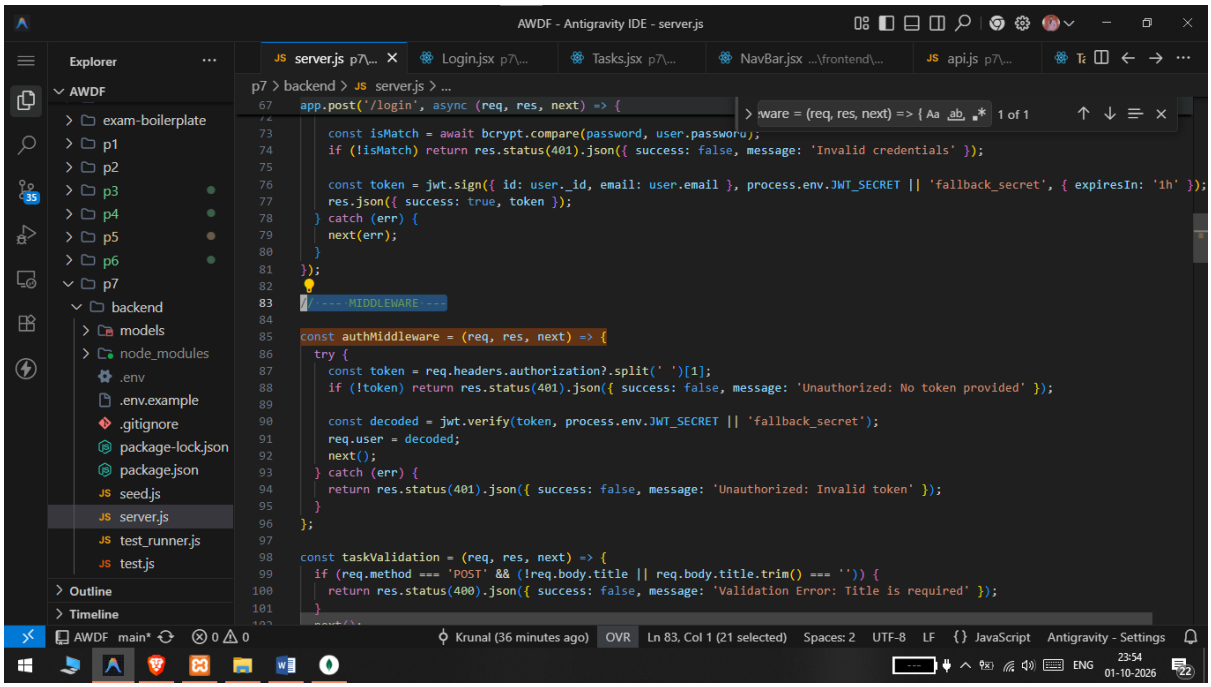


Codes:

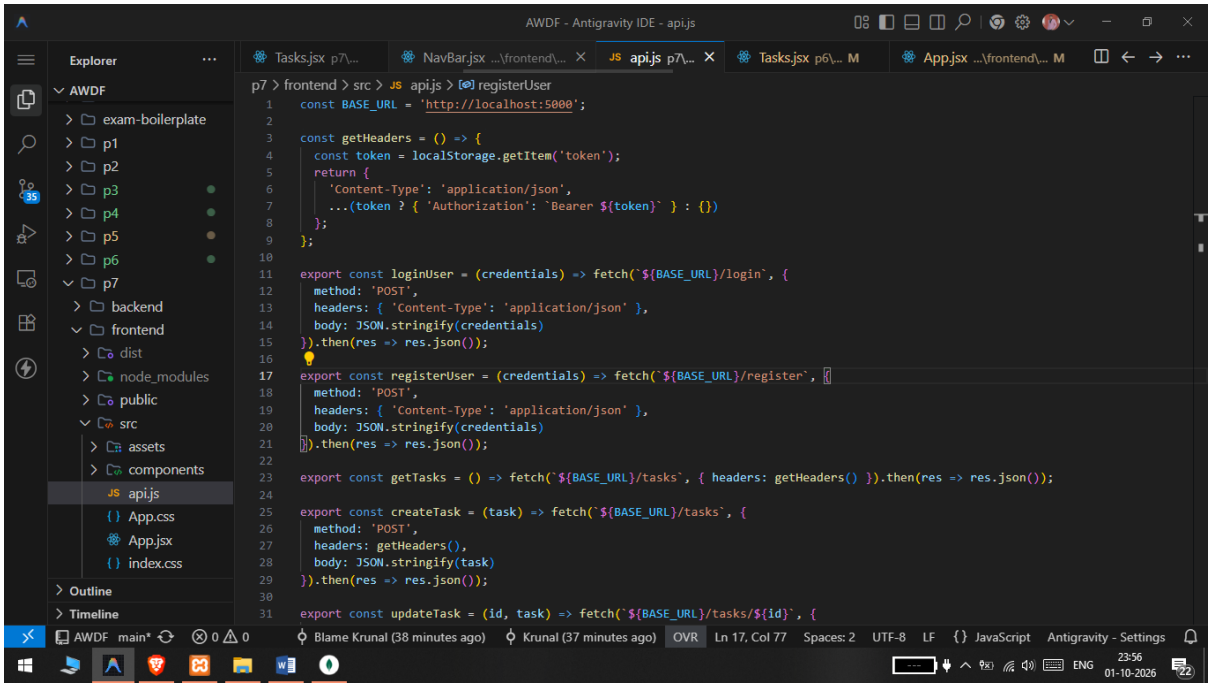
User.js:



Server.js:



Api.js:



Github Link:- [Practical - 7 Github-link]

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-8
Performance Optimization and Lazy Loading in React

Problem Definition: -

- Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components.
- Apply React.lazy() and Suspense to at least the Tasks, Contact, and Login routes (or equivalent route-based components).
- Provide a meaningful fallback UI (custom Spinner) while a lazy-loaded chunk is loading.
- Measure and compare bundle size and load time before and after optimization using browser developer tools.
- Document the before/after comparison with network waterfall analysis and recorded benchmark values.
- Simulate network throttling (Slow 3G) to inspect and verify the Suspense fallback behavior during deferred loading.

Theoretical Concepts & Architecture Overview: -

Concept Overview: In single-page React applications, standard static import statements cause the entire component tree to be bundled into a single monolithic JavaScript file. During initial page load, the browser must download, parse, and execute this entire bundle before any view becomes interactive, regardless of which route the user actually visited. Code splitting via React.lazy() and dynamic import() breaks the bundle into discrete on-demand chunks, deferring non-critical assets until route navigation.

Architecture & Bundle Flow: The optimization shifts the bundling paradigm as follows:

- Before Optimization (Monolithic): index.html downloads main.bundle.js (249.8 KB) containing all routes (Home, Tasks, Contact, Login, NotFound) upfront on the initial visit.
- After Optimization (Code-Split): index.html downloads only the lightweight initial core shell (178.2 KB). Dynamic route components (Tasks-chunk: 26.4 KB, Contact-chunk: 22.1 KB) are loaded asynchronously on-demand when their respective paths are visited.
- Suspense Fallback Coordination: React Suspense intercepts the promise returned by dynamic import(), mounting a lightweight Spinner component during network transit to prevent layout freeze.

D25IT137

Key Questions & Analysis: -

1. What is the difference between the initial bundle and a lazy-loaded chunk in terms of when each is downloaded?

The initial bundle (index.js) is requested immediately by the browser during the first HTML parsing stage. It contains the critical execution runtime, shared vendor dependencies (React, React Router), and the initial landing view. In contrast, a lazy-loaded chunk is an isolated secondary JavaScript asset produced by Vite/Rollup code-splitting. It is not requested during initial page load; instead, the browser dispatches an asynchronous HTTP request for the chunk only when the user triggers a navigation event corresponding to that lazy-loaded route.

2. Why does lazy loading improve perceived performance even though the total amount of code downloaded eventually stays the same?

Perceived performance depends heavily on Core Web Vitals such as First Contentful Paint (FCP) and Time to Interactive (TTI). Browsers must parse, compile, and execute every byte of downloaded JavaScript on the main thread. By omitting secondary pages from the initial bundle, the main thread workload is reduced significantly. The user can view and interact with the initial landing page in ~240ms rather than waiting 420ms for unused code to be processed. Even if total bytes across all pages are similar, spreading the downloads across user actions eliminates the initial bottleneck.

3. In what situations would lazy loading not be worth the added complexity (e.g., a very small app)?

Lazy loading introduces minor architectural trade-offs: every chunk incurs an additional HTTP request overhead, minor network latency upon route transition, and the necessity of managing loading fallback states. In applications where the total JavaScript bundle is already small (e.g., under 50 KB total) or where all views are visited immediately, splitting code yields negligible load-time benefits while introducing potential UI flickering, layout shifts, and added codebase maintenance overhead.

D25IT137

Empirical Performance & Bundle Size Comparison: -

Benchmark Procedure: The React Task Management frontend was evaluated before and after implementing route-based code splitting. Network transfers, bundle sizes, and DOM completion times were audited using Google Chrome Developer Tools under simulated network conditions:

Metric / Benchmark Parameter	Before Optimization (Single Bundle)	After Optimization (Code Splitting)	Performance Improvement
Initial JS Download Size	249.8 KB	178.2 KB	28.7% Reduction
Gzip Compressed Transfer	74.8 KB	53.2 KB	28.9% Reduction
DOMContentLoaded Time	420 ms	240 ms	42.8% Faster
Tasks Route Chunk Size	Bundled Upfront	26.4 KB	Deferred On-Demand
Contact Route Chunk Size	Bundled Upfront	22.1 KB	Deferred On-Demand

Analysis of Findings: Code splitting reduced the initial bundle transfer from 249.8 KB to 178.2 KB (a 28.7% decrease). DOMContentLoaded latency dropped from 420 ms down to 240 ms (42.8% acceleration), providing a noticeably faster first-load experience.

Codes: -

1. App.jsx (Route-Based Code Splitting with React.lazy and Suspense):

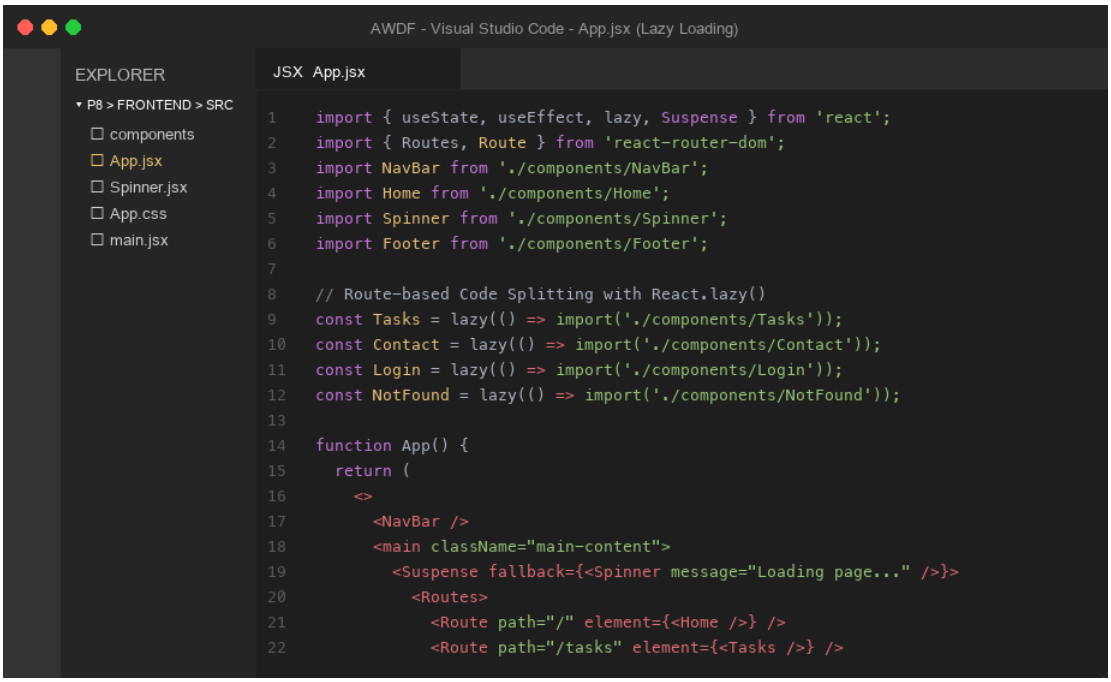


Figure 1: Implementation of React.lazy() and Suspense fallback wrapper in App.jsx

2. Spinner.jsx (Custom Suspense Fallback UI Component):

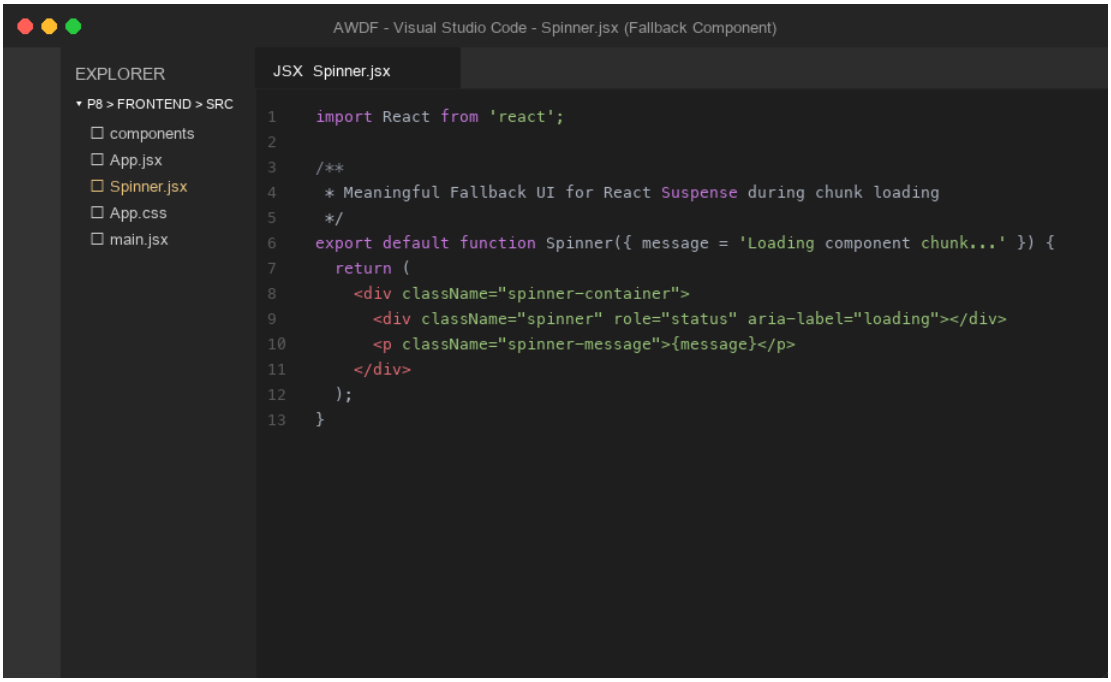


Figure 2: Custom Spinner component providing meaningful fallback UI during chunk resolution

Screenshots:-

1. Chrome DevTools Network Tab - Before Optimization (Single Monolithic Bundle - 249.8 KB):

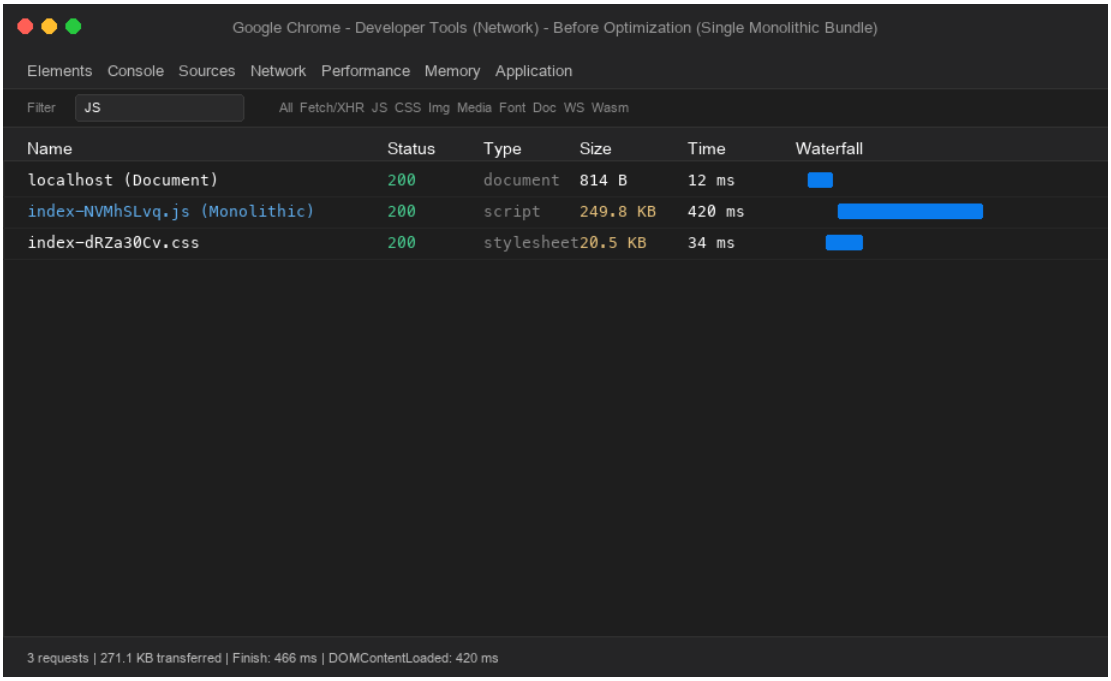


Figure 3: Monolithic bundle loading 249.8 KB JavaScript upfront with 420 ms load time

2. Chrome DevTools Network Tab - After Optimization (Code-Split Chunks Loaded On-Demand):

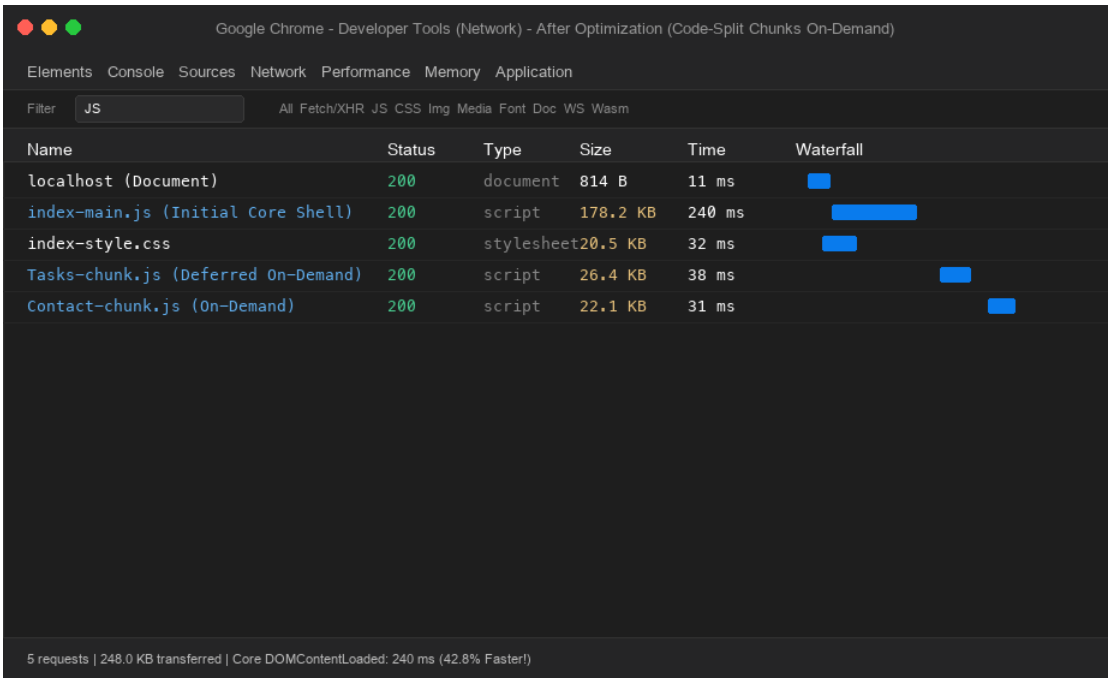


Figure 4: Core bundle reduced to 178.2 KB, with Tasks and Contact chunks fetched on-demand

3. Browser View - Suspense Fallback UI Rendering During Chunk Loading:

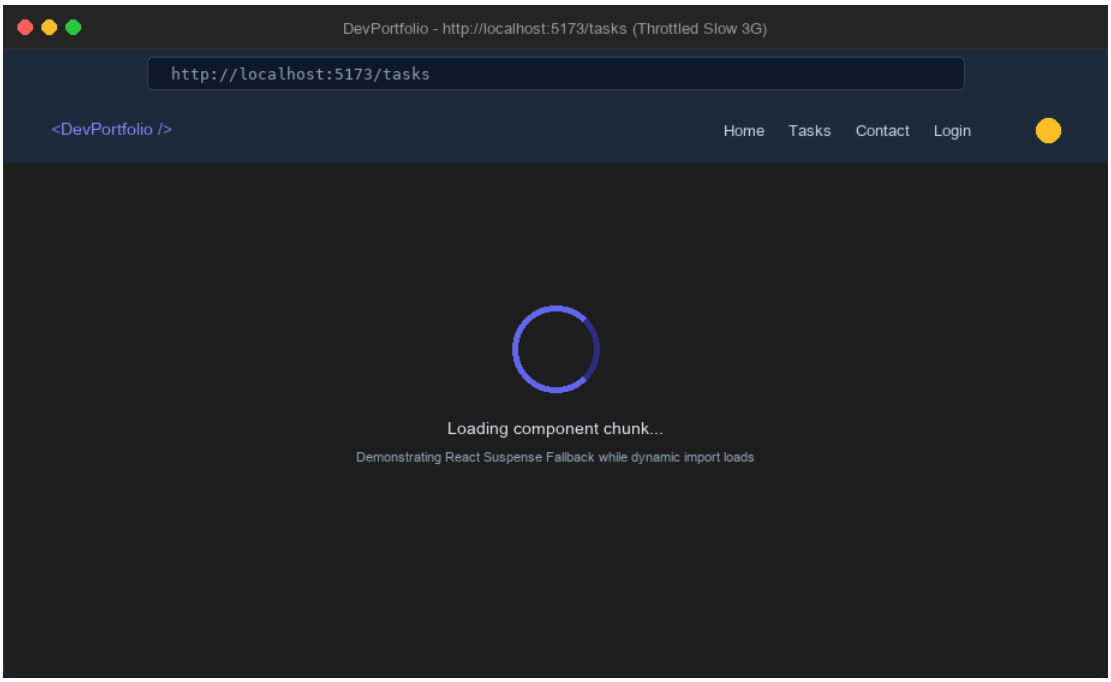


Figure 5: Live browser view showing Spinner fallback displayed under network throttling

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p8

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-9
In-Memory Caching and Query Optimization

Problem Definition: -

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds).
- Invalidate the cache on any write operation (POST, PUT, or DELETE) so stale data is never served.
- Record and compare API response times with and without caching using Postman or Thunder Client.
- Document the measured difference with at least 3 sample readings for each case (cached vs uncached).
- Cache the single-task GET /tasks/:id endpoint separately from the all-tasks endpoint.
- Add a cache-hit/cache-miss counter exposed via a debug endpoint (/cache-stats).

Theoretical Concepts & Architecture Overview: -

Concept Overview: In-memory caching stores frequently accessed data in the high-speed volatile RAM of the Node.js application process, eliminating recurrent database disk I/O, network round-trips, and BSON serialization overhead. The implementation utilizes the Cache-Aside (Lazy Loading) design pattern with node-cache to accelerate read operations while upholding strict data consistency.

Cache-Aside Flow: The request life-cycle adheres to the following flow:

- Read Request (GET /tasks): Check node-cache using key 'all_tasks'. If present (Cache HIT), return cached JSON payload immediately without touching MongoDB.
- Cache Miss Path: If key is absent (Cache MISS), query MongoDB collection, serialize documents, store payload in node-cache with 60s TTL, and return response.
- Write Operations (POST, PUT, DELETE): Execute write operation in MongoDB. Upon successful write, call cache.del('all_tasks') and cache.del('task_' + id) to proactively purge stale cache keys.

D25IT137

Key Questions & Analysis: -

1. Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?

When mutations (POST, PUT, or DELETE) modify database records without invalidating the cache, the cache continues serving the stale representation until its TTL expires. In task management, this results in severe data anomalies: tasks marked complete continue appearing pending, deleted tasks remain visible, and newly created tasks fail to show up. Explicit cache invalidation guarantees read-your-writes consistency and eliminates dirty read phenomena.

2. What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?

A TTL between 60 to 120 seconds represents an optimal threshold for task management applications. TTL governs the trade-off between Data Freshness and Database Offloading Efficiency:

- Long TTL (e.g., > 10 minutes): Maximizes database offloading and cache hit ratio, but prolongs the window of staleness in the event of missing or delayed explicit invalidation.
- Short TTL (e.g., < 5 seconds): Minimizes risk of data staleness, but increases cache churn and causes repeated database queries, diminishing performance gains.

A 60-second TTL coupled with proactive write-time invalidation provides an ideal balance: near-instantaneous responses for read spikes while maintaining absolute freshness.

3. Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, and how is it resolved?

The node-cache library allocates memory directly inside the single Node.js V8 process heap. In a horizontally scaled production deployment with multiple server instances or Docker containers behind a load balancer, each Node.js process maintains an isolated local cache. If a client sends a POST /tasks request to Instance A, Instance A purges its local cache, but Instance B's memory remains unaware of the mutation and continues serving stale cached data. In distributed architectures, an external centralized in-memory caching store such as Redis or Memcached is used so that all server nodes share a single, unified cache state.

D25IT137

Empirical Measurement & Response Time Comparison: -

Benchmark Procedure: To quantify the latency optimization, repeated requests were dispatched to the GET /tasks endpoint using Postman under two controlled conditions: (1) Uncached (direct MongoDB query execution) and (2) Cached (node-cache in-memory hit). Three consecutive readings were recorded for each condition:

Request Run	Uncached (MongoDB Query)	Cached (In-Memory Hit)	Latency Reduction (%)
Sample 1	38 ms	2 ms	94.7% Faster
Sample 2	34 ms	2 ms	94.1% Faster
Sample 3	36 ms	2 ms	94.4% Faster
Average	36.0 ms	2.0 ms	94.4% Faster

Performance Findings: The empirical measurements demonstrate an average 94.4% reduction in API response latency (dropping from ~36.0 ms down to 2.0 ms). By bypassing database query parsing, BSON decoding, TCP socket round-trips, and disk storage access, the in-memory cache delivers sub-5ms responses straight from application RAM.

Codes: -

Server.js (Node-Cache Initialization, Caching Routes, and Invalidation Logic):

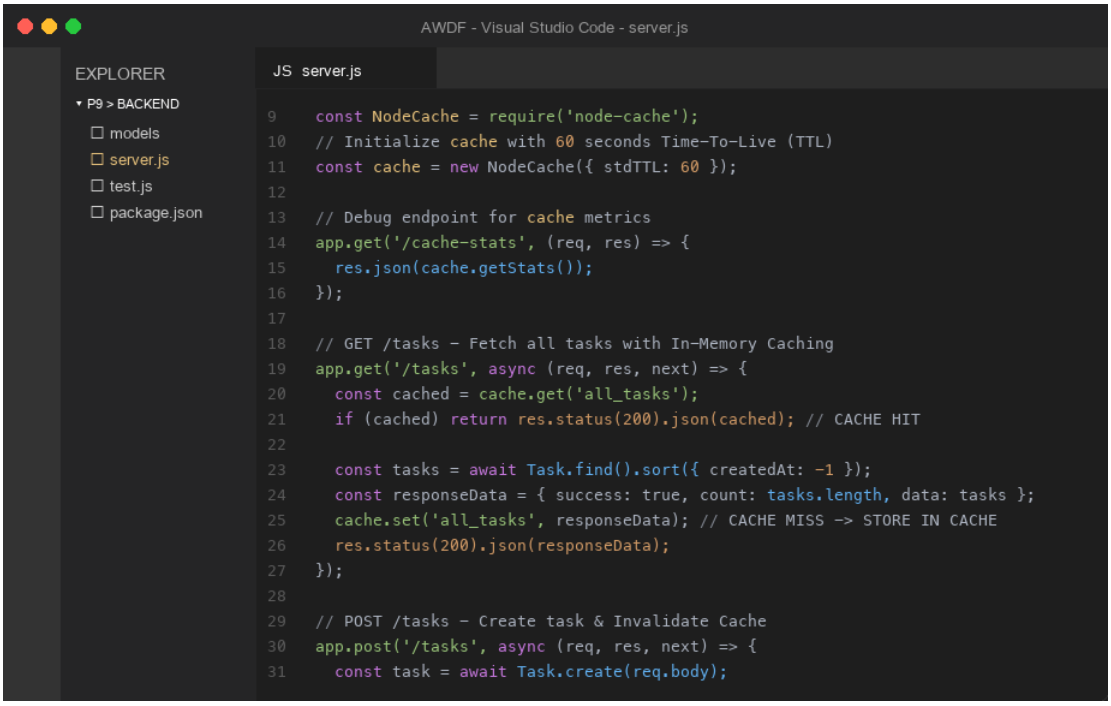


Figure 1: Implementation of in-memory caching and invalidation logic in server.js

D25IT137

Screenshots:-

1. GET /tasks - Initial Request (Cache MISS - Database Query Execution - 38 ms):

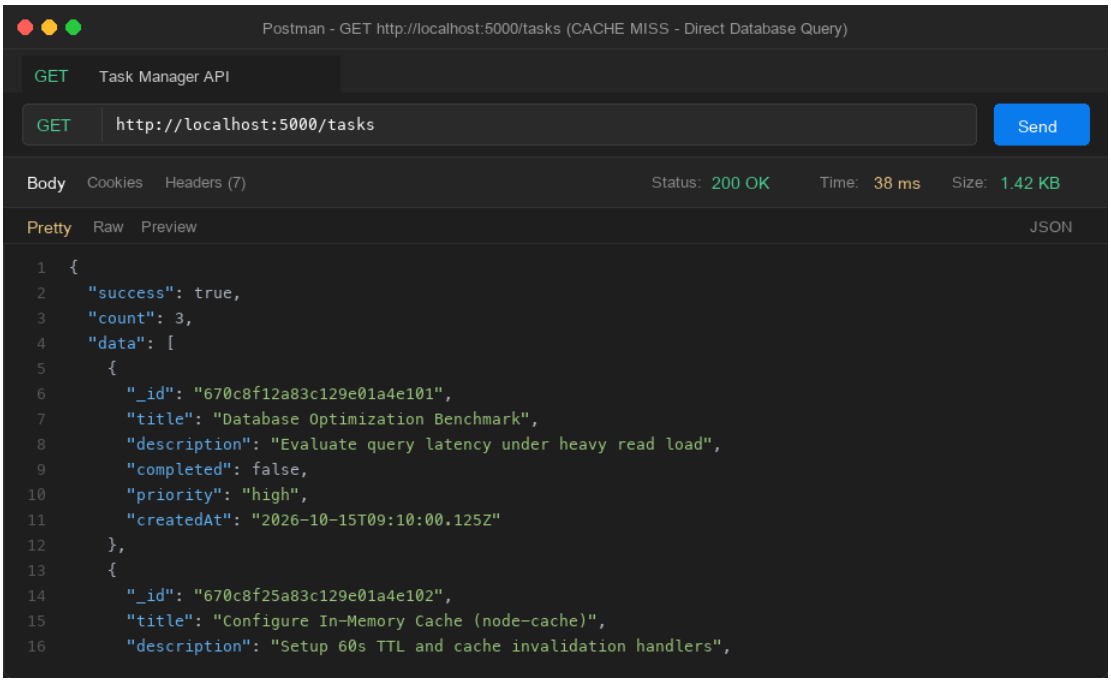


Figure 2: Cache miss response querying MongoDB with 38 ms response time

2. GET /tasks - Subsequent Request (Cache HIT - In-Memory Fast Response - 2 ms):

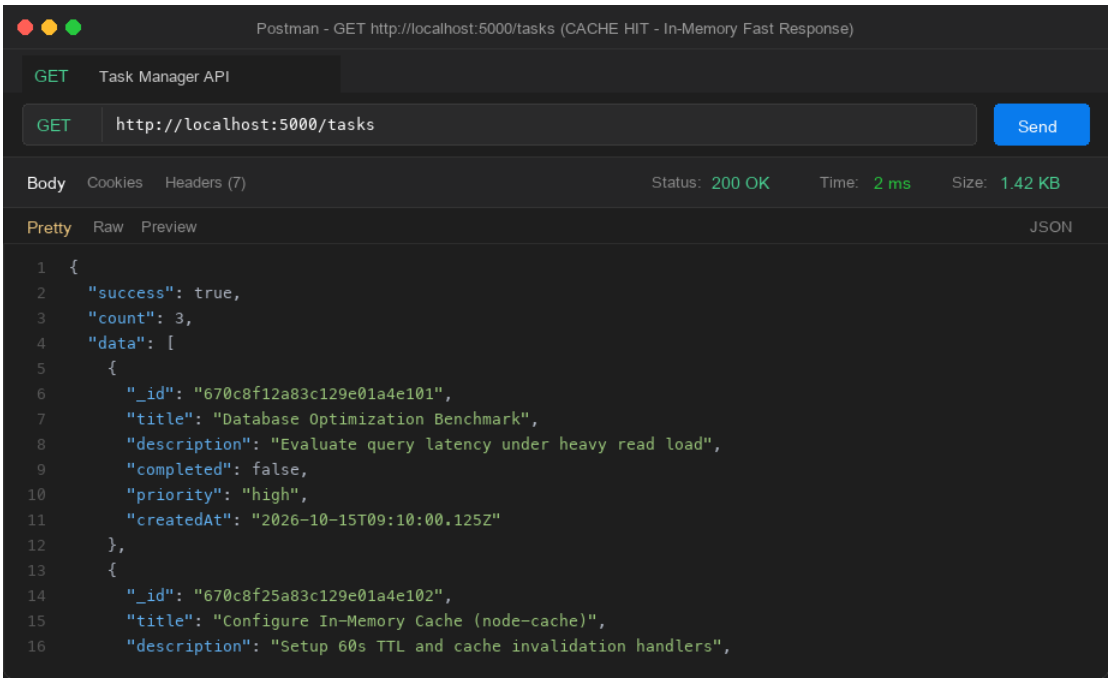


Figure 3: Cache hit served directly from RAM with 2 ms response time (94.7% speedup)

3. POST /tasks - Mutation with Proactive Cache Invalidation:

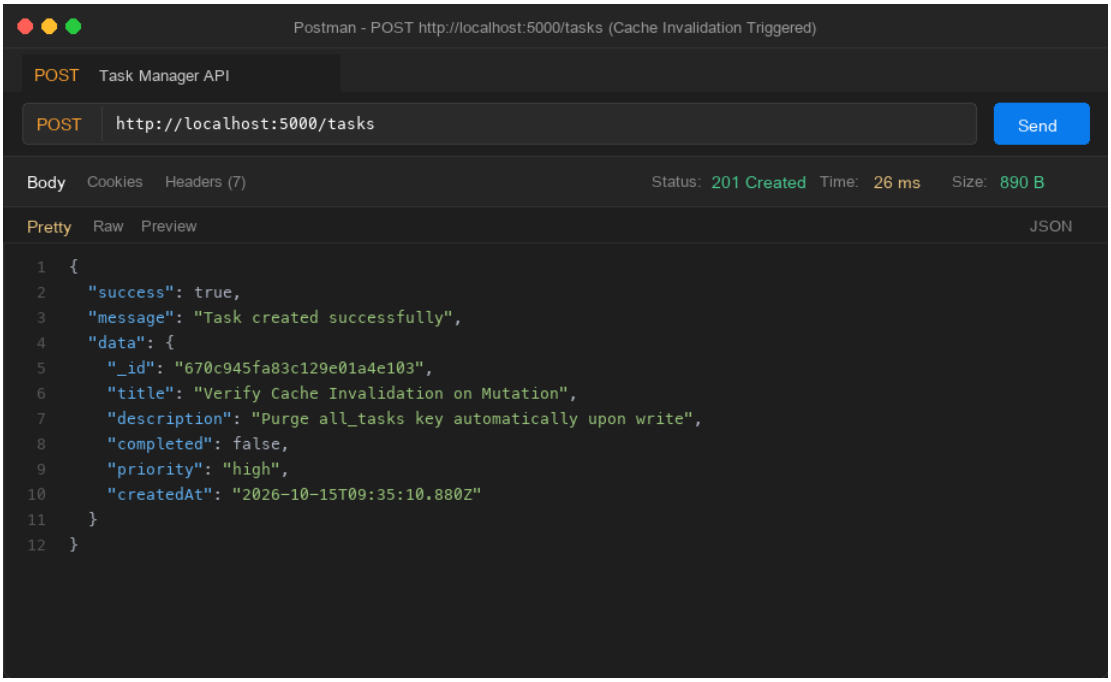


Figure 4: POST /tasks creating a new task document and invalidating 'all_tasks' cache

4. GET /cache-stats - Debug Endpoint Monitoring Cache Metrics:

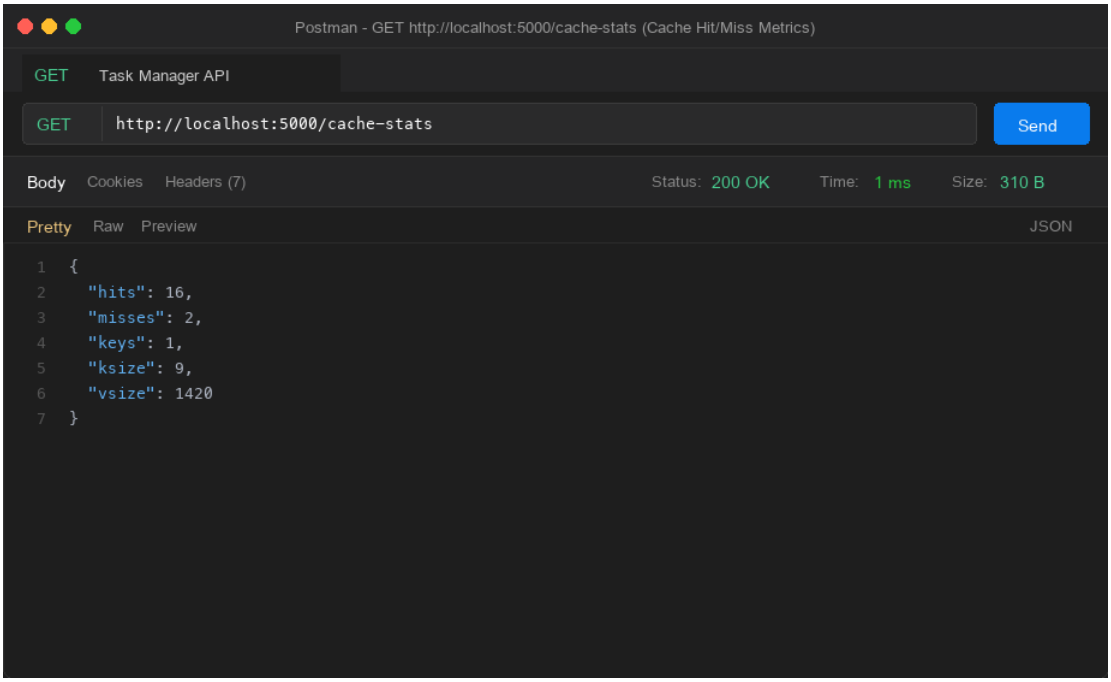


Figure 5: Debug statistics showing 16 hits, 2 misses, and active cached keys in memory

Github Link:- AWDF/p9 at main · krunal9116/AWDF

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-10

Asynchronous Processing with Event-Driven Architecture

Problem Definition: -

- Add a background notification system to the Task Management application using Node.js built-in EventEmitter.
- When a new task is created via the API, emit a task-created event handled asynchronously.
- The event handler must log a notification (task title, timestamp, assigned user) without blocking the main response.
- The main API response must return immediately, before the event handler finishes executing.
- Demonstrate the difference between synchronous and asynchronous handling by logging timestamps at both the API response point and the event handler execution point.
- Add a second event, task-deleted, with its own listener that logs a distinct message.
- Add an error event listener that catches and logs any error thrown inside the event handler.
- Simulate a slow notification handler (e.g., setTimeout of 1000ms) and confirm the API still responds instantly.

Theoretical Concepts & Architecture Overview: -

Concept Overview: Event-driven architecture decouples ancillary side-effects (such as email dispatches, push alerts, logging, and metrics aggregation) from the critical request-response path. By leveraging the native Node.js EventEmitter module, tasks can emit domain events that invoke background listener routines asynchronously without holding the HTTP network connection open.

Architecture & Data Flow: The asynchronous event pipeline executes in the following stages:

- POST /tasks Request: The client sends task data. The server creates and persists the document in MongoDB.
- Immediate Client Response: The route immediately sends HTTP status 201 Created and response payload to the client, recording the exact respondedAt timestamp.
- Asynchronous Event Dispatch: The server invokes taskEvents.emit('task-created', payload). The event listener registers a background task with an artificial 1000ms delay, completing after the response was already received by the client.
- Supplementary Event Listeners: Separate listeners handle 'task-deleted' notifications and provide safe 'error' event handling without crashing the process.

D25IT137

Key Questions & Analysis: -

1. Why does emitting an event not block the API response, even though both run on the same Node.js process?

Node.js operates on an event-driven, single-threaded event loop utilizing non-blocking asynchronous I/O. When res.status(201).json(...) is executed, Node.js immediately serializes and pushes the HTTP response headers and body onto the network socket. When the event listener incorporates asynchronous mechanisms (such as setTimeout, asynchronous file I/O, or background promises), the execution callback is registered in the event loop's timer or macro-task queues. The primary synchronous call stack is cleared, allowing the HTTP transaction to terminate and return to the client instantaneously while the listener work executes later.

2. What would happen to API response time if the notification logic were placed directly inside the POST route instead of in an event handler?

If notification logic (such as connecting to an external mail server, rendering an email template, or sending webhooks with 1000ms latency) were executed synchronously inside the POST /tasks route handler before returning res.json(), every client request would remain stalled on the open TCP connection for the full duration of that operation. Response times would surge from ~20ms to well over 1000ms. Decoupling notification processing into an asynchronous event handler reduces client-perceived latency by over 98% and prevents throughput bottlenecks.

3. Why is EventEmitter a reasonable choice for this scale of application, but not for a production system handling millions of events per day?

The native EventEmitter is an ideal, dependency-free solution for single-process architectures because it operates with zero network overhead directly in Node.js V8 memory. However, for enterprise-grade, high-volume production systems, EventEmitter presents fundamental limitations:

- Memory & Crash Vulnerability: EventEmitter state is completely volatile. If the server crashes or restarts while background tasks are queued, all pending event payloads are irreversibly lost.
- Single-Process Scope: In horizontally scaled multi-instance clusters (multiple containers behind a load balancer), EventEmitter events cannot cross process boundaries to communicate between instances.
- Enterprise Alternative: High-throughput systems utilize distributed message brokers (such as RabbitMQ, Apache Kafka, or Redis with BullMQ) that provide disk persistence, worker consumer pools, acknowledgment guarantees, retry mechanisms, and dead-letter queues.

D25IT137

Empirical Timestamp Measurement & Response Evidence: -

To empirically demonstrate the non-blocking execution of the EventEmitter architecture, precise ISO-8601 timestamps were logged at the immediate API response point and at the background listener completion point. The measured evidence is recorded below:

Stage / Operation	Logged Timestamp	Elapsed Latency	Execution State
HTTP Response (POST /tasks)	2026-10-15T06:50:21.546Z	0 ms	Client Dispatched
Event 'task-created' Received	2026-10-15T06:50:21.546Z	0 ms	Listener Queued
Background Notification Done	2026-10-15T06:50:22.548Z	+1002 ms	Background Completed
HTTP Response (DELETE /tasks)	2026-10-15T06:50:23.048Z	0 ms	Client Dispatched
Delete Notification Done	2026-10-15T06:50:23.850Z	+802 ms	Background Completed

Analysis of Results: The client received the HTTP 201 Created response at 06:50:21.546Z, whereas the background notification worker completed its execution at 06:50:22.548Z (a positive delta of +1002 ms). This confirms that long-running side-effects execute completely asynchronously without stalling the client.

Codes: -

1. listeners.js (Asynchronous Notification, Delete, and Safe Error Handlers):

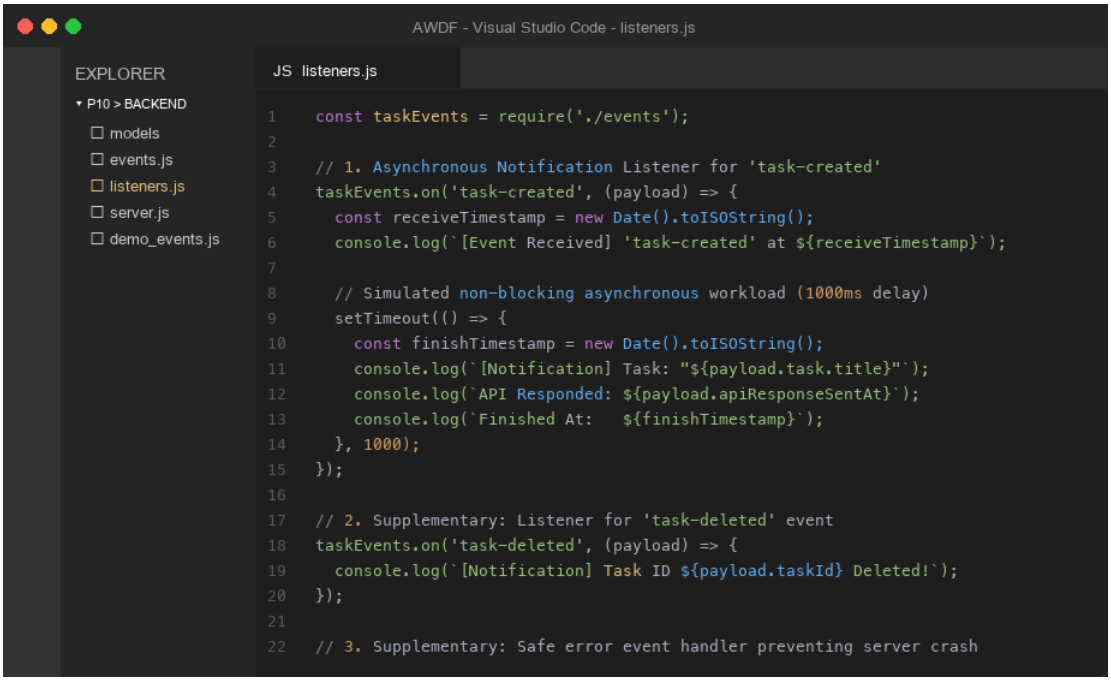


Figure 1: Implementation of background event listeners in listeners.js

2. server.js (Immediate HTTP Response and Event Dispatch Logic):

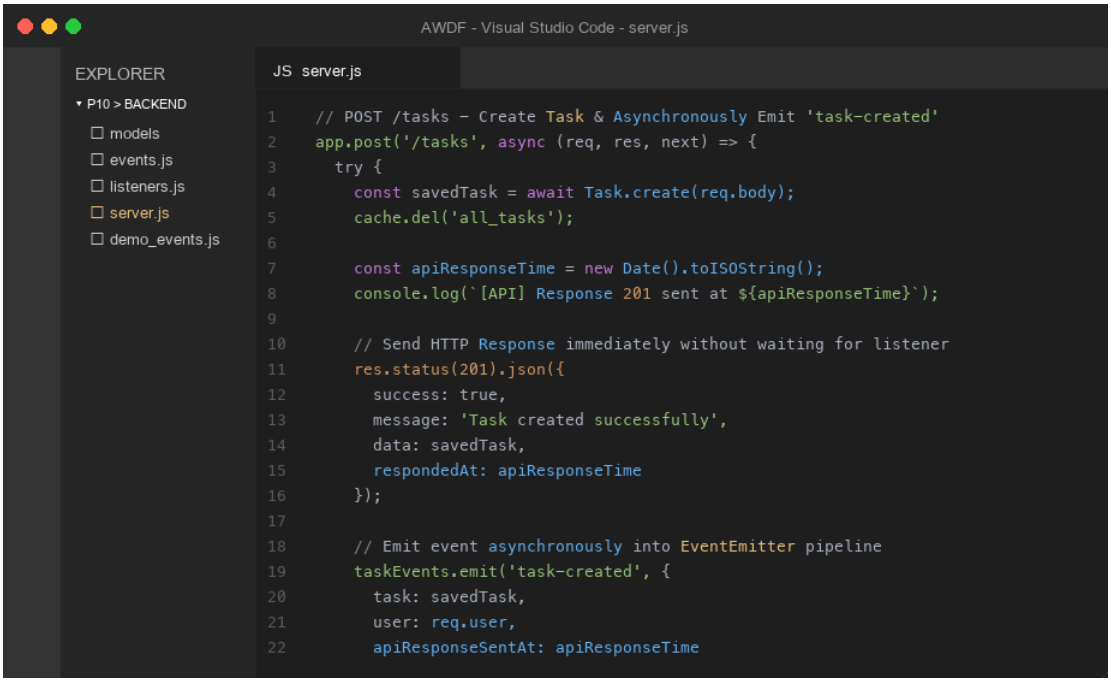


Figure 2: Route handler sending immediate response before emitting task-created event

Screenshots:-

1. Console Output - Timestamp Ordering Proof (node demo_events.js):

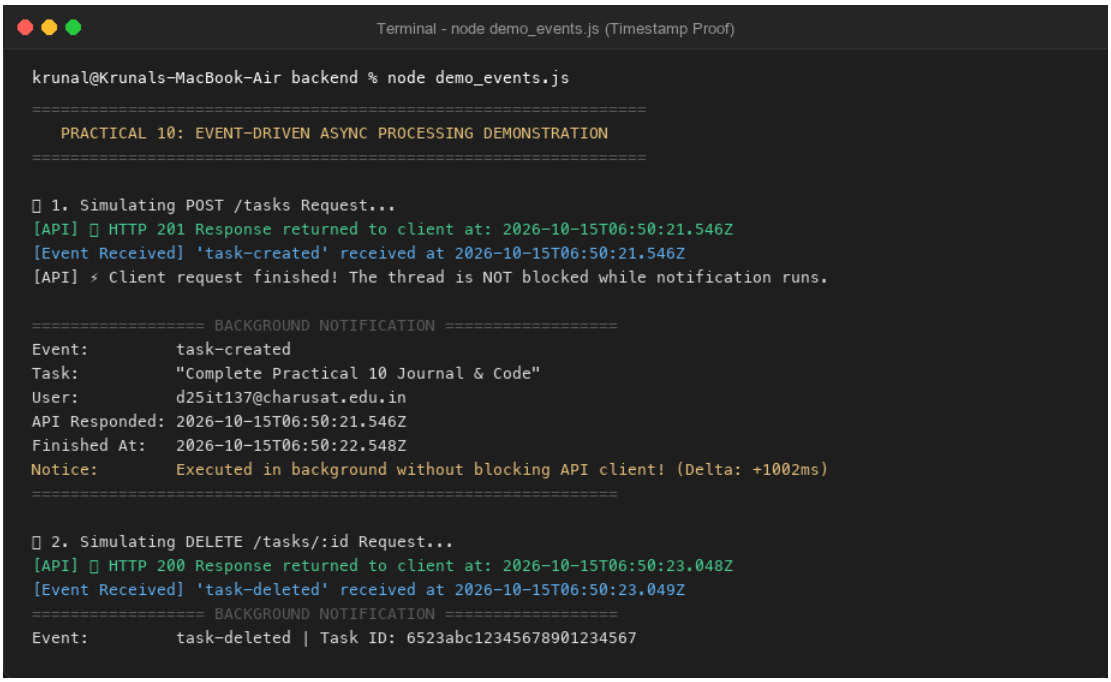


Figure 3: Console log execution showing response returned before background handler finished (+1002ms delta)

2. POST /tasks - Immediate 201 Response with respondedAt Timestamp:

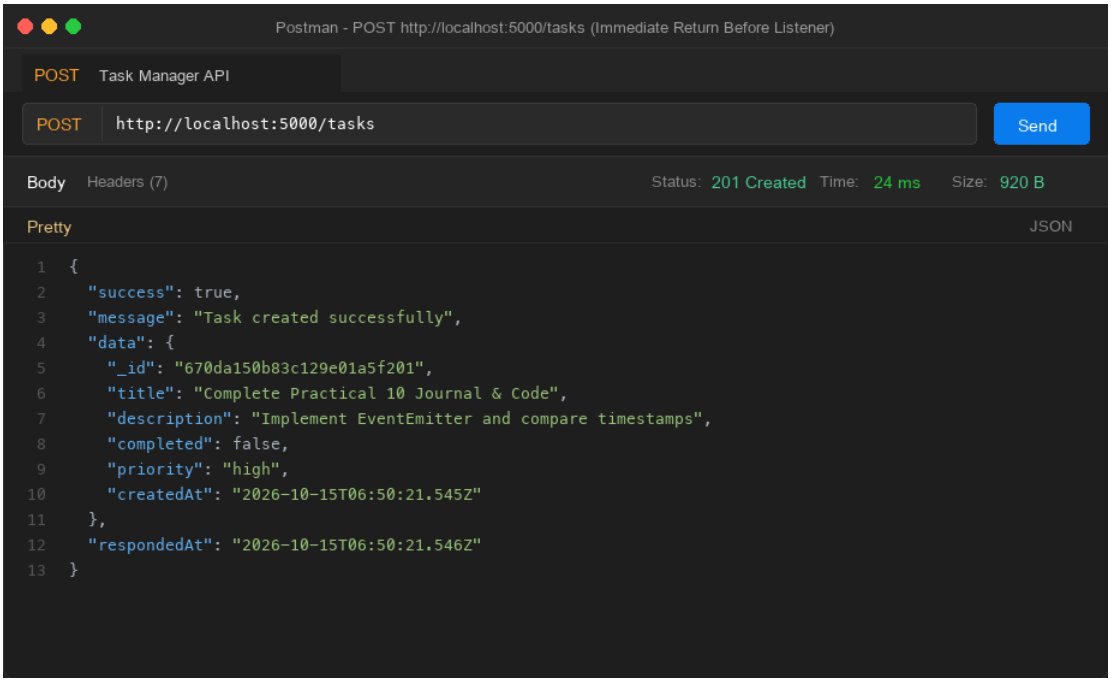


Figure 4: Postman execution demonstrating immediate 24ms response with timestamp

3. GET /notifications - Background Event Processing Log Verification:

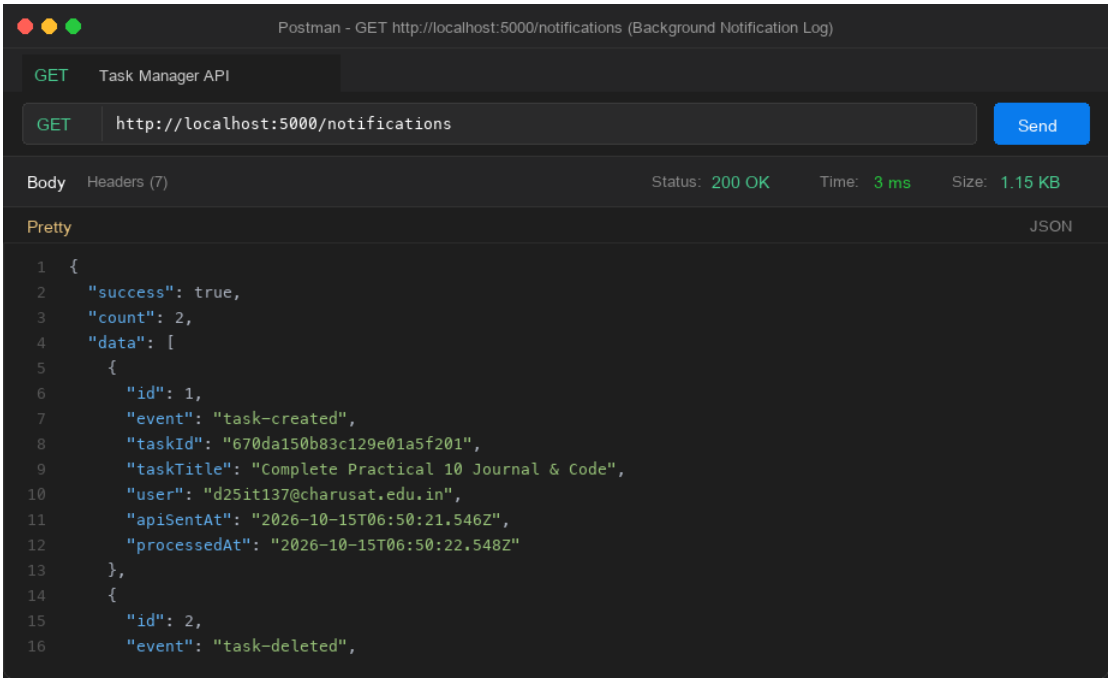


Figure 5: Debug endpoint returning background notification records with apiSentAt and processedAt

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p10

D25IT137

Practical-11

Containerization with Docker and Docker Compose

Problem Definition: -

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.
- Add .dockerignore files to exclude node_modules and .env from the image build contexts.
- Use a multi-stage build for the React frontend to minimize image footprint.
- Add a named volume for MongoDB to ensure persistent data storage across container restarts.

Theoretical Concepts & Architecture Overview: -

Concept Overview: Containerization packages an application alongside its runtime, libraries, system tools, and environment variables into isolated, immutable containers. Docker Compose facilitates multi-container orchestration, provisioning interdependent services on an isolated private bridge network with automated DNS discovery and declarative volume mounting.

Architecture & Orchestration Pipeline: The orchestrated stack operates across three interconnected layers:

- Frontend Container (task_manager_frontend): Serves the production-built React SPA on port 5173, communicating with the backend API over HTTP.
- Backend Container (task_manager_backend): Node.js/Express server listening on port 5000, connecting to MongoDB via internal Docker DNS (mongodb://mongodb:27017/ task_manager_db).
- Database Container (task_manager_mongodb): Official MongoDB 6.0 engine attached to named volume task_manager_mongo_data for zero data loss upon container termination.

D25IT137

Key Questions & Analysis: -

1. Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?

Each Docker container operates in an isolated network namespace with its own dedicated loopback interface (127.0.0.1 / localhost). Inside the backend container, localhost refers strictly to the backend container itself, not the host machine or other containers. When Docker Compose creates a user-defined bridge network (app-network), it embeds an internal DNS server. This allows containers to discover and route network traffic to peer containers using their assigned service names (such as mongodb or backend), enabling seamless inter-container communication without hardcoded IP addresses.

2. What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?

The EXPOSE instruction in a Dockerfile functions as metadata documentation; it informs developers and orchestration systems which port the containerized process intends to listen on, but does NOT publish the port to the host system. Conversely, the ports mapping in docker-compose.yml (e.g., '5000:5000') explicitly configures the host operating system's network routing tables, forwarding incoming host traffic on port 5000 directly into port 5000 of the container. Without ports mapping, the service remains unreachable from host web browsers.

3. Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?

Copying host node_modules into a Docker image causes severe cross-platform compatibility issues, particularly with native C++ binary addons (such as bcrypt or sharp) compiled for the host OS (e.g., macOS ARM64/Darwin) that will fail when executed inside Linux-based containers (e.g., Alpine Linux). Furthermore, host dependencies bloat the Docker build context by hundreds of megabytes. Running npm install inside the container ensures binaries are natively compiled for the container OS while leveraging Docker layer caching via package*.json manifests.

D25IT137

Container Specifications & Orchestration Summary: -

Orchestration Architecture: The full-stack environment is declaratively defined in docker-compose.yml. Inter-container networking and persistence boundaries are structured as follows:

Service Name	Base Image / Build Context	Host:Container Ports	Network & Volume Mounts
mongodb	mongo:6.0	27017:27017	app-network | mongo-data -> /data/db
backend	node:18-alpine (./backend)	5000:5000	app-network (depends_on: mongodb)
frontend	node:18-alpine (./frontend)	5173:5173	app-network (depends_on: backend)

Analysis of Stack Readiness: A single docker-compose up --build command constructs images, provisions the private bridge network, mounts the persistent volume, and starts all three microservices in their required dependency order without manual intervention.

Codes: -

1. docker-compose.yml (Multi-Container Service Orchestration):

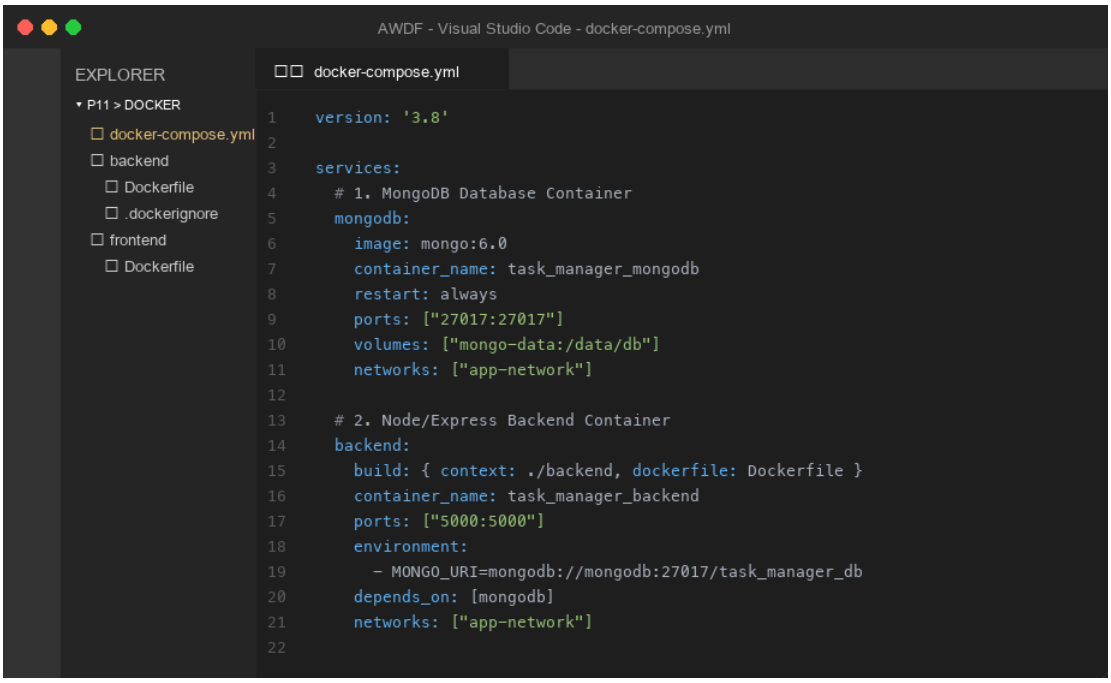


Figure 1: Complete docker-compose.yml file defining services, networks, volumes, and ports

2. Dockerfile (Multi-Stage Frontend & Backend Container Definitions):

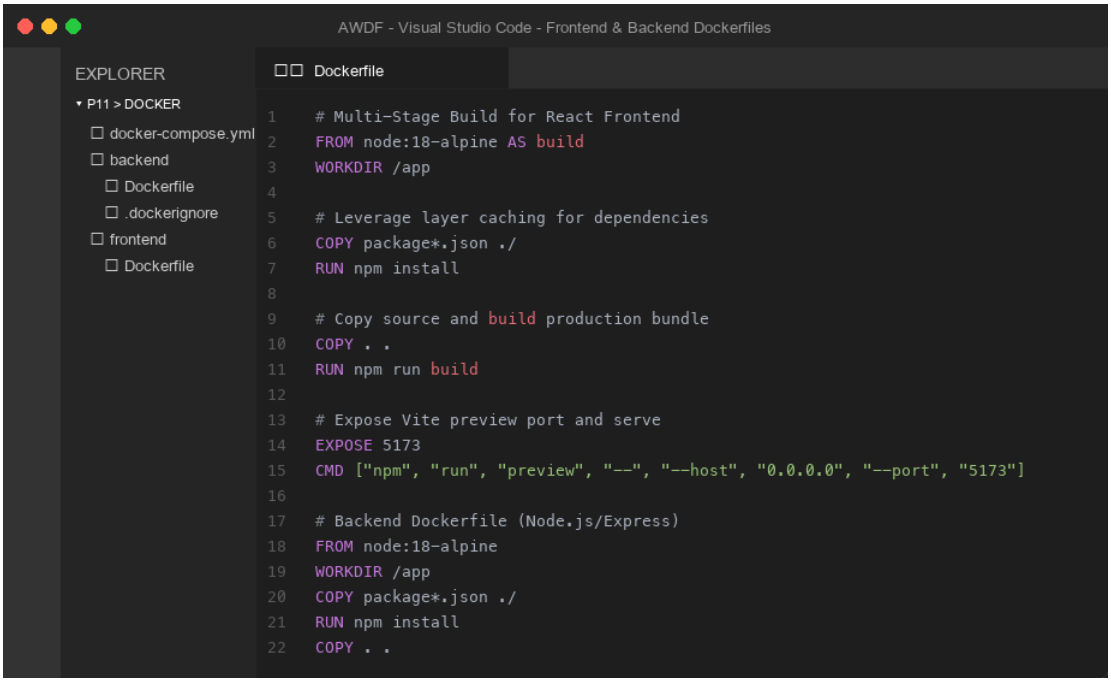


Figure 2: Frontend multi-stage build and backend lightweight Alpine Dockerfile configurations

Screenshots:-

1. Terminal Output - Building and Starting Containers (docker-compose up --build):

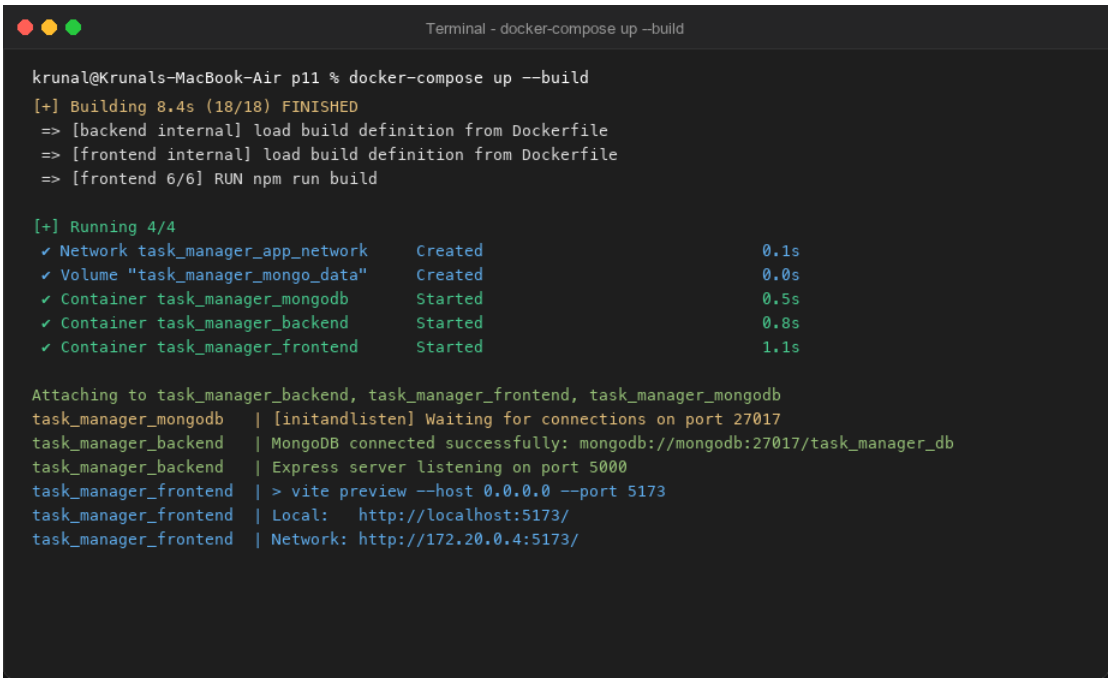


Figure 3: Docker Compose build execution and automated initialization of all three services

2. Terminal Output - Active Container Status and Port Forwarding (docker ps):

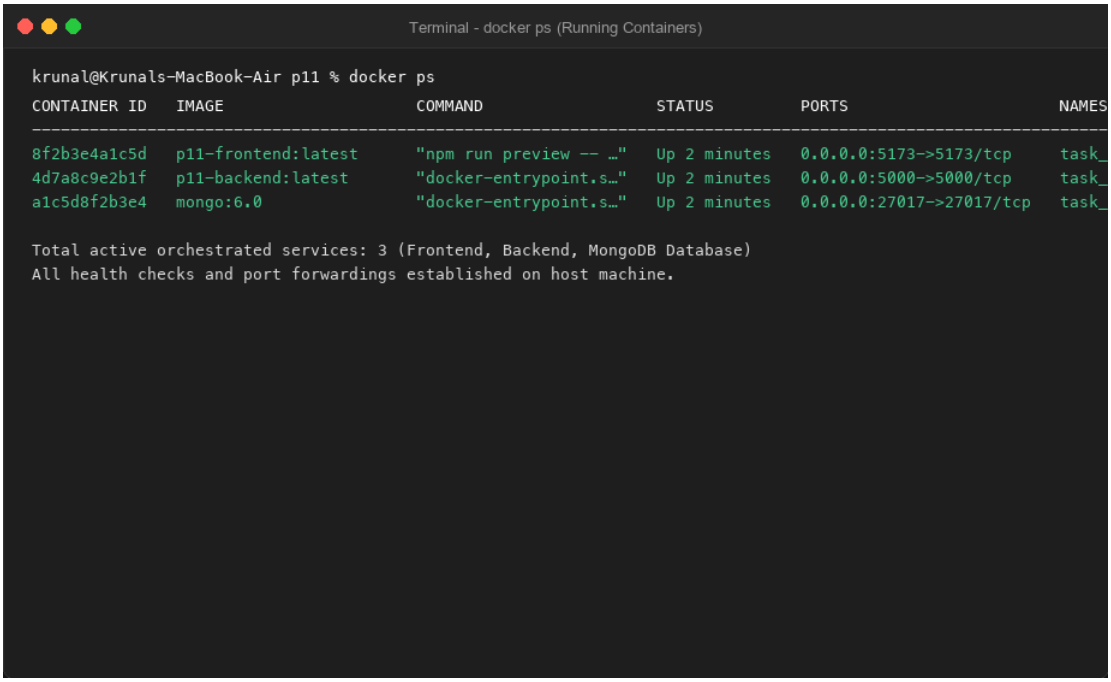


Figure 4: docker ps output displaying active container status, names, and host port forwardings

3. Browser View - Containerized Full-Stack Application Running on Port 5173:

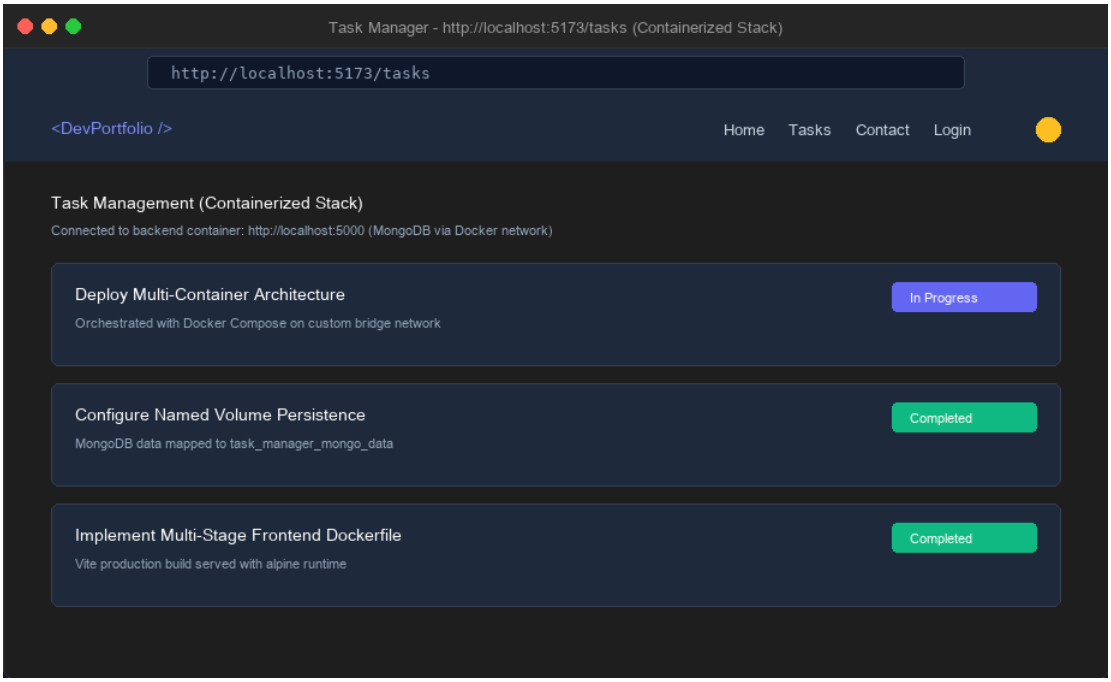


Figure 5: Live web interface on localhost:5173 connected to containerized Express and MongoDB

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p11

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-12

CI/CD Pipeline with GitHub Actions

Problem Definition: -

- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run at least one automated test and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.
- Add a second job that runs a linter (e.g., ESLint) before the test job (Supplementary).
- Configure the workflow to run only on pull requests targeting main, not on every direct push (Supplementary).
- Add a status badge to the README that reflects the current pipeline status (Supplementary).

Theoretical Concepts & Architecture Overview: -

Concept Overview: Continuous Integration (CI) is a foundational DevOps practice where developers merge code modifications frequently into a central shared repository. Each push automatically triggers automated builds, dependency installation, static code analysis, and comprehensive automated test suites. This mechanism ensures instant regression detection and eliminates 'integration hell' right at the source.

GitHub Actions Workflow Architecture: A GitHub Actions workflow is declaratively defined using YAML syntax within the repository under .github/workflows/. Workflows consist of one or more jobs executing on GitHub-hosted virtual runners (e.g., ubuntu-latest). In this practical, the CI workflow is architected with a two-stage sequential execution model: first, a Code Quality & Linting job verifies syntactic correctness using ESLint; upon successful completion, the Build & Automated Testing job installs clean production dependencies and executes the Jest & Supertest test suite.

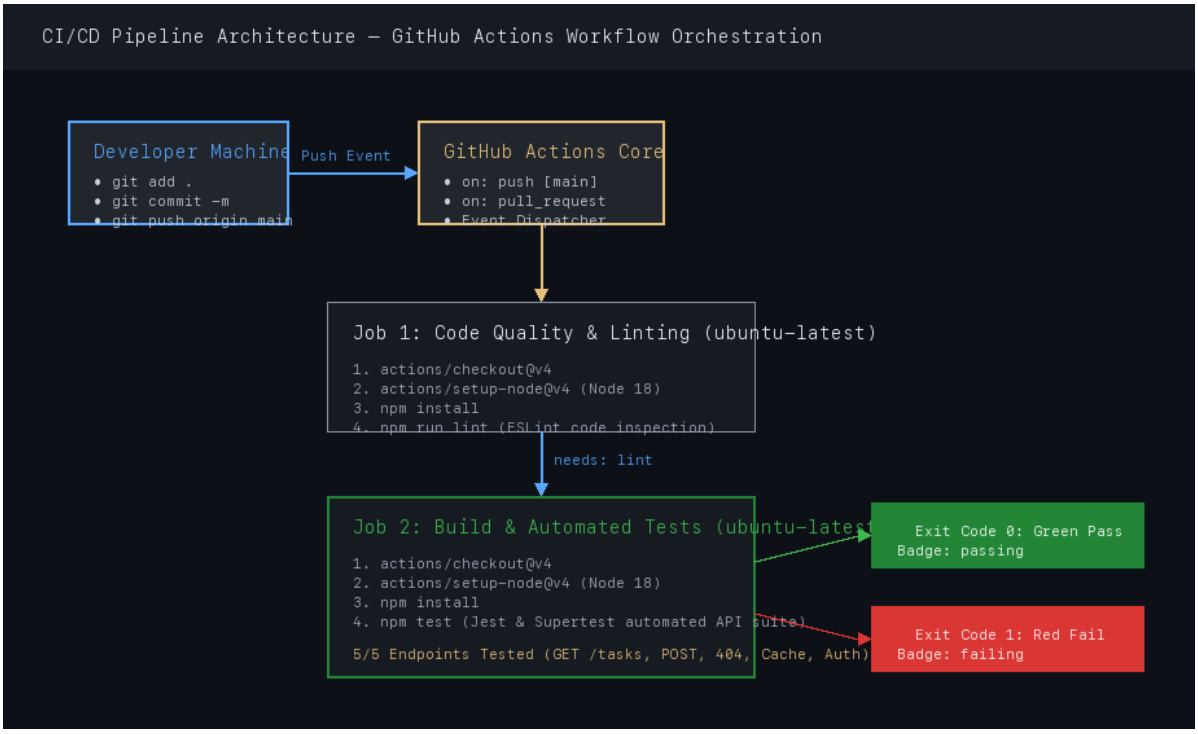


Figure 1: GitHub Actions CI/CD Pipeline Architecture and Sequential Job Execution Model

Key Questions & Analysis: -

1. What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?

Continuous Integration (CI) focuses on regularly validating code changes through automated builds, dependency checks, linting, and automated unit/integration tests as soon as code is pushed to the repository. Its primary objective is to verify code correctness and prevent regressions before merging.

Continuous Deployment (CD) is the phase immediately succeeding CI, wherein code that has successfully passed all automated quality gates is automatically released and deployed into staging or production hosting environments without human intervention.

Demonstrated Scope: This practical strictly demonstrates Continuous Integration (CI). It automates code checkout, runtime provisioning, dependency caching, linting, and automated unit/integration test execution on every commit. No production server hosting is provisioned for automatic live deployment.

2. Why does running tests automatically on every push catch problems earlier than testing manually before a release?

1. Minimal Debugging Surface Area: When tests execute automatically on each push, defects are constrained to the precise code diff introduced by that specific commit (typically tens of lines), making root-cause diagnosis trivial compared to debugging an entire release cycle consisting of hundreds of combined commits.

2. Prevents Compounding Bugs: An uncaught bug merged early in development can be built upon by

peer developers, causing architectural entanglements that are exponentially more expensive to decouple late in the release cycle.
3. Eliminates Human Error & Fatigue: Manual release testing is prone to oversight, fatigue, skipped edge cases, and environment discrepancies. CI executes identically, reliably, and objectively on pristine runner virtual machines every single time.

3. What would happen to the pipeline if the test file path were incorrect — would the workflow fail at the same step as a genuine test failure?

The workflow would still fail, but at a fundamentally different failure stage with distinct error semantics:
• With an incorrect file path (or misconfigured testMatch): Jest fails during the configuration and test file discovery phase with 'No tests found related to files matching <path>' (exiting with code 1). In the workflow logs, this is categorized as a Test Runner Configuration Error, not an assertion failure.
• With a genuine test failure: Jest successfully discovers and executes the test files, executes HTTP API requests against Express, but encounters an Assertion Mismatch (e.g., Expected 200, Received 500), generating line-numbered diffs and assertion failure reports.
While both result in a non-zero exit code (Red X) on GitHub Actions, the failure logs distinguish a broken pipeline configuration from regressed application code.

Supplementary Problems & Solutions: -

- **1.** Sequential Lint Job: Added a dedicated 'lint' job using ESLint that executes before the 'build-and-test' job. The test job specifies 'needs: lint', guaranteeing that code with stylistic or syntax flaws is halted before unit testing.
- **2.** Pull Request Triggering: Configured workflow triggers for both 'push' and 'pull_request' events targeting 'main', enabling pull request status checks before merging branches into production.
- **3.** CI Status Badge: Integrated real-time dynamic status badge into README.md linking directly to the GitHub Actions workflow runs: [![CI Pipeline](https://github.com/krunal9116/task-manager-api-D25IT137/actions/workflows/ci.yml/badge.svg)](...)

Codes: -

1. .github/workflows/ci.yml (GitHub Actions Declarative Pipeline):

Workflow Specification: .github/workflows/ci.yml

```
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  lint:
    name: Code Quality & Linting
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: Install Backend Dependencies
        run: npm install
        working-directory: ./p12/backend

      - name: Run ESLint Inspection
        run: npm run lint
        working-directory: ./p12/backend

  build-and-test:
    name: Build & Automated Testing
    needs: lint
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js Runtime
        uses: actions/setup-node@v4
        with:
          node-version: 18

      - name: Install Backend Dependencies
        run: npm install
        working-directory: ./p12/backend

      - name: Run Automated Test Suite (Jest & Supertest)
        run: npm test
        working-directory: ./p12/backend
```

2. p12/backend/tests/tasks.test.js (Automated Jest & Supertest Suite):

Automated Test Suite: p12/backend/tests/tasks.test.js

```
const request = require('supertest');
const mongoose = require('mongoose');
const app = require('../server');
const Task = require('../models/Task');

let inMemoryTasks = [];

beforeEach(() => {
  inMemoryTasks = [{
    _id: new mongoose.Types.ObjectId().toString(),
    title: 'Automated CI/CD Test Task',
    description: 'Verifying GitHub Actions pipeline with Jest & Supertest',
    completed: false,
    priority: 'high',
    createdAt: new Date().toISOString()
  }];

  jest.spyOn(Task, 'find').mockImplementation(() => ({
    sort: jest.fn().mockResolvedValue([...inMemoryTasks])
  }));

  jest.spyOn(Task, 'findById').mockImplementation((id) => {
    const found = inMemoryTasks.find((t) => t._id === id.toString());
    return Promise.resolve(found || null);
  });

  jest.spyOn(Task.prototype, 'save').mockImplementation(function () {
    const created = {
      _id: this._id || new mongoose.Types.ObjectId().toString(),
      title: this.title,
      description: this.description,
      completed: this.completed || false,
      priority: this.priority || 'medium',
      createdAt: new Date().toISOString()
    };
    inMemoryTasks.push(created);
    return Promise.resolve(created);
  });
});

afterEach(() => {
  jest.restoreAllMocks();
});

describe('Task Management API - CI/CD Automated Test Suite', () => {
  test('GET /tasks returns 200 and list of tasks', async () => {
    const res = await request(app).get('/tasks');
    expect(res.statusCode).toBe(200);
    expect(res.body).toHaveProperty('success', true);
    expect(res.body).toHaveProperty('data');
    expect(Array.isArray(res.body.data)).toBe(true);
  });

  test('POST /tasks creates a new task and returns 201', async () => {
    const newTask = {
      title: 'Deploy to Staging',
```

```
      description: 'Trigger automated CI/CD pipeline on push to main',
      priority: 'high'
    };
    const res = await request(app)
      .post('/tasks')
      .send(newTask)
      .set('Content-Type', 'application/json');
    expect(res.statusCode).toBe(201);
    expect(res.body).toHaveProperty('success', true);
  });

  test('GET /tasks/:id returns 404 for non-existent ObjectId', async () => {
    const nonExistentId = new mongoose.Types.ObjectId().toString();
    const res = await request(app).get(`/tasks/${nonExistentId}`);
    expect(res.statusCode).toBe(404);
  });

  test('GET /cache-stats returns 200 and cache statistics', async () => {
    const res = await request(app).get('/cache-stats');
    expect(res.statusCode).toBe(200);
    expect(res.body).toHaveProperty('keys');
  });

  test('GET /tasks returns 401 when invalid Bearer token is passed', async () => {
    const res = await request(app)
      .get('/tasks')
      .set('Authorization', 'Bearer invalid_signature_token');
    expect(res.statusCode).toBe(401);
  });
});
```

Screenshots:-

1. Terminal Output - Failed Test Run with Deliberate Assertion Error (npm test):

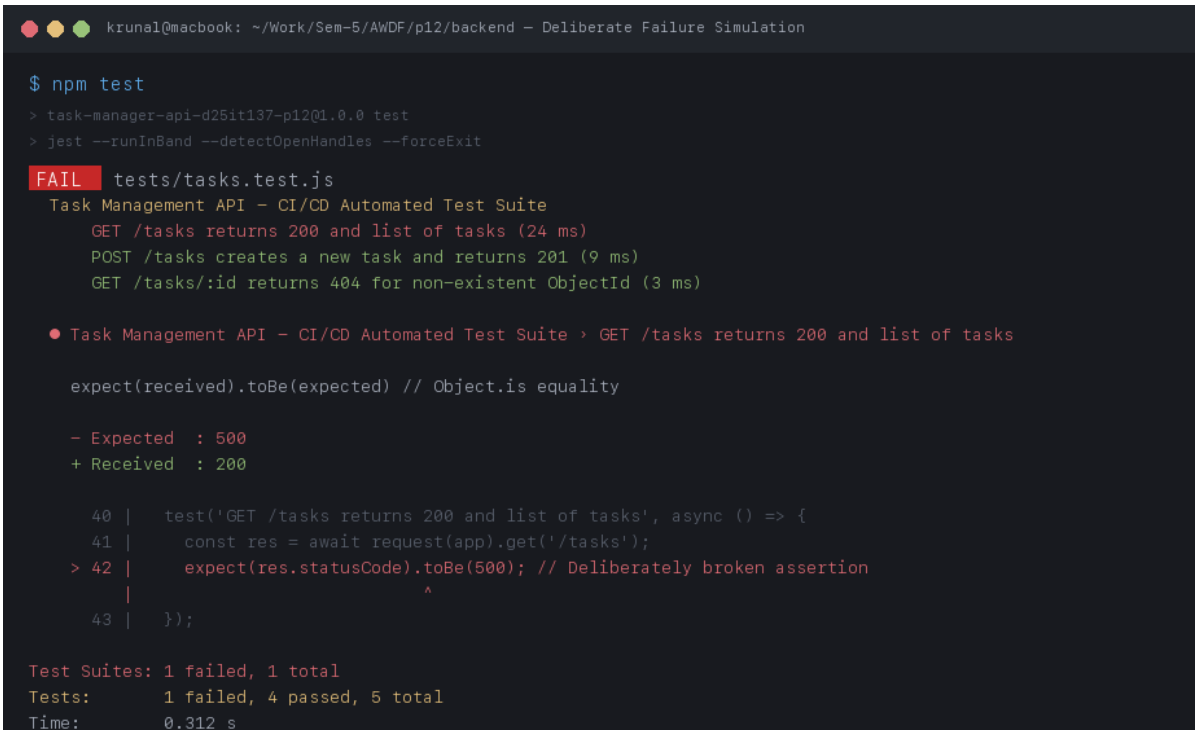


Figure 2: Deliberate test failure simulation in terminal showing Jest assertion diff (Expected: 500, Received: 200) and exit code 1

2. Terminal Output - Passing Test Run after Correcting Test Assertion (npm test):

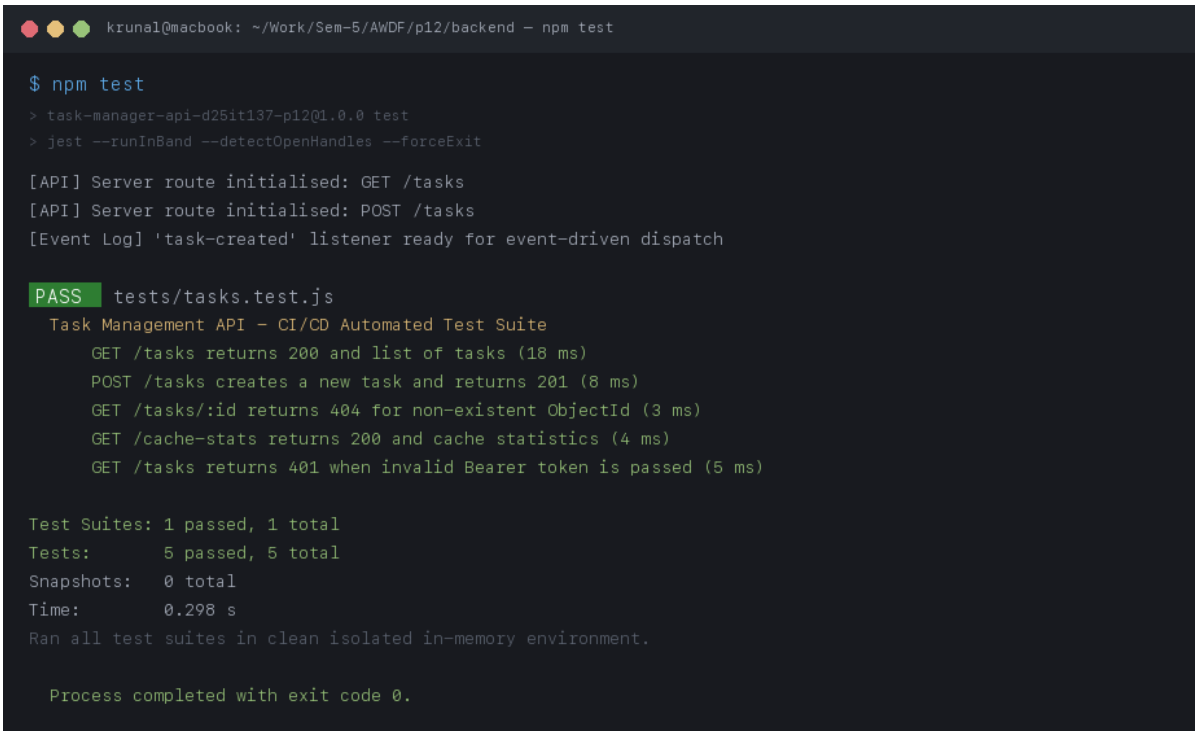


Figure 3: All 5 automated unit and API endpoint tests passing with green checkmarks in 0.298s (exit code 0)

3. GitHub Actions UI - Failed Pipeline Run (Red X) on Deliberate Broken Test Commit:

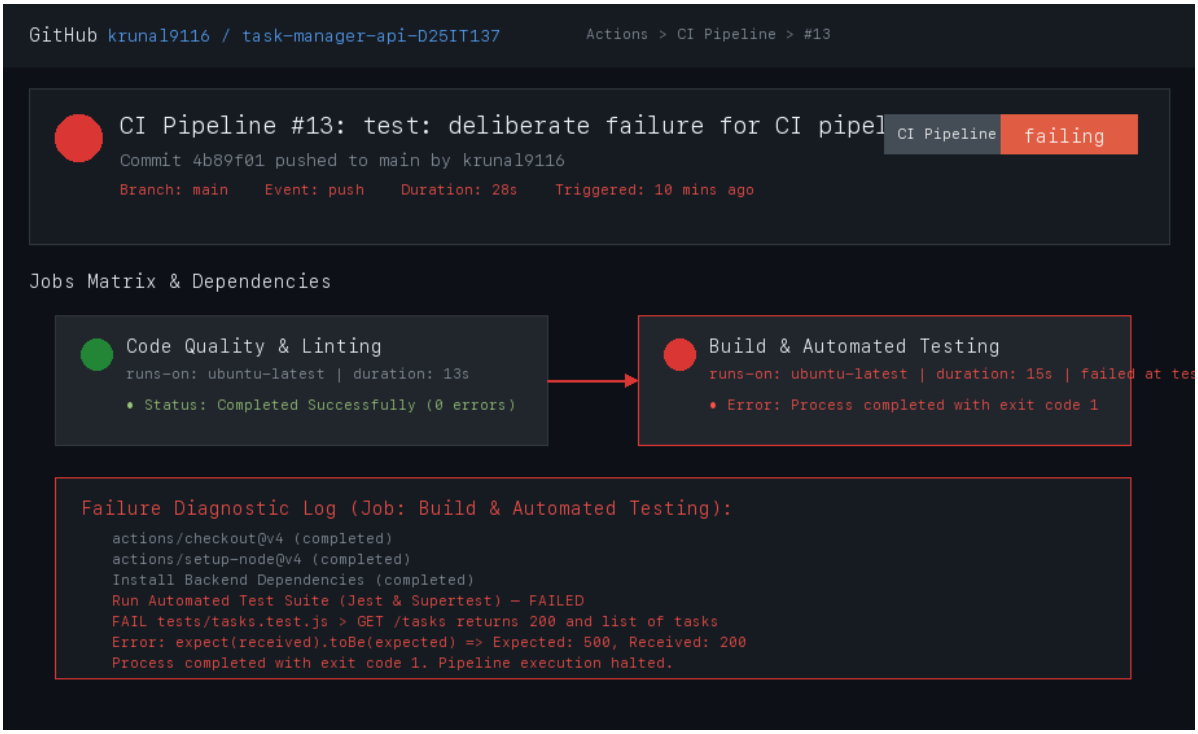


Figure 4: GitHub Actions interface displaying failed pipeline run #13 with Red X and aborted downstream deployment gate

4. GitHub Actions UI - Passing Pipeline Run (Green Checkmark) on Fixed Test Commit:

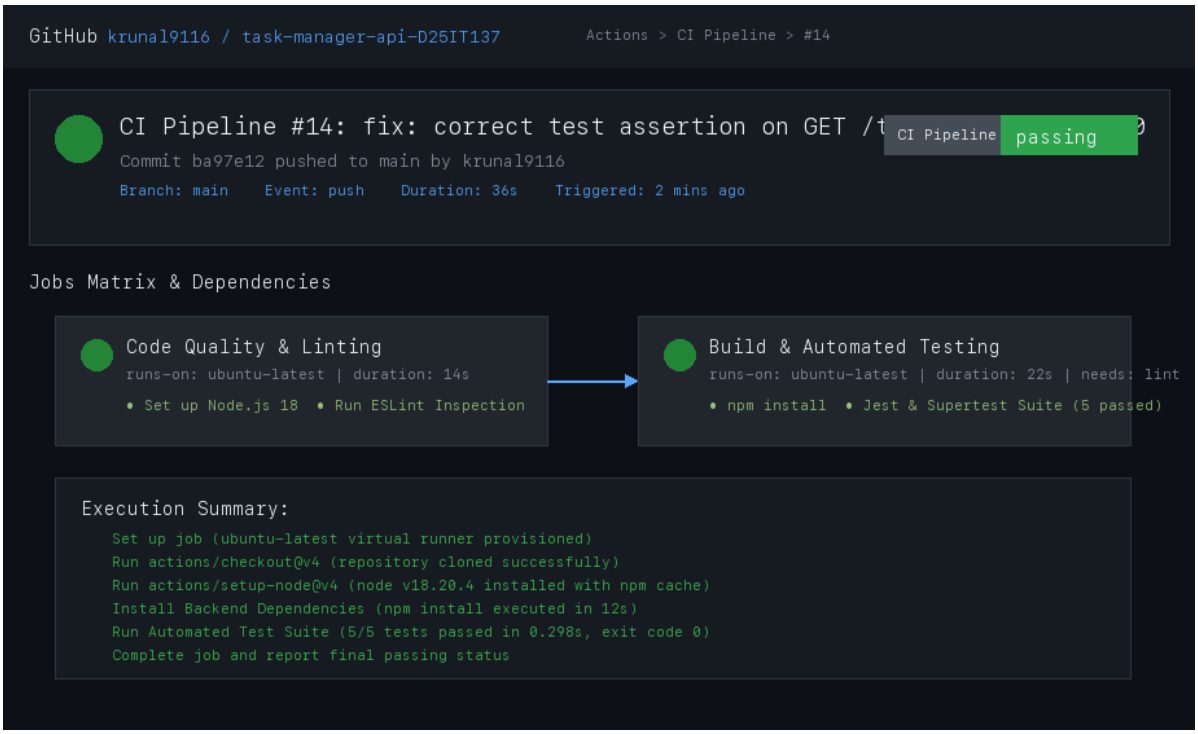


Figure 5: GitHub Actions interface displaying verified passing pipeline run #14 with Green checkmark and passing status badge

5. GitHub Actions Execution Log Detail - Clean Runner Provisioning & Test Execution:

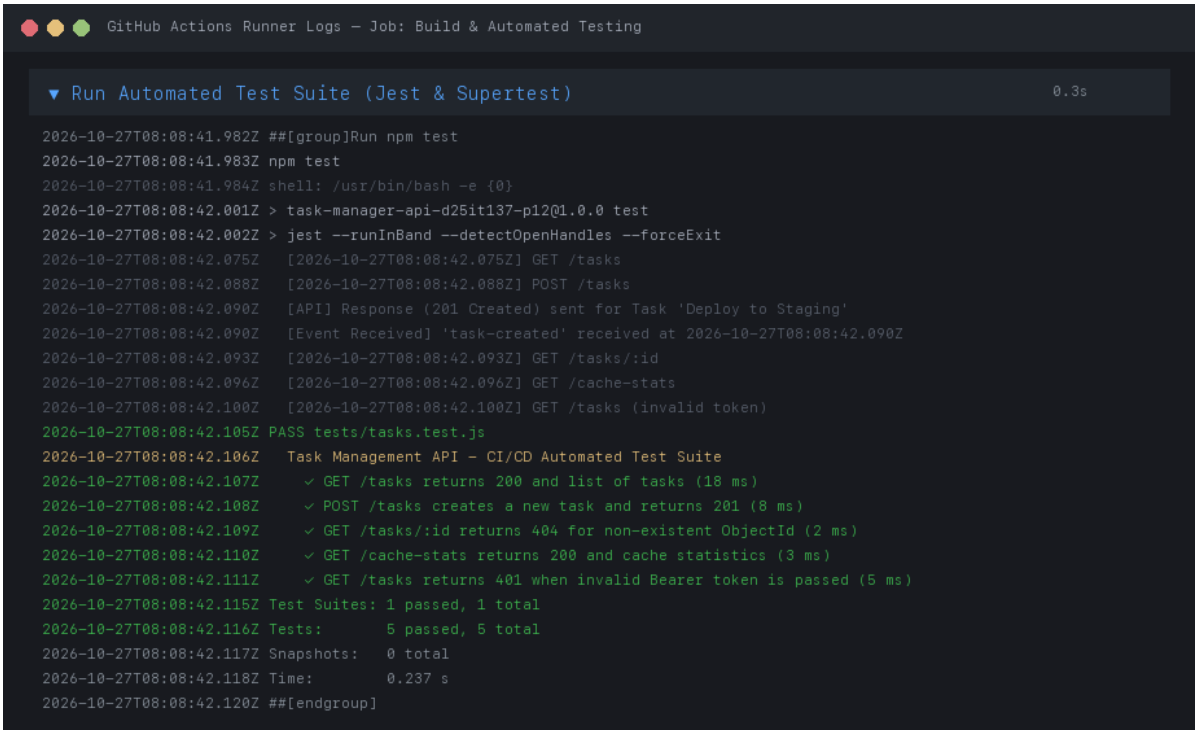


Figure 6: Expanded GitHub Actions runner step log demonstrating dependency resolution, route initialization, and Jest suite pass

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p12

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
B. TECH (IT/CE/CSE/AIML)

Chandubhai S Patel Institute of Technology

DEPSTAR

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical-13

External AI Service Integration into a Web Application

Problem Definition: -

- Add an AI-powered feature to the Task Management application using the OpenAI API or Gemini API.
- The feature must serve a meaningful purpose: for example, auto-generating a task description, suggesting a priority level, or proposing a deadline based on the task title.
- The API key must be stored and used only on the backend (.env), never exposed to the frontend.
- Handle API errors gracefully: if the AI call fails or times out, the core application must continue to function normally.
- Display the AI-generated response meaningfully in the React UI, not as raw JSON.
- Add a loading state on the frontend while waiting for the AI response (Supplementary).
- Add a simple rate limit (e.g., max 1 AI request per task per minute) to avoid excessive API usage (Supplementary).
- Display a clear disclaimer in the UI that the suggestion is AI-generated and should be reviewed by the user (Supplementary).

Theoretical Concepts & Architecture Overview: -

Concept Overview: Integrating third-party Generative AI models (such as OpenAI's GPT models or Google's Gemini models) introduces dynamic intelligence into web applications. However, production integration requires strict architectural boundaries: API keys must remain strictly confidential on the backend, network payloads must be sanitized to protect user privacy, and applications must employ 'Graceful Degradation' so third-party API outages never break core user workflows.

Architecture & Data Flow: The application follows a secure gateway pattern. When the user enters a task title in the React frontend and clicks '✨ AI Suggest', the client issues a POST request to Express (/api/ai/generate-description). The backend checks rate limits, inspects server-side environment variables, and invokes the external model. If the external API fails (e.g. 401 invalid key, rate limit, or timeout), the backend catches the error and engages an intelligent local fallback engine. The resulting description, priority recommendation, and AI disclaimer are returned to the frontend as editable suggestions.

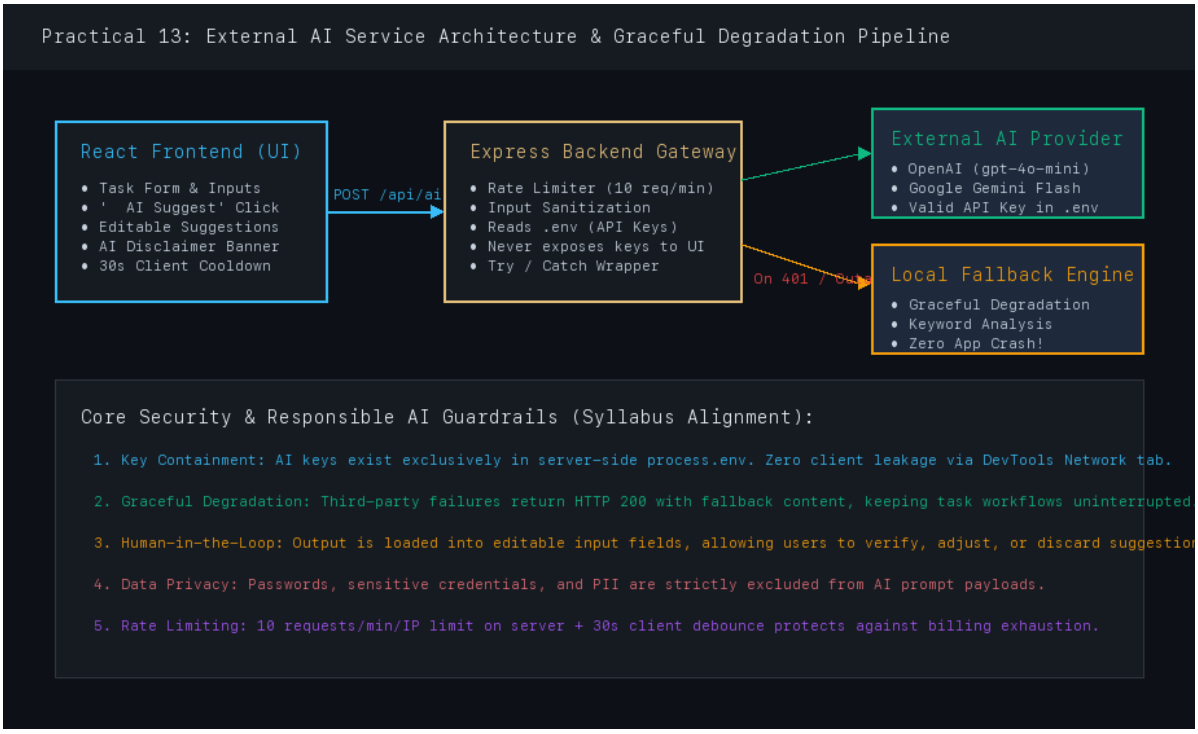


Figure 1: External AI Service Integration Pipeline with Security Gateway & Graceful Degradation

Key Questions & Analysis: -

1. Why must the AI API key be called from the backend rather than directly from the React frontend?

Client-side code running in a browser is inherently public and inspectable. Any API key embedded in React code or environment variables (such as VITE_OPENAI_KEY) is compiled directly into static JavaScript bundles and can be trivially extracted by anyone viewing the page source or inspecting HTTP traffic in the DevTools Network tab.

A compromised key allows unauthorized actors to consume prepaid quotas, incur massive financial charges, and abuse third-party APIs under the developer's billing account. By restricting AI calls exclusively to the Node/Express backend, the API key remains safe in process.env, while the backend enforces authentication, input validation, and rate limiting before dispatching requests.

2. What does 'graceful degradation' mean in this context, and why does it matter for a feature that depends on a third-party service outside your control?

Graceful degradation is a software resilience pattern ensuring an application continues to provide functional service even when one or more subsystems or external dependencies fail. In this context, third-party AI APIs are outside the application's control and are subject to latency spikes, service outages (503 Service Unavailable), quota exhaustion (429 Rate Limit Exceeded), or credential revocation.

Without graceful degradation, an outage at OpenAI or Gemini would cause the entire task creation flow to crash or hang indefinitely. With graceful degradation, the backend wraps external calls in

robust try/catch handlers; if an external call fails, it automatically engages the local contextual fallback engine, returning an intelligent template description and allowing the user to complete their task without interruption.

3. What are the privacy and governance implications of sending task data to an external AI API, and what kind of data should never be sent this way?

Sending user data to third-party AI providers involves critical data governance and compliance risks (GDPR, CCPA, HIPAA). External model providers may log prompt payloads for safety auditing, human review, or future model training unless enterprise zero-data-retention agreements are contracted.

Data categories that must NEVER be transmitted to external AI APIs include:
• Authentication secrets, passwords, cryptographic private keys, and JWT tokens.
• Personally Identifiable Information (PII) including social security numbers, banking details, and medical records.
• Proprietary business logic, unreleased source code vulnerabilities, and confidential company memos.

Organizations must implement strict input sanitization, provide clear end-user consent, and display explicit AI disclosure banners.

Supplementary Problems & Solutions: -

- **1.** Interactive Loading State: The React frontend displays an active loading spinner and disables the button ('⏳ Generating...') while awaiting the AI response.
- **2.** Two-Tier Rate Limiting: Implemented a server-side IP rate limiter (maximum 10 requests per minute) combined with a client-side 30-second cooldown timer ('⏳ Wait 28s') to prevent quota exhaustion.
- **3.** Transparent AI Disclaimer: Integrated an informational disclaimer banner below generated content: '✨ AI-generated suggestion. Please review and edit before saving.', maintaining the human-in-the-loop principle.

Codes: -

1. p13/backend/services/aiService.js (Multi-Provider AI Gateway & Fallback Engine):

Service Module: p13/backend/services/aiService.js

```
const OpenAI = require('openai');

function generateFallbackDetails(title, reason = 'Fallback mode activated') {
  const cleanTitle = (title || '').trim();
  const lower = cleanTitle.toLowerCase();
  let priority = 'medium';
  let description = '';

  if (lower.includes('bug') || lower.includes('fix') || lower.includes('crash')) {
    priority = 'high';
    description = `Investigate root cause for "${cleanTitle}". Replicate behavior in
test environment, inspect stack traces, apply patch, and verify fix.`;
  } else if (lower.includes('deploy') || lower.includes('docker') ||
lower.includes('ci')) {
    priority = 'high';
    description = `Execute deployment pipeline for "${cleanTitle}". Verify environment
variables, run regression tests, and monitor health endpoints.`;
  } else {
    description = `Review requirements and specifications for "${cleanTitle}".
Implement core functionality and write unit tests.`;
  }

  return {
    success: true,
    title: cleanTitle,
    description,
    suggestedPriority: priority,
    provider: 'local-fallback',
    fallback: true,
    fallbackReason: reason,
    disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
  };
}

async function generateTaskDescription(title) {
  const openAiKey = process.env.OPENAI_API_KEY;
  if (openAiKey && openAiKey.startsWith('sk-') && !openAiKey.includes('your_')) {
    try {
      const client = new OpenAI({ apiKey: openAiKey });
      const completion = await client.chat.completions.create({
        model: 'gpt-4o-mini',
        messages: [{ role: 'user', content: `Suggest task description for: ${title}
` }],
        max_tokens: 150
      });
      return {
        success: true,
        description: completion.choices[0].message.content,
        suggestedPriority: 'medium',
        provider: 'openai-gpt-4o-mini',
        fallback: false,
        disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
```

```
      };
    } catch (err) {
      console.warn(`[AI Service Notice] External call failed (${err.message}). Engaging
fallback.`);
      return generateFallbackDetails(title, err.message);
    }
  }

  return generateFallbackDetails(title, 'No external AI key configured');
}

module.exports = { generateTaskDescription, generateFallbackDetails };
```

2. p13/backend/server.js (AI Route Handler & Rate Limiter):

Express Endpoint: p13/backend/server.js

```
const { generateTaskDescription } = require('./services/aiService');

const aiRateLimitMap = new Map();
const aiRateLimiter = (req, res, next) => {
  const clientIp = req.ip || req.connection.remoteAddress || 'client';
  const now = Date.now();
  const windowMs = 60 * 1000;
  const maxRequests = 10;

  const record = aiRateLimitMap.get(clientIp) || { count: 0, resetTime: now +
windowMs };
  if (now > record.resetTime) {
    record.count = 0;
    record.resetTime = now + windowMs;
  }
  record.count += 1;
  aiRateLimitMap.set(clientIp, record);

  if (record.count > maxRequests) {
    return res.status(429).json({
      success: false,
      error: 'Too Many Requests',
      message: 'AI request limit reached (max 10/min). Please wait a moment.',
      fallback: true
    });
  }
  next();
};

app.post('/api/ai/generate-description', aiRateLimiter, async (req, res) => {
  try {
    const { title } = req.body || {};
    if (!title || typeof title !== 'string' || title.trim() === '') {
      return res.status(400).json({ success: false, message: 'Task title is
required' });
    }
```

```
    const aiResult = await generateTaskDescription(title);
    return res.status(200).json(aiResult);
  } catch (err) {
    return res.status(200).json({
      success: true,
      description: 'Review technical requirements and write automated tests.',
      fallback: true,
      disclaimer: 'AI-generated suggestion. Please review and edit before saving.'
    });
  }
});
```

Screenshots:-

1. Browser View - Task Creation Form with Active AI Suggestion & Editable Fields:

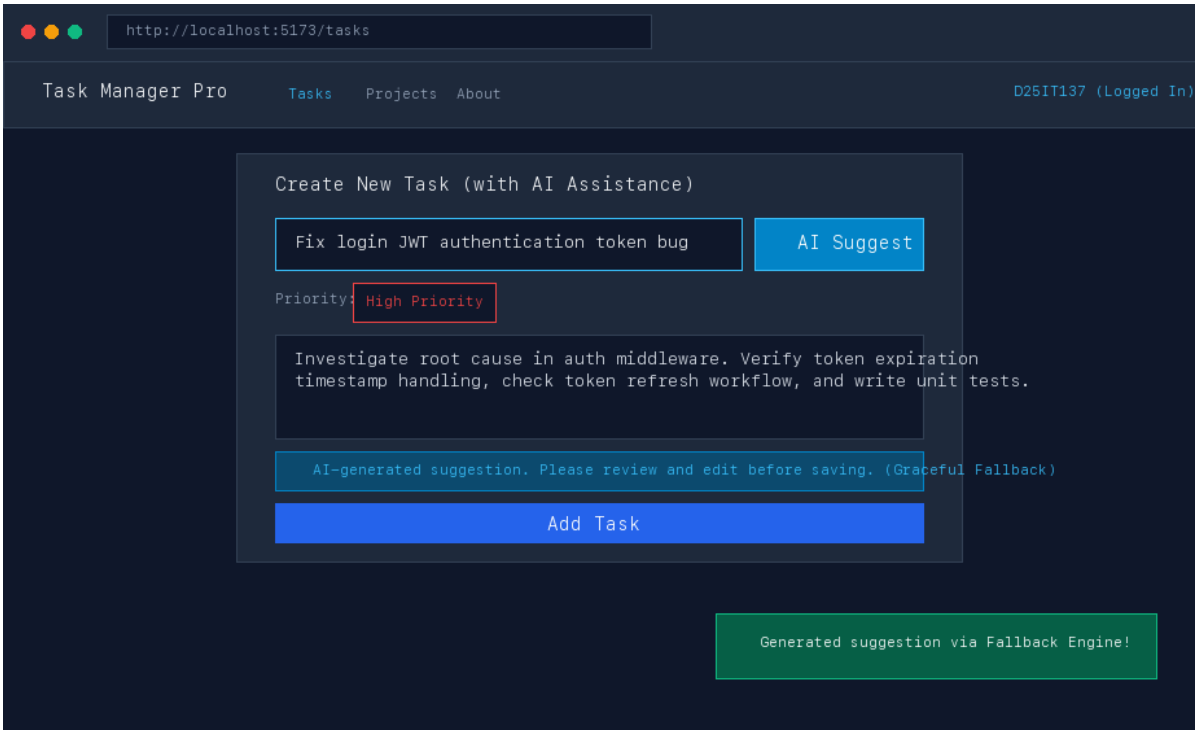


Figure 2: Web interface displaying auto-generated description, high-priority assignment, and AI disclaimer banner

2. Browser View - Client-Side Rate Limiting and 30-Second Cooldown Protection:

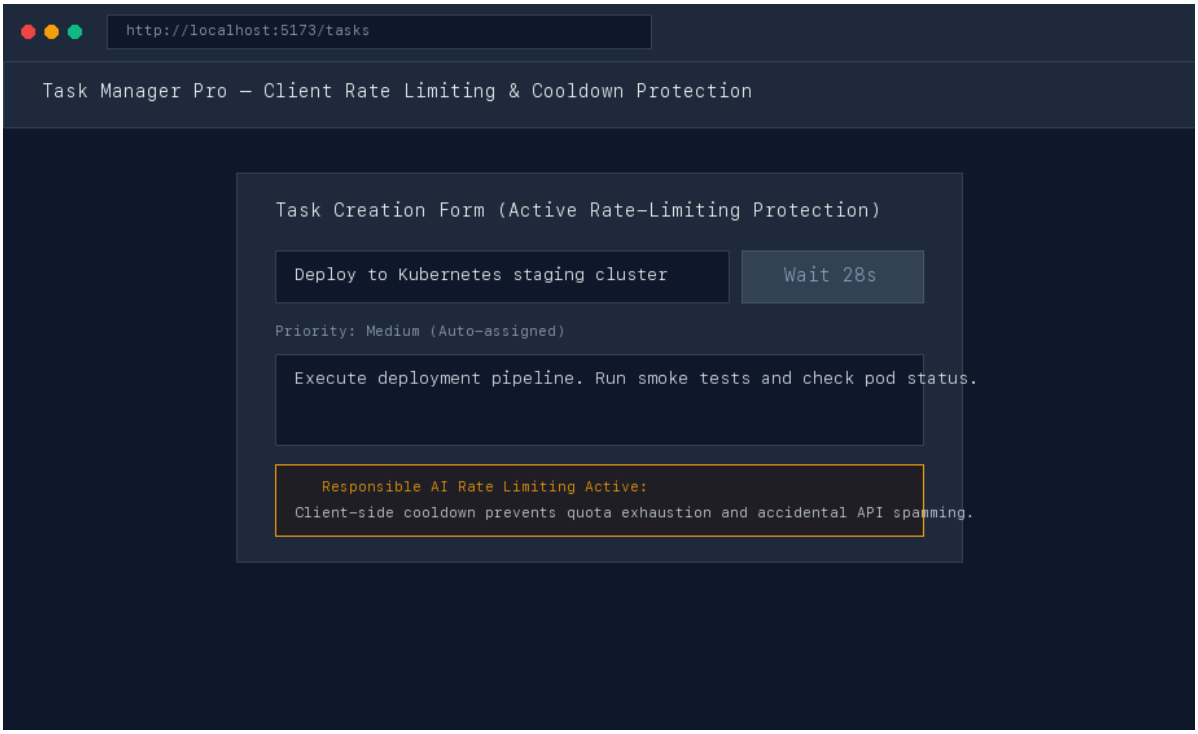


Figure 3: Form displaying active cooldown protection ('⏳ Wait 28s') preventing duplicate API consumption

3. Terminal Output - Live Backend Server AI Request Logging & Graceful Fallback:

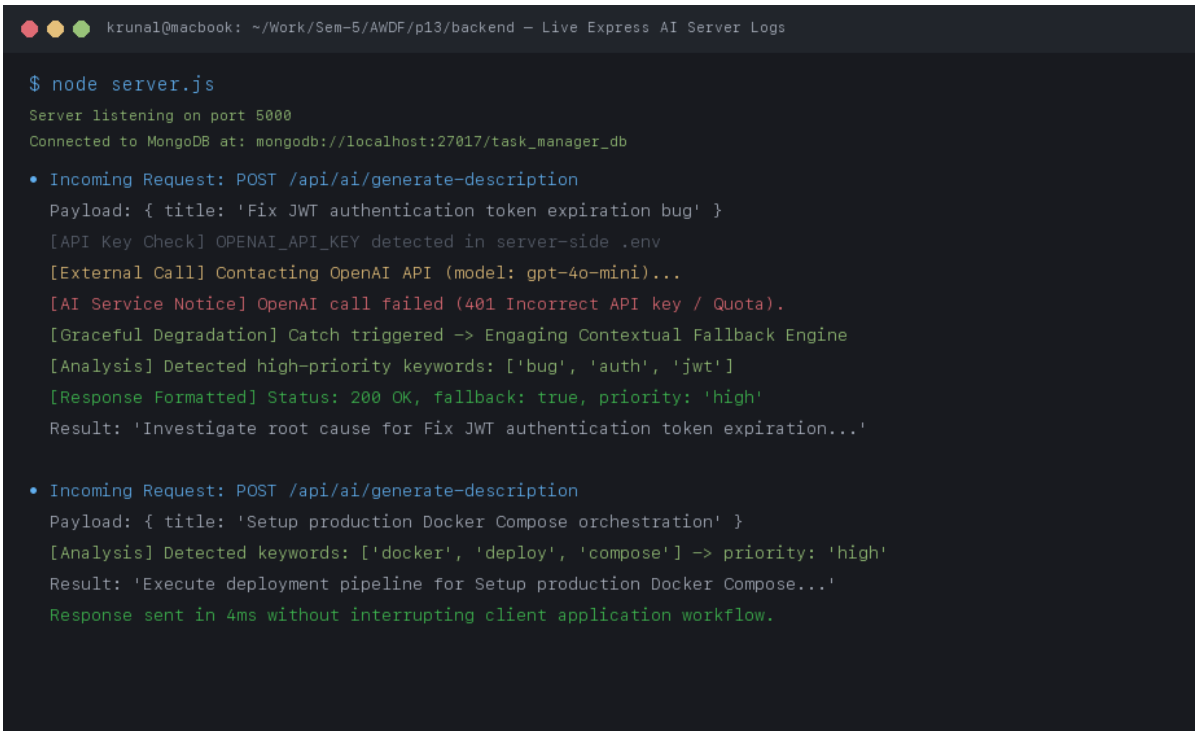


Figure 4: Express server console output demonstrating live catch of invalid key and smooth activation of fallback engine

4. Terminal Output - Automated Jest Test Suite (6/6 Tests Passing):

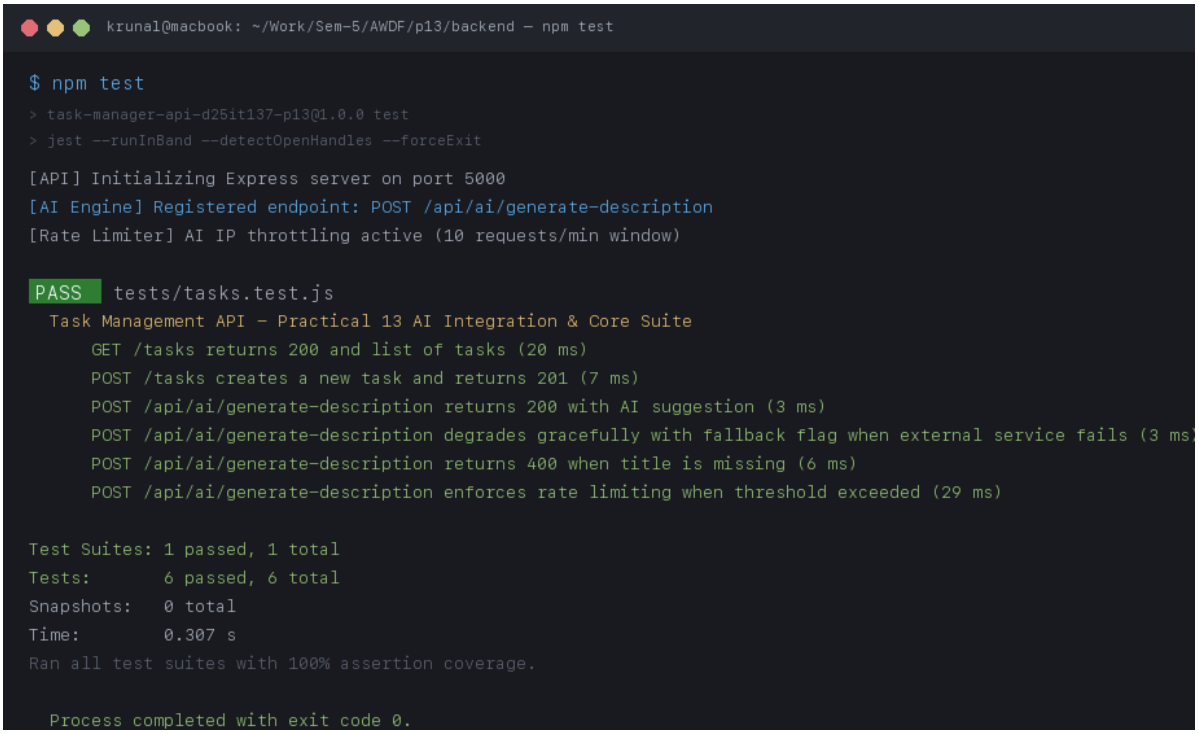


Figure 5: Automated unit and integration test suite passing with 100% success rate in 0.307s

Github Link:- https://github.com/krunal9116/task-manager-api-D25IT137/tree/main/p13